

Python
实战-从菜鸟到大牛的进阶之路

前言

Python是一种解释型、面向对象、动态数据类型的高级程序设计语言，现在它已经成为最受欢迎的程序设计语言之一。本专题收录了Python编程实战教程，分享给大家。

适用人群

Python 进阶学习者、Web 开发程序员、运维人员、有志于从事互联网行业以及各领域应用 Python 的人群。

学习前提

学习本教程之前强烈建议你先熟悉一下，Python基础知识。

更新日期	更新内容
2015-07-31	Python 实战-从菜鸟到大牛的进阶之路

目录

前言 1

第 1 章 Python 处理 cassandra 升级后的回滚脚本 5

第 2 章 多套方案来提高 python web 框架的并发处理能力 7

Python 常见部署方法有： 8

下面开始介绍下 我一般用的方法： 12

第 3 章 python 写报警程序中的声音实现 winsound 17

第 4 章 一个脚本讲述 python 语言的基础规范，适合初学者 20

一、python 脚本的规范： 22

二、python 的缩进 23

三、每一个功能对应一个函数 24

四、系统命令的引用 25

五、异常处理 26

第 5 章 python 计算文件的行数和读取某一行内容的实现方法 28

一、计算文件的行数 29

二、读取文件某一行的内容（测试过 1 G 大小的文件，效率还可以） 30

三、用 linecache 读取文件内容（测试过 1 G 大小的文件，效率还可以） 31

第 6 章 python 中用 string.maketrans 和 translate 巧妙替换字符串 32

第 7 章 python linecache 模块读取文件用法详解 34

第 8 章 python 调用 zabbix 的 api 接口添加主机、查询组、主机、模板 37

第 9 章 python+Django 实现 Nagios 自动化添加监控项目 43

第 10 章 通过 python 和 websocket 构建实时通信系统[扩展 saltstack 监控] 58

简单了解下接口方法和属性： 62

第 11 章 关于 B+tree (附 python 模拟代码) 67

第 12 章　Python 编写的 socket 服务器和客户端 77
第 13 章　python 之 MySQLdb 库的使用 80
第 14 章　python 监控文件或目录变化 85
第 15 章　Mongodb 千万级数据在 python 下的综合压力测试及应用探讨 90
第 16 章　通过 memcached 实现领号排队功能及 python 队列实例 106
前言： 107
第 17 章　python 之利用 PIL 库实现页面的图片验证码及缩略图 114
前言： 107
第 18 章　如何将 Mac OS X10.9 下的 Python2.7 升级到最新的 Python3.3 121
第 19 章　使用 python 构建基于 hadoop 的 mapreduce 日志分析平台 124
第 20 章　报警监控平台扩展功能 url 回调的设计及应用 [python 语言] 133
前言： 107
怎么个灵活法： 135
关于触发式的处理： 136
第 21 章　服务端 socket 开发之多线程和 gevent 框架并发测试[python 语言] 139
第 22 章　利用 pypy 提高 python 脚本的执行速度及测试性能 150
啥是 pypy 151
第 23 章　python 实现 select 和 epoll 模型 socket 网络编程 157
第 24 章　对 Python-memcache 分布式散列和调用的实现 163
第 25 章　Parallel Python 实现程序的并行多 cpu 多核利用【pp 模块】 167
为啥要这个模块： 168
pp 模块的简介 169
下面是看起来很高端的功能介绍！ 170
多线程在并发和并行环境中的不同作用 171
第 26 章　关于 python 调用 zabbix api 接口的自动化实例 [结合 saltstack] 177

前言：. 107
啥时候用? 182
第 27 章 Python 批量更新 nginx 配置文件 183
第 28 章 Python 通过 amqp 消息队列协议中的 Qpid 实现数据通信 187
简介：. 188
几个概念 189
总结：. 196
第 29 章 python simplejson 模块浅谈 197
第 30 章 python Howto 之 logging 模块 202
第 31 章 Python FAQ3-python 中 的原始(raw)字符串 211

1

Python 处理 cassandra 升级后的回滚脚本

前几天开发把分布式存储服务器 cassandra 升级了，担心升级不成功，所以写了一个升级回滚失败的脚本

环境说明：

升级后的目录结构为：
Cassandra 数据文件放在 /opt/cassandra/data/ 下
data 目录下有很多 keyspace 的目录：如：system 目录，Keyspcace 目录下有 coumlfailmly 目录，

如:/opt/cassandra/data/system/peers/snapshots/1370569934254 此下面是所有的数据文件

如：system-peers-ib-10-Summary.db
system-peers-fsdfsfsfd-10-Summary.db

现要把所有 Keyspace 目录下的所有 db 文件挪到 /opt/cassandra/data/system下，（ -ib- 文件除外 ）

如： /opt/cassandra/data/system/peers/snapshots/1370569934254/system-peers-fsdfsfsfd-10-Summary.db=======》/opt/cassandra/data/system/peers-fsdfsfsfd-10-Summary.db (注意还得重命令，把文件名的 keyspace 部份去掉)

上脚本：

```
#!/usr/bin/python2.6
#coding=utf8
import os
import os.path
import re
def cassandra_file(url_dir):
        if not os.path.isdir(url_dir):
                return False
        elif os.path.isdir(url_dir):
                for dirpath,dirname,filenames in os.walk(url_dir):
                        for file in filenames:
                                filepath = os.path.join(dirpath,file)
                                aa = os.path.splitext(file)
                                bb = "".join(aa)
                                if "-ib-" not in str(bb):  #排除某些文件
                                        print "要处理文件名为:" + bb                    # 获取所有文件名
                                        cc = bb.split('-')
                                        dd = '-'.join(cc[1:])
                                        print "重命名后的文件名为:" + dd        #获取最后的文件名名
                                        old_url= os.path.split(filepath)[0]
                                        print "旧目录路径为:" + old_url
                                        old_file_url= os.path.join(old_url,dd)
                                        print "旧文件路径为:" + old_file_url
                                        os.rename(filepath,old_file_url)
                                        if len(url_dir.split('/')) == 3:
                                                dddd=filepath.split("/")[1:4]    #
                                                dsc_dir= "/" + "/".join(dddd)
                                                print "需要挪到这个路径下:"   dsc_dir # 获取要挪的目的路径
                                                eeee = os.path.join(dsc_dir,dd)
                                                print "新文件路径为:" + eeee
                                                os.rename(old_file_url,os.path.join(dsc_dir,dd))
                                        elif len(url_dir.split('/')) == 4:
                                                dddd=filepath.split("/")[1:5]    #
                                                dsc_dir= "/" + "/".join(dddd)
                                                print "需要挪到这个路径下:" + dsc_dir # 获取要挪的目的路径
                                                eeee = os.path.join(dsc_dir,dd)
                                                print "新文件路径为:" + eeee
                                                os.rename(old_file_url,os.path.join(dsc_dir,dd))
cassandra_file("/cassandra/data")
```

51CTO.com 技术博客 Blog

图片 1.1 pic

脚本支持： /cassandra/data 和 /opt/cassasnra/data 这个路径下的目录。

2

多套方案来提高 python web 框架的并发处理能力

Python 常见部署方法有：

1. fcgi： 用 spawn-fcgi 或者框架自带的工具对各个 project 分别生成监听进程，然后和 http 服务互动
2. wsgi： 利用 http 服务的 mod_wsgi 模块来跑各个 project(Web 应用程序或框架简单而通用的 Web 服务器 之间的接口)。
3. uWSGI 是一款像 php-cgi 一样监听同一端口，进行统一管理和负载平衡的工具，uWSGI，既不用 wsgi 协议也不用 fcgi 协议，而是自创了一个 uwsgi 的协议，据说该协议大约是 fcgi 协议的 10 倍那么快。

其实 WSGI 是分成 server 和 framework (即 application) 两部分 (当然还有 middleware)。严格说 WSGI 只是一个协议, 规范 server 和 framework 之间连接的接口。

WSGI server 把服务器功能以 WSGI 接口暴露出来。比如 mod_wsgi 是一种 server, 把 apache 的功能以 WSGI 接口的形式提供出来。

1. WSGI framework 就是我们经常提到的 Django 这种框架。不过需要注意的是, 很少有单纯的 WSGI framework , 基于 WSGI 的框架往往都自带 WSGI server。比如 Django、CherryPy 都自带 WSGI server 主要是测试用途, 发布时则使用生产环境的 WSGI server。而有些 WSGI 下的框架比如 pylons、bfg 等, 自己不实现 WSGI server。使用 paste 作为 WSGI server。
2. Paste 是流行的 WSGI server, 带有很多中间件。还有 flup 也是一个提供中间件的库。
3. 搞清除 WSGI server 和 application, 中间件自然就清楚了。除了 session、cache 之类的应用, 前段时间看到一个 bfg 下的中间件专门用于给网站换肤的 (skin) 。中间件可以想到的用法还很多。
4. 这里再补充一下, 像 django 这样的框架如何以 fastcgi 的方式跑在 apache 上的。这要用到 flup.fcgi 或者 fastcgi.py (eurasia 中也设计了一个 fastcgi.py 的实现) 这些工具, 它们就是把 fastcgi 协议转换成 WSGI 接口 (把 fastcgi 变成一个 WSGI server) 供框架接入。整个架构是这样的: django -> fcgi2wsgiserver -> mod_fcgi -> apache 。
5. 虽然我不是 WSGI 的粉丝, 但是不可否认 WSGI 对 python web 的意义重大。有意自己设计 web 框架, 又不想做 socket 层和 http 报文解析的同学, 可以从 WSGI 开始设计自己的框架。在 python 圈子里有个共识, 自己随手搞个 web 框架跟喝口水一样自然, 非常方便。或许每个 python 玩家都会经历一个倒腾框架的

uWSGI 的主要特点如下：

- 超快的性能。
- 低内存占用（实测为 apache2 的 mod_wsgi 的一半左右）。
- 多app管理。

- 详尽的日志功能（可以用来分析 app 性能和瓶颈）。
- 高度可定制（内存大小限制，服务一定次数后重启等）。

uwsgi 的官方文档：

http://projects.unbit.it/uwsgi/wiki/Doc

```
nginx.conf

location / {
  include uwsgi_params
  uwsgi_pass 127.0.0.1:9090
}
```

启动 app

```
uwsgi -s :9090 -w myapp
```

uwsgi 的调优参数~

```
uwsgi 的参数
以上是单个 project 的最简单化部署，uwsgi 还是有很多令人称赞的功能的，例如：
并发 4 个线程：
  uwsgi -s :9090 -w myapp -p 4
主控制线程 +4 个线程：
  uwsgi -s :9090 -w myapp -M -p 4
执行超过 30 秒的 client 直接放弃：
  uwsgi -s :9090 -w myapp -M -p 4 -t 30
限制内存空间 128M：
  uwsgi -s :9090 -w myapp -M -p 4 -t 30 --limit-as 128
服务超过 10000 个 req 自动 respawn：
  uwsgi -s :9090 -w myapp -M -p 4 -t 30 --limit-as 128 -R 10000
后台运行等：
  uwsgi -s :9090 -w myapp -M -p 4 -t 30 --limit-as 128 -R 10000 -d uwsgi.log
```

为了让多个站点共享一个 uwsgi 服务，必须把 uwsgi 运行成虚拟站点：去掉“-w myapp”加上”–vhost”：

uwsgi -s :9090 -M -p 4 -t 30 --limit-as 128 -R 10000 -d uwsgi.log --vhost

然后必须配置 virtualenv，virtualenv 是 Python 的一个很有用的虚拟环境工具，这样安装：

最后配置 nginx，注意每个站点必须单独占用一个 server，同一 server 不同 location 定向到不同的应用不知为何总是失败，估计也 算是一个 bug。

```
server {
   listen      80;
   server_name  app1.mydomain.com;
   location / {
          include uwsgi_params;
          uwsgi_pass 127.0.0.1:9090;
          uwsgi_param UWSGI_PYHOME /var/www/myenv;
          uwsgi_param UWSGI_SCRIPT myapp1;
          uwsgi_param UWSGI_CHDIR /var/www/myappdir1;
    }
}
server {
   listen      80;
   server_name  app2.mydomain.com;
   location / {
          include uwsgi_params;
          uwsgi_pass 127.0.0.1:9090;
          uwsgi_param UWSGI_PYHOME /var/www/myenv;
          uwsgi_param UWSGI_SCRIPT myapp2;
          uwsgi_param UWSGI_CHDIR /var/www/myappdir2;
   }
}
```

这样，重启 nginx 服务，两个站点就可以共用一个 uwsgi 服务了。

再来搞下 fastcgi 的方式

```
location / {
       fastcgi_param REQUEST_METHOD $request_method;
       fastcgi_param QUERY_STRING $query_string;
       fastcgi_param CONTENT_TYPE $content_type;
       fastcgi_param CONTENT_LENGTH $content_length;
       fastcgi_param GATEWAY_INTERFACE CGI/1.1;
       fastcgi_param SERVER_SOFTWARE nginx/$nginx_version;
       fastcgi_param REMOTE_ADDR $remote_addr;
       fastcgi_param REMOTE_PORT $remote_port;
       fastcgi_param SERVER_ADDR $server_addr;
       fastcgi_param SERVER_PORT $server_port;
       fastcgi_param SERVER_NAME $server_name;
       fastcgi_param SERVER_PROTOCOL $server_protocol;
       fastcgi_param SCRIPT_FILENAME $fastcgi_script_name;
       fastcgi_param PATH_INFO $fastcgi_script_name;
       fastcgi_pass 127.0.0.1:9002;
}
```

```
location /static/ {
        root /path/to/www;
        if (-f $request_filename) {
          rewrite ^/static/(.*)$  /static/$1 break;
        }
    }
```

启动一个 fastcgi 的进程

```
spawn-fcgi -d /path/to/www -f /path/to/www/index.py -a 127.0.0.1 -p 9002
```

用 web.py 写的一个小 demo 测试

```
 #!/usr/bin/env python

 # -*- coding: utf-8 -*-

import web
urls = ("/.*", "hello")
app = web.application(urls, globals())
class hello:
    def GET(self):
        return 'Hello, world!'
if __name__ == "__main__":
    web.wsgi.runwsgi = lambda func, addr=None: web.wsgi.runfcgi(func, addr)
    app.run()
```

启动 nginx

```
nginx
```

这样就 ok 了~

下面开始介绍下 我一般用的方法：



图片 2.1 pic

前端 nginx 用负责负载分发：

部署的时候采用了单 IP 多端口方式，服务器有 4 个核心，决定开 4 个端口对应，分别是 8885~8888，修改

```
upstream backend {
        server 127.0.0.1:8888;
        server 127.0.0.1:8887;
        server 127.0.0.1:8886;
        server 127.0.0.1:8885;
}
 server{
        listen  80;
        server_name message.test.com;
        keepalive_timeout 65;    #
        proxy_read_timeout 2000; #
        sendfile on;
        tcp_nopush on;
        tcp_nodelay on;
    location / {
        proxy_pass_header Server;
```

```
        proxy_set_header Host $http_host;
        proxy_redirect off;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Scheme $scheme;
        proxy_pass  http://backend;
        }
}
```

然后运行四个 python 程序，端口为咱们配置好的端口

我这里用 tornado 写了一个执行系统程序的例子：

```
import subprocess
import tornado.ioloop
import time
import fcntl
import functools
import os
class GenericSubprocess (object):
    def __init__ ( self, timeout=-1, **popen_args ):
        self.args = dict()
        self.args["stdout"] = subprocess.PIPE
        self.args["stderr"] = subprocess.PIPE
        self.args["close_fds"] = True
        self.args.update(popen_args)
        self.ioloop = None
        self.expiration = None
        self.pipe = None
        self.timeout = timeout
        self.streams = []
        self.has_timed_out = False
    def start(self):
        """Spawn the task.
        Throws RuntimeError if the task was already started."""
        if not self.pipe is None:
            raise RuntimeError("Cannot start task twice")
        self.ioloop = tornado.ioloop.IOLoop.instance()
        if self.timeout > 0:
            self.expiration = self.ioloop.add_timeout( time.time() + self.timeout, self.on_timeout )
        self.pipe = subprocess.Popen(**self.args)
        self.streams = [ (self.pipe.stdout.fileno(), []),
                         (self.pipe.stderr.fileno(), []) ]
        for fd, d in self.streams:
            flags = fcntl.fcntl(fd, fcntl.F_GETFL)| os.O_NDELAY
            fcntl.fcntl( fd, fcntl.F_SETFL, flags)
            self.ioloop.add_handler( fd,
```

```
                        self.stat,
                        self.ioloop.READ|self.ioloop.ERROR)
def on_timeout(self):
    self.has_timed_out = True
    self.cancel()
def cancel (self ) :
    """Cancel task execution
    Sends SIGKILL to the child process."""
    try:
        self.pipe.kill()
    except:
        pass
def stat( self, *args ):
    "'Check process completion and consume pending I/O data'"
    self.pipe.poll()
    if not self.pipe.returncode is None:
        "'cleanup handlers and timeouts'"
        if not self.expiration is None:
            self.ioloop.remove_timeout(self.expiration)
        for fd, dest in  self.streams:
            self.ioloop.remove_handler(fd)
        "'schedulle callback (first try to read all pending data)'"
        self.ioloop.add_callback(self.on_finish)
    for fd, dest in  self.streams:
        while True:
            try:
                data = os.read(fd, 4096)
                if len(data) == 0:
                    break
                dest.extend([data])
            except:
                break
@property
def stdout(self):
    return self.get_output(0)
@property
def stderr(self):
    return self.get_output(1)
@property
def status(self):
    return self.pipe.returncode
def get_output(self, index ):
    return "".join(self.streams[index][1])
def on_finish(self):
    raise NotImplemented()
```

```
class Subprocess (GenericSubprocess):
    """Create new instance
    Arguments:
        callback: method to be called after completion. This method should take 3 arguments: statuscode(int), stdout(str), std
        timeout: wall time allocated for the process to complete. After this expires Task.cancel is called. A negative timeout va
    The task is not started until start is called. The process will then be spawned using subprocess.Popen(**popen_args).
    """
    def __init__ ( self, callback, *args, **kwargs):
        """Create new instance
        Arguments:
            callback: method to be called after completion. This method should take 3 arguments: statuscode(int), stdout(str),
            timeout: wall time allocated for the process to complete. After this expires Task.cancel is called. A negative timeout
        The task is not started until start is called. The process will then be spawned using subprocess.Popen(**popen_args
        """
        self.callback = callback
        self.done_callback = False
        GenericSubprocess.__init__(self, *args, **kwargs)
    def on_finish(self):
        if not self.done_callback:
            self.done_callback = True
            '''prevent calling callback twice'''
            self.ioloop.add_callback(functools.partial(self.callback, self.status, self.stdout, self.stderr, self.has_timed_out))
if __name__ == "__main__":
    ioloop = tornado.ioloop.IOLoop.instance()
    def print_timeout( status, stdout, stderr, has_timed_out) :
        assert(status!=0)
        assert(has_timed_out)
        print "OK status:", repr(status), "stdout:", repr(stdout), "stderr:", repr(stderr), "timeout:", repr(has_timed_out)
    def print_ok( status, stdout, stderr, has_timed_out) :
        assert(status==0)
        assert(not has_timed_out)
        print "OK status:", repr(status), "stdout:", repr(stdout), "stderr:", repr(stderr), "timeout:", repr(has_timed_out)
    def print_error( status, stdout, stderr, has_timed_out):
        assert(status!=0)
        assert(not has_timed_out)
        print "OK status:", repr(status), "stdout:", repr(stdout), "stderr:", repr(stderr), "timeout:", repr(has_timed_out)
    def stop_test():
        ioloop.stop()
    t1 = Subprocess( print_timeout, timeout=3, args=[ "sleep", "5" ] )
    t2 = Subprocess( print_ok, timeout=3, args=[ "sleep", "1" ] )
    t3 = Subprocess( print_ok, timeout=3, args=[ "sleepdsdasdas", "1" ] )
    t4 = Subprocess( print_error, timeout=3, args=[ "cat", "/etc/sdfsdfsdfsdfsdfsdfsdf" ] )
    t1.start()
    t2.start()
    try:
```

```
    t3.start()
    assert(false)
except:
    print "OK"
t4.start()
ioloop.add_timeout(time.time() + 10, stop_test)
ioloop.start()
```

大家可以先用 uwsgi，要还是有压力和堵塞的话，可以用用 nginx 做负载。

我自己的经验来看还是这个靠谱~

3

python 写报警程序中的声音实现 winsound

写 windowns 下的报警程序，有一个报警声音的实现，在 python 中有个 winsound 模块可以来实现，方法也很简单：

```
import time
import winsound
def play_music():
    winsound.PlaySound('alert', winsound.SND_ASYNC)
    time.sleep(3)
   >import winsound
   PlaySound(sound, flags)
```

sound 是声音文件名字，该文件为 wav 格式的。flags 为其播放的一些参数，如：

SND_LOOP

重复地播放声音。SND_ASYNC标识也必须被用来避免堵塞。不能用 SND_MEMORY。

SND_MEMORY

提供给 PlaySound() 的 sound 参数是一个 WAV 文件的内存映像(memory image)，作为一个字符串。 注意：这个模块不支持从内存映像中异步播放，因此这个标识和 SND_ASYNC 的组合将挂起 RuntimeError。

SND_PURGE

停止播放所有指定声音的实例。

SND_ASYNC

立即返回，允许声音异步播放。

SND_NODEFAULT

不过指定的声音没有找到，不播放系统缺省的声音。

SND_NOSTOP

不中断当前播放的声音。

SND_NOWAIT

如果声音驱动忙立即返回。

MB_ICONASTERISK

播放 SystemDefault 声音。

MB_ICONEXCLAMATION

播放 SystemExclamation 声音。

MB_ICONHAND

播放 SystemHand 声音。

MB_ICONQUESTION
播放 SystemQuestion 声音。

MB_OK
播放 SystemDefault 声音。

python 蜂鸣，通过 python 让电脑发声:

```
import winsound
winsound.Beep(37, 2000)
```

37 是频率(Hz), 2000 是蜂鸣持续多少毫秒(ms).
第一个参数 frequency 表示分贝数，大小在 37 到 32767 之间。第二个参数是持续时间，以毫秒为单位

4

一个脚本讲述 python 语言的基础规范，适合初学者

最近学 python 的人比较多，今天讲一下 python 的基础：python 脚本的规范、缩进、编写功能函数时注意事项等，这些都是自己编程过程中的心得体会。

一、python 脚本的规范：

每个脚本都有自己的规范，以下的规范不是强制的，但是规范一下，可以使你的脚本规范、易懂、方便使用。

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
```

这个写在开头，定义脚本编码。现在多数都是 UTF8 格式，所以写脚本尽量用这个编码，遇到中文可以做编码处理，字符串编码处理主要就是 encode 和 decode

```
import os,urllib,MySQLdb,time,platform
```

导入需要的模块。

```
main():
  pass
```

定义函数

```
if __name__ == "__main__":
  main()
```

这个就是说脚本从这里往下执行，如果是其他的脚本调用这个脚本，这个脚本不至于执行其他的部分

提示：以上是整个脚本中的规范，大家在写脚本的时候尽量这么做。

二、python 的缩进

python 的对缩进要求很严格，缩进不对，就会报语法错误；python 中缩进就是一个 tab 键或是 4 个空格，4 个空格比较麻烦，直接一个 tab 键简单，所以没有特别的需求，缩进一般用 tab 键。缩进类似于分层，同一缩进就是相同的层次。见如下实例：

```
if a==0:
   print a
else:
   print b
```

三、每一个功能对应一个函数

这一点我认为最重要，每一个功能就写一个函数，这样你的脚本清晰易懂，脚本其他复用这个功能也方便，脚本也不冗余。不建议不要一个函数里面有好多功能，使函数模块化。

四、系统命令的引用

引用系统命令的时候，特别是 linux 命令，一定要写命令的全路径，比如：

```
os.popen("/sbin/ifconfig eth0").read()
```

这个你直接

```
os.popen("ifconfig eth0").read()
```

这样也是没有问题的，起码是你手动执行脚本时，这个是会执行的，但是脚本做 cron 的时候，就不会执行了。所以这个要特别注意。

五、异常处理

```
try:
  pass
except Exception,e:
 print e
```

其中 e 就是错误错误信息。try 的异常处理这么写就足够用了，还有其他的方法，不常用。

以下是一个获取本地 ip 地址，从数据库查询 ip 的用途，去连接一个 URL，判断这个 URL 是否可以用，并写日志。主要讲了讲 python 操作数据库的常用用法。

```
 #!/usr/bin/env python

 # -*- coding: utf-8 -*-

import os,urllib,MySQLdb,time,platform
def log_w(text):
    logfile = "/tmp/websocket.log"
    if os.path.isfile(logfile):
        if (os.path.getsize(logfile)/1024/1024) > 100:
            os.remove(logfile)
    now = time.strftime("%Y-%m-%d %H:%M:%S")
    tt = str(now) + "\t" + str(text) + "\n"
    f = open(logfile,'a+')
    f.write(tt)
    f.close()
def get_idcname(ip):
    try:
        conn = MySQLdb.connect(host = '192.168.8.43',port=3306,user = 'read_app',passwd = '123456',charset='utf8',conne
        cursor = conn.cursor()#查询出的结果是元组形式，元组和列表基本一样
        #cursor = conn.cursor(cursorclass = MySQLdb.cursors.DictCursor)#查询结果是字典形式
        sql = "select host,user from mysql.user where host='%s'" % ip#python中执行sql语句一次只能是一个sql语句，一次只执行
        cursor.execute(sql)#执行sql语句
        #cursor.executemany("""insert into dist_sniffer.sniffer_order_day values(%s,%s,%s,%s,%s,%s,%s,%s,%s) """,value
        alldata = cursor.fetchall()#接收sql执行结果，如果是写操作的，这个就不用了
        #conn.commit()如果是写操作，需要这个去提交
        cursor.close()
        conn.close()#关闭数据库回话
        return alldata[0][0].encode('UTF8')#如果是写操作的话就没有返回值了。
    except Exception,e:
        return 0
def get_ip():
```

```
    os = platform.system()
    if os == "Linux":
        ip = os.popen("/sbin/ifconfig eth0|grep 'inet addr'").read().strip().split(":")[1].split()[0]
    elif os == "Windows":
        import wmi
        c=wmi.WMI()
        network = c.Win32_NetworkAdapterConfiguration (IPEnabled=1)
        for interface in network:
            if interface.DefaultIPGateway:
                ip = interface.IPAddress[0]
                return ip
                #print interface.IPAddress[0],interface.MACAddress,interface.IPSubnet[0],interface.DefaultIPGateway[0],interfa
                #获取出网的ip地址、MAC地址、子网掩码、默认网关、DNS
def web_status():
    ip = get_ip()
    idc_name = get_idcname(ip)
    url = "http://www.text.com/index.php?idc_ip=%s&idc_name=%s" % (ip,idc_name)
    get = urllib.urlopen(url)
    if get.getcode() == 200:
        aa = int(get.read().strip())
        if aa == 1:
            text = "Webservice return OK"
        else:
            text =  "Webservice return Error"
    else:
        text = "Conect webservice Error"
    print text
    log_w(text)
if __name__ == "__main__":
    web_status()
```

一开始就要养成一个好习惯，这样对以后 python 编程是十分有益的。自己的深切体会。

5

python 计算文件的行数和读取某一行内容的实现方法

一、计算文件的行数

最简单的办法是把文件读入一个大的列表中,然后统计列表的长度.如果文件的路径是以参数的形式filepath传递的,那么只用一行代码就可以完成我们的需求了:

```
count = len(open(filepath,'rU').readlines())
```

如果是非常大的文件,上面的方法可能很慢,甚至失效.此时,可以使用循环来处理:

```
count = -1
for count, line in enumerate(open(thefilepath, 'rU')):
    pass
count += 1
```

另外一种处理大文件比较快的方法是统计文件中换行符的个数 '\n '(或者包含 '\n' 的字串,如在 windows 系统中):

```
count = 0
thefile = open(thefilepath, 'rb')
while True:
    buffer = thefile.read(8192*1024)
    if not buffer:
        break
    count += buffer.count('\n')
thefile.close( )
```

参数 'rb' 是必须的,否则在 windows 系统上,上面的代码会非常慢.

linecache 是专门支持读取大文件，而且支持行式读取的函数库。 linecache 预先把文件读入缓存起来，后面如果你访问该文件的话就不再从硬盘读取

二、读取文件某一行的内容（测试过 1 G 大小的文件，效率还可以）

```
import linecache
count = linecache.getline(filename,linenum)
```

三、用 linecache 读取文件内容（测试过 1 G 大小的文件，效率还可以）

```
str = linecache.getlines(filename)
```

str 为列表形式，每一行为列表中的一个元素

6

python 中用 string.maketrans 和 translate 巧妙替换字符串

python 中用 string.maketrans 和 translate 巧妙替换字符串

将 nginx 日志中字符串 [2013-07-03T00:29:40-05:00] HTTP 格式化为："2013-07-03 00:29:40-05:00"

整条日志如下：

92.82.22.46 - - [2013-07-03T00:29:40-05:00] "GET /images/mask_bg.png HTTP/1.1" 200 195 "htt
p://www.chlinux.net/" "Mozilla/5.0 (compatible; MSIE 10.0; Windows NT 6.1; WOW64; Trident/6.0)" "-"

将[2013-07-03T00:29:40-05:00] 替换成为："2013-07-03 00:29:40-05:00"

把 [] 换成"",然后把 T 替换成空格

做法如下：

```
>>> s='92.82.22.46 - - [2013-07-03T00:29:40-05:00] "GET /images/mask_bg.png HTTP/1.1" 200 195 "http://www.chlinu
>>> table = string.maketrans('[]','"')
>>> s.translate(table)
'92.82.22.46 - - "2013-07-03T00:29:40-05:00" "GET /images/mask_bg.png HTTP/1.1" 200 195 "http://www.chlinux.net/"
>>> s.translate(table).replace('T', ' ',1)#替换掉第一个T为空格
'92.82.22.46 - - "2013-07-03 00:29:40-05:00" "GET /images/mask_bg.png HTTP/1.1" 200 195 "http://www.chlinux.net/" "
也可以这样：
>>> table = re.sub('\[|\]','"',s).replace('T', ' ',1)
>>>print table
'92.82.22.46 - - "2013-07-03 00:29:40-05:00" "GET /images/mask_bg.png HTTP/1.1" 200 195 "http://www.chlinux.net/" "
```

7

python linecache 模块读取文件用法详解

linecache 模块允许从任何文件里得到任何的行，并且使用缓存进行优化，常见的情况是从单个文件读取多行。

linecache.getlines(filename)
从名为 filename 的文件中得到全部内容，输出为列表格式，以文件每行为列表中的一个元素,并以 linenum-1 为元素在列表中的位置存储

linecache.getline(filename,lineno)
从名为 filename 的文件中得到第 lineno 行。这个函数从不会抛出一个异常－产生错误时它将返回”（换行符将包含在找到的行里）。
如果文件没有找到，这个函数将会在 sys.path 搜索。

linecache.clearcache()
清除缓存。如果你不再需要先前从 getline() 中得到的行

linecache.checkcache(filename)
检查缓存的有效性。如果在缓存中的文件在硬盘上发生了变化，并且你需要更新版本，使用这个函数。如果省略 filename，将检查缓存里的所有条目。

linecache.updatecache(filename)
更新文件名为 filename 的缓存。如果 filename 文件更新了，使用这个函数可以更新 linecache.getlines(filename)返回的列表。

用法举例：

```
 # cat a.txt

1a
2b
3c
4d
5e
6f
7g
```

1、获取 a.txt 文件的内容

```
>>> a=linecache.getlines('a.txt')
>>> a
['1a\n', '2b\n', '3c\n', '4d\n', '5e\n', '6f\n', '7g\n']
```

2、获取 a.txt 文件中第 1-4 行的内容

```
>>> a=linecache.getlines('a.txt')[0:4]
>>> a
['1a\n', '2b\n', '3c\n', '4d\n']
```

3、获取 a.txt 文件中第4行的内容

```
>>> a=linecache.getline('a.txt',4)
>>> a
'4d\n'
```

注意：使用 linecache.getlines('a.txt') 打开文件的内容之后，如果 a.txt 文件发生了改变，如你再次用 linecache.getlines 获取的内容，不是文件的最新内容，还是之前的内容，此时有两种方法：

1. 使用 linecache.checkcache(filename) 来更新文件在硬盘上的缓存，然后在执行 linecache.getlines('a.txt') 就可以获取到 a.txt 的最新内容；
2. 直接使用 linecache.updatecache('a.txt')，即可获取最新的 a.txt 的最新内容

另：读取文件之后你不需要使用文件的缓存时需要在最后清理一下缓存，使 linecache.clearcache() 清理缓存，释放缓存。

这个模块是使用内存来缓存你的文件内容，所以需要耗费内存，打开文件的大小和打开速度和你的内存大小有关系。

8

python 调用 zabbix 的 api 接口添加主机、查询组、主机、模板

zabbix 有一个 API 接口，可以调用这些几口来自动添加主机，查询 zabbix 中监控的主机，监控的模板、监控的主机组等信息，使用也非常的方便。以下是用 python 调用 zabbix 的 API 接口来实现上述功能：

```
#!/usr/bin/env python

# -*- coding: utf-8 -*-

import json
import urllib2
import sys
class zabbixtools:
    def __init__(self):
        self.url = "http://192.168.100.200/zabbix/api_jsonrpc.php"
        self.header = {"Content-Type": "application/json"}
        self.authID = self.user_login()
    def user_login(self):
        data = json.dumps(
                {
                    "jsonrpc": "2.0",
                    "method": "user.login",
                    "params": {
                        "user": "Admin",
                        "password": "zabbix"
                        },
                    "id": 0
                    })
        request = urllib2.Request(self.url,data)
        for key in self.header:
            request.add_header(key,self.header[key])
        try:
            result = urllib2.urlopen(request)
        except URLError as e:
            print "Auth Failed, Please Check Your Name And Password:",e.code
        else:
            response = json.loads(result.read())
            result.close()
            authID = response['result']
            return authID
    def get_data(self,data,hostip=""):
        request = urllib2.Request(self.url,data)
        for key in self.header:
            request.add_header(key,self.header[key])
        try:
            result = urllib2.urlopen(request)
        except URLError as e:
```

```
        if hasattr(e, 'reason'):
            print 'We failed to reach a server.'
            print 'Reason: ', e.reason
        elif hasattr(e, 'code'):
            print 'The server could not fulfill the request.'
            print 'Error code: ', e.code
        return 0
    else:
        response = json.loads(result.read())
        result.close()
        return response
def host_get(self,hostip):
    #hostip = raw_input("\033[1;35;40m%s\033[0m" % 'Enter Your Check Host:Host_ip :')
    data = json.dumps(
            {
                "jsonrpc": "2.0",
                "method": "host.get",
                "params": {
                    "output":["hostid","name","status","host"],
                    "filter": {"host": [hostip]}
                    },
                "auth": self.authID,
                "id": 1
            })
    res = self.get_data(data)['result']
    if (res != 0) and (len(res) != 0):
        #for host in res:
        host = res[0]
        if host['status'] == '1':
            print "\t","\033[1;31;40m%s\033[0m" % "Host_IP:","\033[1;31;40m%s\033[0m" % host['host'].ljust(15),'\t',"\033[1;31;
            return host['hostid']
        elif host['status'] == '0':
            print "\t","\033[1;32;40m%s\033[0m" % "Host_IP:","\033[1;32;40m%s\033[0m" % host['host'].ljust(15),'\t',"\033[1;32
            return host['hostid']
        print
    else:
        print '\t',"\033[1;31;40m%s\033[0m" % "Get Host Error or cannot find this host,please check !"
        return 0
def host_del(self):
    hostip = raw_input("\033[1;35;40m%s\033[0m" % 'Enter Your Check Host:Host_ip :')
    hostid = self.host_get(hostip)
    if hostid == 0:
        print '\t',"\033[1;31;40m%s\033[0m" % "This host cannot find in zabbix,please check it !"
        sys.exit()
    data = json.dumps(
```

```
            {
                "jsonrpc": "2.0",
                "method": "host.delete",
                "params": [{"hostid": hostid}],
                "auth": self.authID,
                "id": 1
            })
    res = self.get_data(data)['result']
    if 'hostids' in res.keys():
        print "\t","\033[1;32;40m%s\033[0m" % "Delet Host:%s success !" % hostip
    else:
        print "\t","\033[1;31;40m%s\033[0m" % "Delet Host:%s failure !" % hostip
def hostgroup_get(self):
    data = json.dumps(
            {
                "jsonrpc": "2.0",
                "method": "hostgroup.get",
                "params": {
                    "output": "extend",
                    },
                "auth": self.authID,
                "id": 1,
                })
    res = self.get_data(data)
    if 'result' in res.keys():
        res = res['result']
        if (res !=0) or (len(res) != 0):
            print "\033[1;32;40m%s\033[0m" % "Number Of Group: ", "\033[1;31;40m%d\033[0m" % len(res)
            for host in res:
                print "\t","HostGroup_id:",host['groupid'],"\t","HostGroup_Name:",host['name'].encode('GBK')
            print
    else:
        print "Get HostGroup Error,please check !"
def template_get(self):
    data = json.dumps(
            {
                "jsonrpc": "2.0",
                "method": "template.get",
                "params": {
                    "output": "extend",
                    },
                "auth": self.authID,
                "id": 1,
                })
    res = self.get_data(data)#['result']
```

```
    if 'result' in res.keys():
        res = res['result']
        if (res !=0) or (len(res) != 0):
            print "\033[1;32;40m%s\033[0m" % "Number Of Template: ", "\033[1;31;40m%d\033[0m" % len(res)
            for host in res:
                print "\t","Template_id:",host['templateid'],"\t","Template_Name:",host['name'].encode('GBK')
            print
    else:
        print "Get Template Error,please check !"
def host_create(self):
    hostip = raw_input("\033[1;35;40m%s\033[0m" % 'Enter your:Host_ip :')
    groupid = raw_input("\033[1;35;40m%s\033[0m" % 'Enter your:Group_id :')
    templateid = raw_input("\033[1;35;40m%s\033[0m" % 'Enter your:Tempate_id :')
    g_list=[]
    t_list=[]
    for i in groupid.split(','):
        var = {}
        var['groupid'] = i
        g_list.append(var)
    for i in templateid.split(','):
        var = {}
        var['templateid'] = i
        t_list.append(var)
    if hostip and groupid and templateid:
        data = json.dumps(
              {
                 "jsonrpc": "2.0",
                 "method": "host.create",
                 "params": {
                    "host": hostip,
                    "interfaces": [
                       {
                          "type": 1,
                          "main": 1,
                          "useip": 1,
                          "ip": hostip,
                          "dns": "",
                          "port": "10050"
                       }
                    ],
                    "groups": g_list,
                    "templates": t_list,
              },
                 "auth": self.authID,
                 "id": 1,
```

```
                    })
            res = self.get_data(data,hostip)
            if 'result' in res.keys():
                res = res['result']
                if 'hostids' in res.keys():
                    print "\033[1;32;40m%s\033[0m" % "Create host success"
            else:
                print "\033[1;31;40m%s\033[0m" % "Create host failure: %s" % res['error']['data']
        else:
            print "\033[1;31;40m%s\033[0m" % "Enter Error: ip or groupid or tempateid is NULL,please check it !"
def main():
    test = zabbixtools()
    #test.template_get()
    #test.hostgroup_get()
    #test.host_get()
    test.host_del()
    #test.host_create()
if __name__ == "__main__":
    main()
```

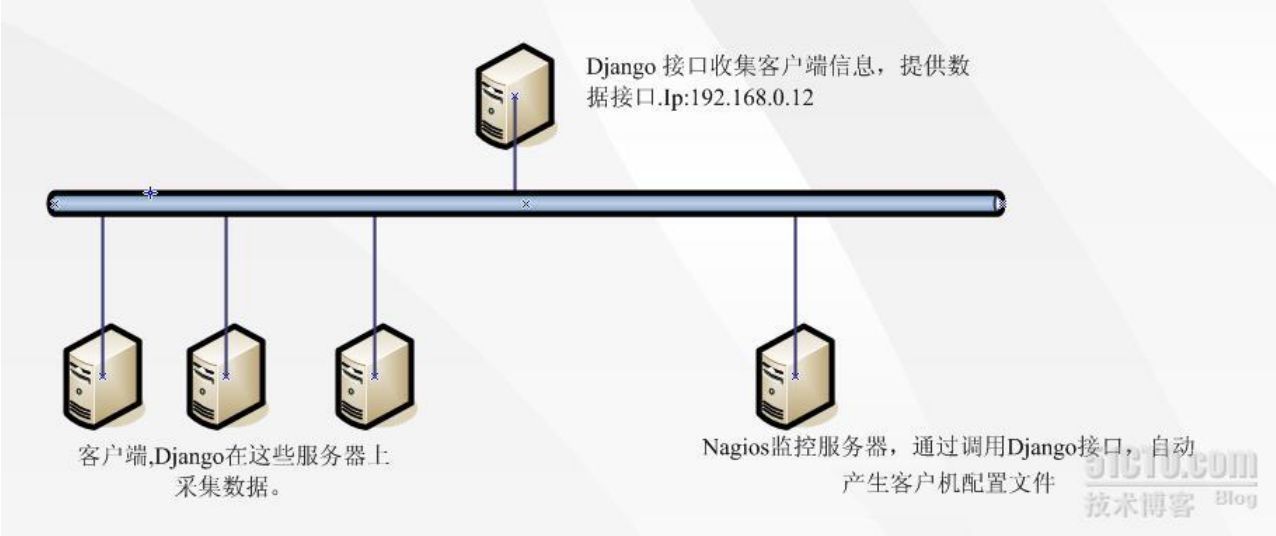
相关的材料的可以参考官方文档。这个只是一些功能模块，包含获取主机，主机组、模板、删除主机等功能，可以根据需要进行调整，实现 zabbix 的批量化和自动化管理。因为是在 linux 运行，所以设置了输出终端的字体颜色方便区分，如果不需要，自行删除即可。

9

python+Django 实现 Nagios 自动化添加监控项目

最近机房刚上了一批机器(有 100 台左右)，需要使用 Nagios 对这一批机器进行监控。领导要求两天时间完成所有主机的监控。从原来的经验来看，两天时间肯定完成不了。那怎么办？按照之前的想法，肯定是在 nagios 配置文件逐一添加每台客户端的监控信息，工作量巨大。突然，想到一个想法，是否可以通过脚本来实现批量对主机进行监控，也就是运维自动化。

写脚本，最重要的就是思路。思路压倒一切，经过思考最终决定就这么做了。先贴出来一张网路拓扑图：



图片 9.1 pic

整个过程可以分为三部分。

- cmdb 端:主要用来实现对数据的收集，采用两个 API，一个是提供给客户机的 API。用于将客户端的数据上传的 cmdb 服务器；另外一 个 API 是 nagios 通过此 API 可以得到要监控主机的信息，然后对该信息进行整理，做成 nagios 监控模板。
- Client 端：通过 Python 脚本收集本机器要监控的软硬件信息，然后通过 cmdb 端提供的 API 接口将数据上传到 cmdb 端的数据库。
- Nagios 端：通过 cmdb 端提供的 API 接口实现对 cmdb 收集到的信息进行抓取，然后将数据写入到模板，最后 copy 到 naigos 指定的 objects 目录，最终实现对客户机的监控。

这三部分最重要的应该是 CMDB 端。接下来通过安装 django 和编写 API 接口实现 cmdb 可以正常工作。可以将 cmdb 端分为三个步骤来完成：

- 安装 django
- 配置 django
- 编写 API 接口

首先来进行安装 django：

在安装 django 之前首先应该安装 python (版本建议 2.7.)

```
1.下载 django 软件包
  可以到 django 官方网站下载最新 django 软件包(https://www.djangoproject.com).
2.解压缩并安装软件包
   tar -zxvf Django-1.5.1.tar.gz
   cd Django-1.5.1
   python setup.py install
```

创建项目和应用:

```
1.创建一个项目
python startproject simplecmdb
2.创建一个应用
python startapp hostinfo
```

配置 django:

1.修改 setting.py

```
DATABASES = {'ENGIN':'django.db.backends.sqlite','name':path.join('CMDB.db')} #使用的数据库及数据库名
INSTALLED_APPS = ( hostinfoINSTALLED_APPS = （ 'hostinfo' )
INSTALLED_APPS = ('hostinfo')  #应用的名称
```

2.修改urls.py

```
url(r'^api/gethost\.json$','hostinfo.views.gethosts'), #nagios客户端访问API接口地址
url(r'^api/clooect$','hostinfo.views.collect'), #客户端访问API进行上传数据的API
url(r'^admin/',include(admin.site.urls)), #django后台管理登入url
from django.contrib import admin
admin.autodiscover()
```

3.修改项目 hostinfo 下的 views.py

代码如下:

```
 # Create your views here.

 #包含以下模块

from django.shortcuts import render_to_response
from django.http import HttpResponse
from models import Host, HostGroup
 #包含json模块

try:
```

```
    import json
except ImportError,e:
    import simplejson as json
 #用来接收客户端服务器发送过来的数据

def collect(request):
    req = request
    if req.POST:
        vendor = req.POST.get('Product_Name')
        sn = req.POST.get('Serial_Number')
        product = req.POST.get('Manufacturer')
        cpu_model = req.POST.get('Model_Name')
        cpu_num = req.POST.get('Cpu_Cores')
        cpu_vendor = req.POST.get('Vendor_Id')
        memory_part_number = req.POST.get('Part_Number')
        memory_manufacturer = req.POST.get('Manufacturer')
        memory_size = req.POST.get('Size')
        device_model = req.POST.get('Device_Model')
        device_version = req.POST.get('Firmware_Version')
        device_sn = req.POST.get('Serial_Number')
        device_size = req.POST.get('User_Capacity')
        osver = req.POST.get('os_version')
        hostname = req.POST.get('os_name')
        os_release = req.POST.get('os_release')
        ipaddrs = req.POST.get('Ipaddr')
        mac = req.POST.get('Device')
        link = req.POST.get('Link')
        mask = req.POST.get('Mask')
        device = req.POST.get('Device')
        host = Host()
        host.hostname = hostname
        host.product = product
        host.cpu_num = cpu_num
        host.cpu_model = cpu_model
        host.cpu_vendor = cpu_vendor
        host.memory_part_number = memory_part_number
        host.memory_manufacturer = memory_manufacturer
        host.memory_size = memory_size
        host.device_model = device_model
        host.device_version = device_version
        host.device_sn = device_sn
        host.device_size = device_size
        host.osver = osver
        host.os_release = os_release
        host.vendor = vendor
```

```
        host.sn = sn
        host.ipaddr = ipaddrs
        host.save()      #将客户端传过来的数据通过POST接收，存入数据库

        return HttpResponse('OK')  #如果插入成功，返回'ok'
    else:
        return HttpResponse('no post data')
 #提供给NAGIOS 的API

def gethosts(req):
    d = []
    hostgroups = HostGroup.objects.all()
    for hg in hostgroups:
        ret_hg = {'hostgroup':hg.name,'members':[]}
        members = hg.members.all()
        for h in members:
            ret_h = {'hostname':h.hostname,    #API接口返回的数据
                'ipaddr':h.ipaddr
            }
            ret_hg['members'].append(ret_h)
        d.append(ret_hg)
    ret = {'status':0,'data':d,'message':'ok'}
    return HttpResponse(json.dumps(ret))
```

4.修改 model.py 文件

代码如下：

```
from django.db import models
 # Create your models here.

 #插入数据库的Host表,主要存储客户端主机的信息

class Host(models.Model):
    """store host information"""
    vendor = models.CharField(max_length=30,null=True)
    sn = models.CharField(max_length=30,null=True)
    product = models.CharField(max_length=30,null=True)
    cpu_model = models.CharField(max_length=50,null=True)
    cpu_num = models.CharField(max_length=2,null=True)
    cpu_vendor = models.CharField(max_length=30,null=True)
    memory_part_number = models.CharField(max_length=30,null=True)
    memory_manufacturer = models.CharField(max_length=30,null=True)
    memory_size = models.CharField(max_length=20,null=True)
    device_model = models.CharField(max_length=30,null=True)
    device_version = models.CharField(max_length=30,null=True)
```

```
    device_sn = models.CharField(max_length=30,null=True)
    device_size = models.CharField(max_length=30,null=True)
    osver = models.CharField(max_length=30,null=True)
    hostname = models.CharField(max_length=30,null=True)
    os_release = models.CharField(max_length=30,null=True)
    ipaddr = models.IPAddressField(max_length=15)
    def __unicode__(self):
        return self.hostname
 #主机组表，用来对主机进行分组

class HostGroup(models.Model):
    name = models.CharField(max_length=30)
    members = models.ManyToManyField(Host)
```

5.修改 admin.py 文件

```
 #from models import Host, IPaddr

from models import Host, HostGroup
from django.contrib import admin
 #设置在django在admin后天显示的名称

class HostAdmin(admin.ModelAdmin):
    list_display = ['vendor',
        'sn',
        'product',
        'cpu_model',
        'cpu_num',
        'cpu_vendor',
        'memory_part_number',
        'memory_manufacturer',
        'memory_size',
        'device_model',
        'device_version',
        'device_sn',
        'device_size',
        'osver',
        'hostname',
        'os_release'
        ]
 #在django后台amdin显示的组名称

class HostGroupAdmin(admin.ModelAdmin):
    list_display = ['name',]
 #将如上两个类的数据展示到django的后台
```

```
admin.site.register(HostGroup,HostGroupAdmin)
admin.site.register(Host, HostAdmin)
```

6.创建数据库

```
python manager.py syncdb  #创建数据库
```

7.启动应用

```
python manager.py runserver 0.0.0.0:8000
```

8.测试

http://132.96.77.12:8000/admin



图片 9.2 pic



图片 9.3 pic

通过上图可以看到，django 已经配置成功。

接下来可以在客户端编写收集主机信息的脚本了，主要抓取 cpu、内存、硬盘、服务器型号、服务器 sn、ip 地址、主机名称、操作系统版本等信息，共 7 个脚本:

1.cpu 抓取脚本：

```
#!/usr/local/src/python/bin/python

#-*- coding:utf-8 -*-

from subprocess import PIPE,Popen
import re
def getCpuInfo():
    p = Popen(['cat','/proc/cpuinfo'],shell=False,stdout=PIPE)
    stdout, stderr = p.communicate()
    return stdout.strip()
def parserCpuInfo(cpudata):
    pd = {}
    model_name = re.compile(r'.*model name\s+:\s(.*)')
    vendor_id = re.compile(r'vendor_id\s+:(.*)')
    cpu_cores = re.compile(r'cpu cores\s+:\s([\d]+)')
    lines = [line for line in cpudata.split('\n')]
    for line in lines:
        model = re.match(model_name,line)
        vendor = re.match(vendor_id,line)
        cores = re.match(cpu_cores,line)
        if model:
            pd['Model_Name'] = model.groups()[0].strip()
        if vendor:
            pd['Vendor_Id'] = vendor.groups()[0].strip()
        if cores:
            pd['Cpu_Cores'] = cores.groups()[0]
        else:
            pd['Cpu_Cores'] = int('1')
    return pd
if __name__ == '__main__':
    cpudata = getCpuInfo()
    print parserCpuInfo(cpudata)
```

2.硬盘抓取脚本：

```
#!/usr/local/src/python/bin/python

#-*- coding:utf-8 -*-

from subprocess import PIPE,Popen
import re
def getDiskInfo():
    disk_dev = re.compile(r'Disk\s/dev/[a-z]{3}')
```

```
    disk_name = re.compile(r'/dev/[a-z]{3}')
    p = Popen(['fdisk','-l'],shell=False,stdout=PIPE)
    stdout, stderr = p.communicate()
    for i in stdout.split('\n'):
        disk = re.match(disk_dev,i)
        if disk:
            dk = re.search(disk_name,disk.group()).group()
    n = Popen('smartctl -i %s' % dk,shell=True,stdout=PIPE)
    stdout, stderr = n.communicate()
    return stdout.strip()
def parserDiskInfo(diskdata):
    ld = []
    pd = {}
    device_model = re.compile(r'(Device Model):(\s+.*)')
    serial_number = re.compile(r'(Serial Number):(\s+[\d\w]{1,30})')
    firmware_version = re.compile(r'(Firmware Version):(\s+[\w]{1,20})')
    user_capacity = re.compile(r'(User Capacity):(\s+[\d\w, ]{1,50})')
    for line in diskdata.split('\n'):
        serial = re.search(serial_number,line)
        device = re.search(device_model,line)
        firmware = re.search(firmware_version,line)
        user = re.search(user_capacity,line)
        if device:
            pd['Device_Model'] = device.groups()[1].strip()
        if serial:
            pd['Serial_Number'] = serial.groups()[1].strip()
        if firmware:
            pd['Firmware_Version'] = firmware.groups()[1].strip()
        if user:
            pd['User_Capacity'] = user.groups()[1].strip()
    return pd
if __name__ == '__main__':
    diskdata = getDiskInfo()
    print parserDiskInfo(diskdata)
```

3.内存抓取脚本：

```
 #!/usr/local/src/python/bin/python

 #-*- coding:utf-8 -*-

from subprocess import PIPE,Popen
import re
import sys
def getMemInfo():
    p = Popen(['dmidecode'],shell=False,stdout=PIPE)
```

```
    stdout, stderr = p.communicate()
    return stdout.strip()
def parserMemInfo(memdata):
    line_in = False
    mem_str = ''
    pd = {}
    fd = {}
    for line in memdata.split('\n'):
        if line.startswith('Memory Device') and line.endswith('Memory Device'):
            line_in = True
            mem_str+='\n'
            continue
        if line.startswith('\t') and line_in:
            mem_str+=line
        else:
            line_in = False
    for i in mem_str.split('\n')[1:]:
        lines = i.replace('\t','\n').strip()
        for ln in lines.split('\n'):
            k, v = [i for i in ln.split(':')]
            pd[k.strip()] = v.strip()
        if pd['Size'] != 'No Module Installed':
            mem_info = 'Size:%s  ; Part_Number:%s ; Manufacturer:%s' % (pd['Size'],pd['Part Number'],pd['Manufacturer'])
            for line in mem_info.split('\n'):
                for word in line.split(';'):
                    k, v = [i.strip() for i in word.split(':')]
                    fd[k] = v.strip()
                yield fd
if __name__ == '__main__':
    memdata = getMemInfo()
    for i in  parserMemInfo(memdata):
        print i
```

4.抓取服务器信息脚本：

```
 #!/usr/local/src/python/bin/python

 # -*- coding:utf-8 -*-

from subprocess import PIPE,Popen
import urllib, urllib2
def getDMI():
    p = Popen('dmidecode',shell=True,stdout=PIPE)
    stdout, stderr = p.communicate()
    return stdout
def parserDMI(dmidata):
```

```
    pd = {}
    fd = {}
    line_in = False
    for line in dmidata.split('\n'):
        if line.startswith('System Information'):
            line_in = True
            continue
        if line.startswith('\t') and line_in:
            k, v  = [i.strip() for i in line.split(':')]
            pd[k] = v
        else:
            line_in = False
    name = "Manufacturer:%s ; Serial_Number:%s ; Product_Name:%s" % (pd['Manufacturer'],pd['Serial Number'],pd['Pro
    for i in name.split(';'):
        k, v = [j.strip() for j in i.split(':')]
        fd[k] = v
    return fd
if __name__ == '__main__':
    dmidata = getDMI()
    postdata = parserDMI(dmidata)
    print postdata
```

5.抓取主机信息

```
 #!/usr/local/src/python/bin/python

 #-*- coding:utf-8 -*-

import platform
def getHostInfo():
    pd ={}
    version = platform.dist()
    os_name = platform.node()
    os_release = platform.release()
    os_version = '%s %s' % (version[0],version[1])
    pd['os_name'] = os_name
    pd['os_release'] = os_release
    pd['os_version'] = os_version
    return pd
if __name__ == '__main__':
    print getHostInfo()
```

6.抓取 ip 地址：

```
 #!/usr/local/src/python/bin/python
```

```
 #-*- coding:utf-8 -*-

from subprocess import PIPE,Popen
import re
def getIpaddr():
    p = Popen(['ifconfig'],shell=False,stdout=PIPE)
    stdout, stderr = p.communicate()
    return stdout.strip()
def parserIpaddr(ipdata):
    device = re.compile(r'(eth\d)')
    ipaddr = re.compile(r'(inet addr:[\d.]{7,15})')
    mac = re.compile(r'(HWaddr\s[0-9A-Fa-f:]{17})')
    link = re.compile(r'(Link encap:[\w]{3,14})')
    mask = re.compile(r'(Mask:[\d.]{9,15})')
    for lines in ipdata.split('\n\n'):
        pd = {}
        eth_device = re.search(device,lines)
        inet_ip = re.search(ipaddr,lines)
        hw = re.search(mac,lines)
        link_encap = re.search(link,lines)
        _mask = re.search(mask,lines)
        if eth_device:
            if eth_device:
                Device = eth_device.groups()[0]
            if inet_ip:
                Ipaddr =  inet_ip.groups()[0].split(':')[1]
            if hw:
                Mac = hw.groups()[0].split()[1]
            if link_encap:
                Link = link_encap.groups()[0].split(':')[1]
            if _mask:
                Mask = _mask.groups()[0].split(':')[1]
            pd['Device'] = Device
            pd['Ipaddr'] = Ipaddr
            pd['Mac'] = Mac
            pd['Link'] = Link
            pd['Mask'] = Mask
            yield pd
if __name__ == '__main__':
    ipdata = getIpaddr()
    for i in parserIpaddr(ipdata):
        print i
```

7.对这些信息进行合并，并通过API形式将数据发送给cmdb端

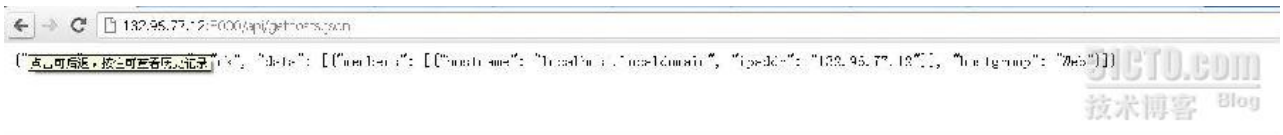
```
 #!/usr/local/src/python/bin/python

import urllib, urllib2
from cpuinfo import *
from diskinfo import *
from meminfo import *
from product import *
from hostinfo import *
from ipaddress import *
def getHostTotal():
    ld = []
    cpuinfo = parserCpuInfo(getCpuInfo())
    diskinfo = parserDiskInfo(getDiskInfo())
    for i in  parserMemInfo(getMemInfo()):
        meminfo = i
    productinfo = parserDMI(getDMI())
    hostinfo = getHostInfo()
    ipaddr = parserIpaddr(getIpaddr())
    for i in ipaddr:
        ip = i
    for k in cpuinfo.iteritems():
        ld.append(k)
    for i in diskinfo.iteritems():
        ld.append(i)
    for j in meminfo.iteritems():
        ld.append(j)
    for v in productinfo.iteritems():
        ld.append(v)
    for x in hostinfo.iteritems():
        ld.append(x)
    for y in ip.iteritems():
        ld.append(y)
    return ld
def parserHostTotal(hostdata):
    pg = {}
    for i in hostdata:
        pg[i[0]] = i[1]
    return pg
def urlPost(postdata):
    data = urllib.urlencode(postdata)
    req = urllib2.Request('http://132.96.77.12:8000/api/collect',data)
    response = urllib2.urlopen(req)
    return response.read()
if __name__ == '__main__':
    hostdata = getHostTotal()
```

```
    postdata = parserHostTotal(hostdata)
    print urlPost(postdata)
```

到目前为止，cmdb 系统已经可以将所有客户端的主机信息写入到数据库，并且可以通过 nagios 端的 API 接口直接调到数据：

http://132.96.77.12:8000/api/gethosts.json



图片 9.4 pic

通过图可以看到，已经成功调用到 API 接口的数据。

接下来可以在 nagios 端进行调用 API 接口的数据，对数据进行格式化。并写入文件。

1.nagios脚本

```
 #!/opt/data/py/bin/python

 #!-*- coding:utf-8 -*-

import urllib, urllib2
import json
import os
import shutil
CURR_DIR = os.path.abspath(os.path.dirname(__file__))
HOST_CONF_DIR = os.path.join(CURR_DIR,'hosts')
HOST_TMP = """define host {
    use       linux-server
    host_name  %(hostname)s
    check_command check-host-alive
    alias     %(hostname)s
    address    %(ipaddr)s
    contact_groups admins
}
"""
def getHosts():
    url = 'http://132.96.77.12:8000/api/gethosts.json'
    return json.loads(urllib2.urlopen(url).read())
def initDir():
    if not os.path.exists(HOST_CONF_DIR):
        os.mkdir(HOST_CONF_DIR)
def writeFile(f,s):
    with open(f,'w') as fd:
```

```
        fd.write(s)
def genNagiosHost(hostdata):
    initDir()
    conf = os.path.join(HOST_CONF_DIR,'hosts.cfg')
    hostconf = ""
    for hg in hostdata:
        for h in hg['members']:
            hostconf+=HOST_TMP %h
    writeFile(conf,hostconf)
    return "ok"
def main():
    result = getHosts()
    if result['status'] == 0:
        print genNagiosHost(result['data'])
    else:
        print 'Error: %s' % result['message']
    if os.path.exists(os.path.join(HOST_CONF_DIR,'hosts.cfg')):
        os.chdir(HOST_CONF_DIR)
        shutil.copyfile('hosts.cfg','/etc/nagios/objects/hosts.cfg')
if __name__ == "__main__":
    main()
```

现在已经生成 nagios 主机的配置文件，并 copy 到 nagios/objects 目录下 hosts.cfg。接下来可以测试是否 nagios 配置有问题，如果没有问题，就可以启动 nagios 服务

```
[root@yetcomm-v2 bin]# ./nagios -v /etc/nagios/nagios.cfg
```

通过测试，nagios 没有发生错误或警告信息，现在可以启动 nagios 服务：

```
[root@yetcomm-v2 bin]# service nagios restart
```

最后，可以通过浏览器查看 nagios 的监控界面：



图片 9.5 pic

通过上图，可以看到已经将一台主机加入到监控组。由于是生产环境，所有只能拿测试服务器进行测试。其实测试环境和生产环境的代码完全一致。

10

通过 python 和 websocket 构建实时通信系统[扩展 saltstack 监控]

先放一个小 demo~

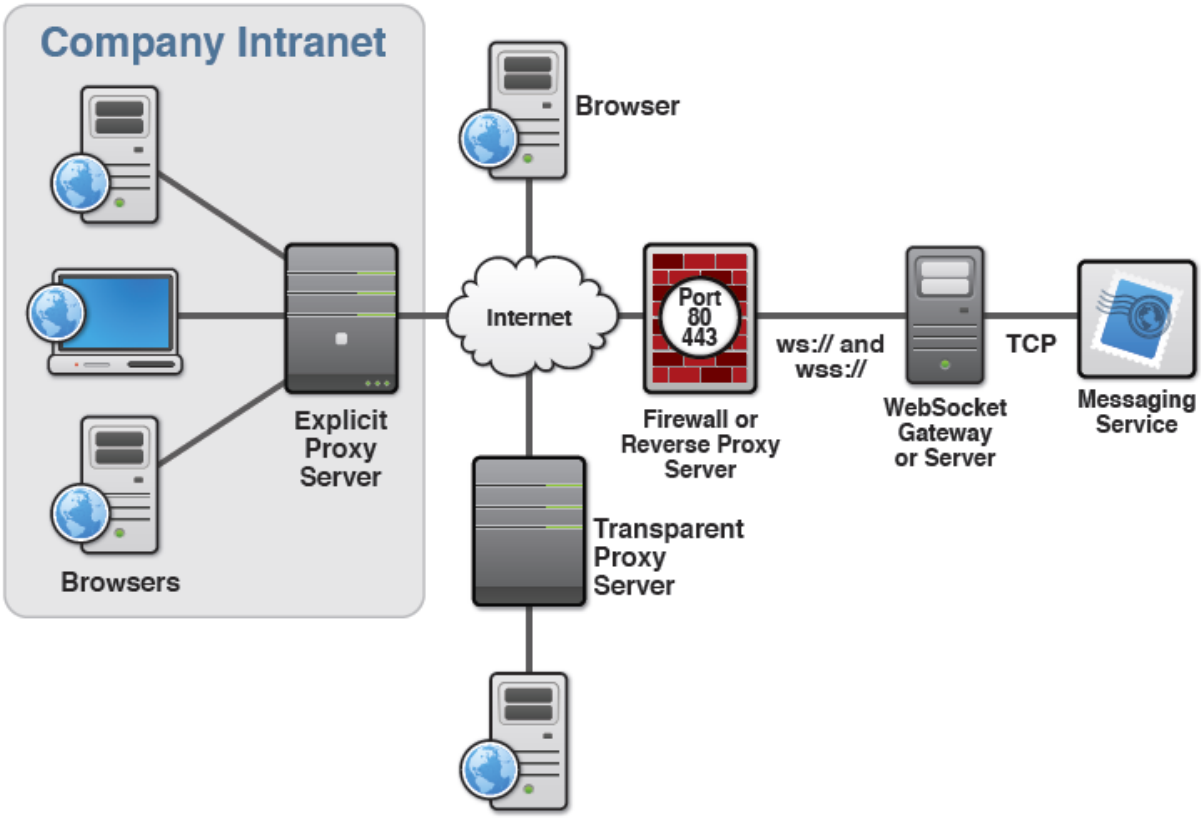
用 html5 的 websocket 实现的聊天平台。后端用的是 python bottle 框架。

后期要改成监控，可能要联合 saltstack 做实时的监控。

像上篇博客说的那样，实时监控就那点东西，就是接收数据、显示数据 。

像下面这样：

原文地址：http://rfyiamcool.blog.51cto.com/1030776/1269232



图片 10.1 pic

WebSocket API 是下一代客户端–服务器的异步通信方法。该通信取代了单个的 TCP 套接字，使用 ws 或 wss 协议，可用于任意的客户端和服务器程序。

WebSocket 目前由 W3C 进行标准化。WebSocket 已经受到 Firefox 4、Chrome 、Opera 10.70 以及 Safari 5 等浏览器的支持。

WebSocket API最伟大之处在于服务器和客户端可以在给定的时间范围内的任意时刻，相互推送信息。WebSocket 并不限于以 Ajax(或 XHR)方式通信，因为 Ajax 技术需要客户端发起请求，而 WebSocket 服务器和客户端可以彼此相互推送信息；XHR 受到域的限制，而 WebSocket 允许跨域通信。

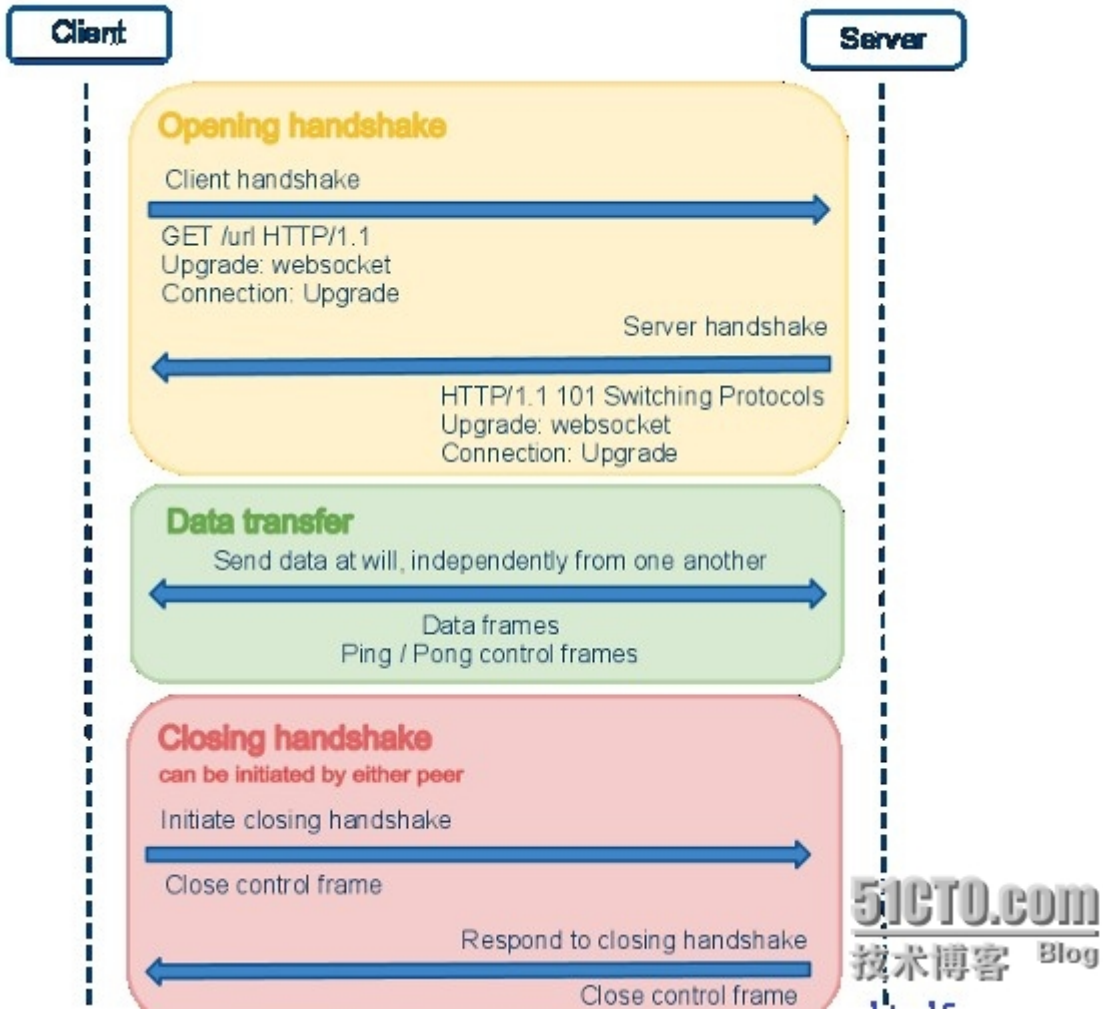
WebSocket 的优点

a)、服务器与客户端之间交换的标头信息很小，大概只有 2 字节;

b)、客户端与服务器都可以主动传送数据给对方;

c)、不用频率创建 TCP 请求及销毁请求，减少网络带宽资源的占用，同时也节省服务器资源;

建立连接的握手
当 Web 应用程序调用 new WebSocket(url)接口时，Browser 就开始了与地址为 url 的 WebServer 建立握手连接的过程。
1. Browser 与 WebSocket 服务器通过 TCP 三次握手建立连接，如果这个建立连接失败，那么后面的过程就不会执行，Web应用程序
2. 在 TCP 建立连接成功后，Browser/UA 通过 http 协议传送 WebSocket 支持的版本号，协议的字版本号，原始地址，主机地址等等
3. WebSocket 服务器收到 Browser/UA 发送来的握手请求后，如果数据包数据和格式正确，客户端和服务器端的协议版本号匹配等等
4. Browser 收到服务器回复的数据包后，如果数据包内容、格式都没有问题的话，就表示本次连接成功，触发 onopen 消息，此时 W
这个握手很像 HTTP，但是实际上却不是，它允许服务器以 HTTP 的方式解释一部分 handshake 的请求，然后切换为 websocket
数据传输
WebScoket 协议中，数据以帧序列的形式传输。
考虑到数据安全性，客户端向服务器传输的数据帧必须进行掩码处理。服务器若接收到未经过掩码处理的数据帧，则必须主动关闭连接。
服务器向客户端传输的数据帧一定不能进行掩码处理。客户端若接收到经过掩码处理的数据帧，则必须主动关闭连接。
针对上情况，发现错误的一方可向对方发送 close 帧(状态码是 1002，表示协议错误)，以关闭连接。



图片 10.2 pic

ws的连接状态：

```
GET /chat HTTP/1.1
Upgrade: WebSocket
Connection: Upgrade
Host: 66.xiaorui.cc:10000
Origin: http://66.xiaorui.cc
Cookie: somenterCookie
```

简单了解下接口方法和属性：

readyState 表示连接有四种状态：

- CONNECTING (0)：表示还没建立连接；
- OPEN (1)： 已经建立连接，可以进行通讯；
- CLOSING (2)：通过关闭握手，正在关闭连接；
- CLOSED (3)：连接已经关闭或无法打开；

url 是代表 WebSocket 服务器的网络地址，协议通常是”ws”或“wss(加密通信)”,send 方法就是发送数据到服务器端；

close 方法就是关闭连接；
onopen 连接建立，即握手成功触发的事件；
onmessage 收到服务器消息时触发的事件；
onerror 异常触发的事件；
onclose 关闭连接触发的事件；

来个例子，咱们用 js 来搞搞

```
var wsServer = 'ws://localhost:8888/Demo'; //服务器地址
var websocket = new WebSocket(wsServer); //创建 WebSocket 对象
websocket.send("hello");//向服务器发送消息
alert(websocket.readyState);//查看 websocket 当前状态
websocket.onopen = function (evt) {
    //已经建立连接
};
websocket.onclose = function (evt) {
    //已经关闭连接
};
websocket.onmessage = function (evt) {
    //收到服务器消息，使用 evt.data 提取
};
websocket.onerror = function (evt) {
    //产生异常
};
```

我的后端代码：

python 的后端实现 websocket 的处理，有很多方法的。

比较常见的是 gevent 的 websocket 的方式。

```
from bottle import get, run, template
from bottle.ext.websocket import GeventWebSocketServer
from bottle.ext.websocket import websocket
import gevent
users = set()
@get('/')
def index():
    return template('index')
@get('/websocket', apply=[websocket])
def chat(ws):
    users.add(ws)
    while True:
        msg = ws.receive()
        if msg is not None:
            for u in users:
                print type(u)
                u.send(msg)
                print u,msg
        else: break
    users.remove(ws)
run(host='10.10.10.66', port=10000, server=GeventWebSocketServer)
```

后端的东西比较的简单，就是把接收到的数据，原路打回去。。。

我前端的代码

这个是连接 webscoket，然后接收和发数据的 js

```
<script>
        $(document).ready(function() {
            if (!window.WebSocket) {
                if (window.MozWebSocket) {
                    window.WebSocket = window.MozWebSocket;
                } else {
                    $('#messages').append("<li>Your browser doesn't support WebSockets.</li>");
                }
            }
            ws = new WebSocket('ws://10.10.10.66:10000/websocket');
            ws.onopen = function(evt) {
                $('#messages').append('<li>Connected to chat.</li>');
            }
            ws.onmessage = function(evt) {
                $('#messages').append('<li>' + evt.data + '</li>');
```

```
        }
        $('#send-message').submit(function() {
            ws.send($('#name').val() + ": " + $('#message').val());
            $('#message').val('').focus();
            return false;
        });
    });
</script>
```

用来呈现结果的 div

```
form id="send-message" class="form-inline">
      <input id="name" type="text" value="可以更换名字">
      <input id="message" type="text" value="要扯淡的内容" />
       <button class="btn btn-success" type="submit">Send</button>
</form>
<div id="messages"></div>
```

这里有个 tornado 后端的代码，实现的过程和我差不多的~我需要的朋友可以跑一下~

```
import logging
import os.path
import uuid
import tornado.httpserver
import tornado.ioloop
import tornado.options
import tornado.web
import tornado.websocket
def send_message(message):
    for handler in ChatSocketHandler.socket_handlers:
        try:
            handler.write_message(message)
        except:
            logging.error('Error sending message', exc_info=True)
class MainHandler(tornado.web.RequestHandler):
    def get(self):
        self.render('index.html')
class ChatSocketHandler(tornado.websocket.WebSocketHandler):
    socket_handlers = set()
    def open(self):
        ChatSocketHandler.socket_handlers.add(self)
        send_message('A new user has entered the chat room.')
    def on_close(self):
        ChatSocketHandler.socket_handlers.remove(self)
        send_message('A user has left the chat room.')
    def on_message(self, message):
```

```
        send_message(message)
def main():
    settings = {
        'template_path': os.path.join(os.path.dirname(__file__), 'templates'),
        'static_path': os.path.join(os.path.dirname(__file__), 'static')
    }
    application = tornado.web.Application([
        ('/', MainHandler),
        ('/new-msg/', ChatHandler),
        ('/new-msg/socket', ChatSocketHandler)
    ], **settings)
    http_server = tornado.httpserver.HTTPServer(application)
    http_server.listen(8000)
    tornado.ioloop.IOLoop.instance().start()
if __name__ == '__main__':
    main()
```

我和沈灿的对话~



图片 10.3 pic

沈灿和我的对话



图片 10.4 pic

本文出自 “峰云，就她了。” 博客，谢绝转载!

11

关于 B+tree (附 python 模拟代码)

前几天我写了点 btree 的东西(http://thuhak.blog.51cto.com/2891595/1261783)，今天继续这个思路，继续写 b+tree。

而且 b+tree 才是我的目的，更加深入理解文件和数据库索引的基本原理。

在之前，我一直只把 b+tree 当成是 btree 的一种变形，或者说是在某种情况下的一种优化，另外一些情况可能还是 btree 好些。但是做完之后才发现，b+tree 在各种情况都可以完全取代 btree，并能够让索引性能得到比 btree 更好的优化。因为 b+tree 设计的核心要点，是为了弥补 btree 最大的缺陷。

btree 最大的缺陷是什么?

首先,我们知道对于 btree 和 b+tree 这种多路搜索树来说，一个很重要的特点就是树的度数非常大。因为只有这样才能够降低树的深度，减少磁盘读取的次数。而树的度数越大，叶子节点在树中的比例就越大。假设度数为 1000，那么叶子节点比他上一层内部节点的数量至少要多 1000 倍，在上一层就更加可以忽略不计了。可以说树种 99.9% 的节点都是叶子节点。 但是对于 btree 来说，所有节点都是一样的结构，都含有一定数量的数据和指向节点的指针。这两项数据占据 btree 节点的几乎全部的空间。一个节点内的数据的数量比硬盘指针的数量少一，可以说和指针的数量几乎相等。对于 python 这种动态类型语言感觉不出来，但是对于 C 这种固定类型语言来说，即使这个 children list 数组为空，这个数组的空间也都是预留出去的。导致的结果就是占绝大多数的叶子节点的 children list 指针数组所占的磁盘空间完全浪费。

一个数据的大小和硬盘指针的大小取决于 key-value 中 key 和 value 大小的比值。假如说这个比值是 2:1。那么 btree 浪费了几乎 1/3 的空间。

b+tree 针对这个问题的，把叶子节点和内节点的数据结构分开设计，让叶子节点不存放指针。因此同样大小的叶子节点，b+tree 所能包含数据数量要比 btree 大。按照上面的假设就是大 1/2。数的深度很可能比 btree 矮，大范围搜索或遍历所需要的载入磁盘的次数也少。

另外，b+tree 还有一个特点是所有数据都存放在叶子节点，这些叶子节点也可以组成一个链表，并把这个链表的表头拿出来，方便直访问数据。有些文章认为这对于范围搜索来说是个巨大的优化。但是就我的看法，这个特性最大的作用仅仅是让代码更容易一些，性能上，只会比树的遍历差，而不会比树的遍历好。因为不管是用指向叶子节点的指针搜，还是用树的遍历搜，所搜索的节点的数量都是几乎相同的。在相同大小的范围搜索的性能，只取决于访问顺序的连续性。从树根向下遍历，那么一次可以取得大量的子节点的范围，并针对这些节点做访问排序，得到更好的访问连续性。如果是沿着指向兄弟节点的指针搜索，一是兄弟节点也许是后插入的，存放并不一定和自己是连续的，二是只有每次从硬盘中将该节点载入到内存，才知道兄弟节点放在硬盘哪个位置，这又变成了对硬盘的一个随机的同步操作，性能的下降可想而知。

说 b+tree 因为有指向兄弟节点的指针方便数据库扫库这种结论，是不正确的。

还是上代码吧,依旧只是在内存对数据结构插入删除查找的模拟

be

```
 #!/usr/bin/env python

from random import randint,choice
from bisect import bisect_right,bisect_left
from collections import deque
class InitError(Exception):
    pass
class ParaError(Exception):
    pass
class KeyValue(object):
    __slots__=('key','value')
    def __init__(self,key,value):
        self.key=key
        self.value=value
    def __str__(self):
        return str((self.key,self.value))
    def __cmp__(self,key):
        if self.key>key:
            return 1
        elif self.key==key:
            return 0
        else:
            return -1
class Bptree_InterNode(object):
    def __init__(self,M):
        if not isinstance(M,int):
            raise InitError,'M must be int'
        if M<=3:
            raise InitError,'M must be greater then 3'
        else:
            self.__M=M
            self.clist=[]
            self.ilist=[]
            self.par=None
    def isleaf(self):
        return False
    def isfull(self):
        return len(self.ilist)>=self.M-1
    def isempty(self):
        return len(self.ilist)<=(self.M+1)/2-1
    @property
    def M(self):
        return self.__M
```

```
class Bptree_Leaf(object):
    def __init__(self,L):
        if not isinstance(L,int):
            raise InitError,'L must be int'
        else:
            self.__L=L
            self.vlist=[]
            self.bro=None
            self.par=None
    def isleaf(self):
        return True
    def isfull(self):
        return len(self.vlist)>self.L
    def isempty(self):
        return len(self.vlist)<=(self.L+1)/2
    @property
    def L(self):
        return self.__L
class Bptree(object):
    def __init__(self,M,L):
        if L>M:
            raise InitError,'L must be less or equal then M'
        else:
            self.__M=M
            self.__L=L
            self.__root=Bptree_Leaf(L)
            self.__leaf=self.__root
    @property
    def M(self):
        return self.__M
    @property
    def L(self):
        return self.__L
    def insert(self,key_value):
        node=self.__root
        def split_node(n1):
            mid=self.M/2
            newnode=Bptree_InterNode(self.M)
            newnode.ilist=n1.ilist[mid:]
            newnode.clist=n1.clist[mid:]
            newnode.par=n1.par
            for c in newnode.clist:
                c.par=newnode
            if n1.par is None:
                newroot=Bptree_InterNode(self.M)
```

```
            newroot.ilist=[n1.ilist[mid-1]]
            newroot.clist=[n1,newnode]
            n1.par=newnode.par=newroot
            self.__root=newroot
        else:
            i=n1.par.clist.index(n1)
            n1.par.ilist.insert(i,n1.ilist[mid-1])
            n1.par.clist.insert(i+1,newnode)
        n1.ilist=n1.ilist[:mid-1]
        n1.clist=n1.clist[:mid]
        return n1.par
    def split_leaf(n2):
        mid=(self.L+1)/2
        newleaf=Bptree_Leaf(self.L)
        newleaf.vlist=n2.vlist[mid:]
        if n2.par==None:
            newroot=Bptree_InterNode(self.M)
            newroot.ilist=[n2.vlist[mid].key]
            newroot.clist=[n2,newleaf]
            n2.par=newleaf.par=newroot
            self.__root=newroot
        else:
            i=n2.par.clist.index(n2)
            n2.par.ilist.insert(i,n2.vlist[mid].key)
            n2.par.clist.insert(i+1,newleaf)
            newleaf.par=n2.par
        n2.vlist=n2.vlist[:mid]
        n2.bro=newleaf
    def insert_node(n):
        if not n.isleaf():
            if n.isfull():
                insert_node(split_node(n))
            else:
                p=bisect_right(n.ilist,key_value)
                insert_node(n.clist[p])
        else:
            p=bisect_right(n.vlist,key_value)
            n.vlist.insert(p,key_value)
            if n.isfull():
                split_leaf(n)
            else:
                return
    insert_node(node)
def search(self,mi=None,ma=None):
    result=[]
```

```
    node=self.__root
    leaf=self.__leaf
    if mi is None and ma is None:
        raise ParaError,'you need to setup searching range'
    elif mi is not None and ma is not None and mi>ma:
        raise ParaError,'upper bound must be greater or equal than lower bound'
    def search_key(n,k):
        if n.isleaf():
            p=bisect_left(n.vlist,k)
            return (p,n)
        else:
            p=bisect_right(n.ilist,k)
            return search_key(n.clist[p],k)
    if mi is None:
        while True:
            for kv in leaf.vlist:
                if kv<=ma:
                    result.append(kv)
                else:
                    return result
            if leaf.bro==None:
                return result
            else:
                leaf=leaf.bro
    elif ma is None:
        index,leaf=search_key(node,mi)
        result.extend(leaf.vlist[index:])
        while True:
            if leaf.bro==None:
                return result
            else:
                leaf=leaf.bro
                result.extend(leaf.vlist)
    else:
        if mi==ma:
            i,l=search_key(node,mi)
            try:
                if l.vlist[i]==mi:
                    result.append(l.vlist[i])
                    return result
                else:
                    return result
            except IndexError:
                return result
        else:
```

```
            i1,l1=search_key(node,mi)
            i2,l2=search_key(node,ma)
            if l1 is l2:
                if i1==i2:
                    return result
                else:
                    result.extend(l.vlist[i1:i2])
                    return result
            else:
                result.extend(l1.vlist[i1:])
                l=l1
                while True:
                    if l.bro==l2:
                        result.extend(l2.vlist[:i2+1])
                        return result
                    else:
                        result.extend(l.bro.vlist)
                        l=l.bro
    def traversal(self):
        result=[]
        l=self.__leaf
        while True:
            result.extend(l.vlist)
            if l.bro==None:
                return result
            else:
                l=l.bro
    def show(self):
        print 'this b+tree is:\n'
        q=deque()
        h=0
        q.append([self.__root,h])
        while True:
            try:
                w,hei=q.popleft()
            except IndexError:
                return
            else:
                if not w.isleaf():
                    print w.ilist,'the height is',hei
                    if hei==h:
                        h+=1
                    q.extend([[i,h] for i in w.clist])
                else:
                    print [v.key for v in w.vlist],'the leaf is,',hei
```

```
def delete(self,key_value):
    def merge(n,i):
        if n.clist[i].isleaf():
            n.clist[i].vlist=n.clist[i].vlist+n.clist[i+1].vlist
            n.clist[i].bro=n.clist[i+1].bro
        else:
            n.clist[i].ilist=n.clist[i].ilist+[n.ilist[i]]+n.clist[i+1].ilist
            n.clist[i].clist=n.clist[i].clist+n.clist[i+1].clist
        n.clist.remove(n.clist[i+1])
        n.ilist.remove(n.ilist[i])
        if n.ilist==[]:
            n.clist[0].par=None
            self.__root=n.clist[0]
            del n
            return self.__root
        else:
            return n
    def tran_l2r(n,i):
        if not n.clist[i].isleaf():
            n.clist[i+1].clist.insert(0,n.clist[i].clist[-1])
            n.clist[i].clist[-1].par=n.clist[i+1]
            n.clist[i+1].ilist.insert(0,n.ilist[i])
            n.ilist[i]=n.clist[i].ilist[-1]
            n.clist[i].clist.pop()
            n.clist[i].ilist.pop()
        else:
            n.clist[i+1].vlist.insert(0,n.clist[i].vlist[-1])
            n.clist[i].vlist.pop()
            n.ilist[i]=n.clist[i+1].vlist[0].key
    def tran_r2l(n,i):
        if not n.clist[i].isleaf():
            n.clist[i].clist.append(n.clist[i+1].clist[0])
            n.clist[i+1].clist[0].par=n.clist[i]
            n.clist[i].ilist.append(n.ilist[i])
            n.ilist[i]=n.clist[i+1].ilist[0]
            n.clist[i+1].clist.remove(n.clist[i+1].clist[0])
            n.clist[i+1].ilist.remove(n.clist[i+1].ilist[0])
        else:
            n.clist[i].vlist.append(n.clist[i+1].vlist[0])
            n.clist[i+1].vlist.remove(n.clist[i+1].vlist[0])
            n.ilist[i]=n.clist[i+1].vlist[0].key
    def del_node(n,kv):
        if not n.isleaf():
            p=bisect_right(n.ilist,kv)
```

```
            if p==len(n.ilist):
                if not n.clist[p].isempty():
                    return del_node(n.clist[p],kv)
                elif not n.clist[p-1].isempty():
                    tran_l2r(n,p-1)
                    return del_node(n.clist[p],kv)
                else:
                    return del_node(merge(n,p),kv)
            else:
                if not n.clist[p].isempty():
                    return del_node(n.clist[p],kv)
                elif not n.clist[p+1].isempty():
                    tran_r2l(n,p)
                    return del_node(n.clist[p],kv)
                else:
                    return del_node(merge(n,p),kv)
        else:
            p=bisect_left(n.vlist,kv)
            try:
                pp=n.vlist[p]
            except IndexError:
                return -1
            else:
                if pp!=kv:
                    return -1
                else:
                    n.vlist.remove(kv)
                    return 0
    del_node(self.__root,key_value)
def test():
    mini=2
    maxi=60
    testlist=[]
    for i in range(1,10):
        key=i
        value=i
        testlist.append(KeyValue(key,value))
    mybptree=Bptree(4,4)
    for kv in testlist:
        mybptree.insert(kv)
    mybptree.delete(testlist[0])
    mybptree.show()
    print '\nkey of this b+tree is \n'
    print [kv.key for kv in mybptree.traversal()]
    #print [kv.key for kv in mybptree.search(mini,maxi)]
```

```
if __name__=='__main__':
    test()
```

实现过程和 btree 很像，不过有几点显著不同。

1. 内节点不存储 key-value，只存放 key
2. 沿着内节点搜索的时候，查到索引相等的数要向树的右边走。所以二分查找要选择 bisect_right
3. 在叶子节点满的时候，并不是先分裂再插入而是先插入再分裂。因为 b+tree 无法保证分裂的两个节点的大小都是相等的。在奇数大小的数据分裂的时候右边的子节点会比左边的大。如果先分裂再插入无法保证插入的节点一定会插在数量更少的子节点上，满足节点数量平衡的条件。
4. 在删除数据的时候，b+tree 的左右子节点借数据的方式比 btree 更加简单有效，只把子节点的子树直接剪切过来，再把索引变一下就行了，而且叶子节点的兄弟指针也不用动。

12

Python 编写的 socket 服务器和客户端

服务器端：

```
#!/usr/bin/python

import socket
host='127.0.0.1'
port=8123
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.bind((host,port))
s.listen(2)
try:
    while True:
        conn,add=s.accept()
        while True:
            data2=''
            data1=conn.recv(3)
            if data1=='EOF':
                conn.send('hello clietn1')
                break
            if data1=='FOE':
                conn.send('hello client2')
                break
            data2+=data1
            print data2
except KeyboardInterrupt:
    print "you have CTRL+C,Now quit"
    s.close()
```

注：服务器端一次只接收 3 个字节的数据，我让读取进入循环，然后不断累加到 data2 中，当读取到 EOF 时，退出打印 data2，当读取 FOE 时，退出打印 data2，（EOF 和 FOE 是客户端发送完数据时发送的结束符），当接收到 CTRLC+C 时，关闭 socket

客户端 1：

```
#!/usr/bin/env python

import socket
import os
ss=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
ss.connect(('127.0.0.1',8123))
#f=open('aa','wb')

ss.sendall('hello serverdddddd
```

```
data=ss.recv(1024)
print "server dafu %s"%data
ss.close()
```

客户端 2：

```
 #!/usr/bin/env python

import socket
import os
ss=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
ss.connect(('127.0.0.1',8123))
 #f=open('aa','wb')

ss.sendall('wokao sile')
os.system('sleep 1')
ss.send('FOE')
data=ss.recv(1024)
print "server dafu %s"%data
ss.close()
```

13

python 之 MySQLdb 库的使用

在开发的过程中避免不了和数据库的交互，在实际环境中用的最多的 Mysql 数据库，那 python 是怎么和 Mysql 进行交互的呢，python 使用一个叫 MySQLdb 的库来连接 MySQL，好的，下面最要从 MySQLdb 的安装、连接 MySQL、执行 sql 语句、如何取得结果、关闭数据库连接来讲述一下：

1. MySQLdb 的安装

我使用的是 ubuntu 系统，安装方法为：apt-get install python-MySQLdb，这样当在 python 环境执行 import MySQLdb 不报错就是安

```
root@ubuntu:~# python
Python 2.7.4 (default, Apr 19 2013, 18:32:33)
[GCC 4.7.3] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import MySQLdb
>>>
```

1. 如何连接 Mysql

MySQLdb 提供的 connect 方法用来和数据库建立连接,接收数个参数,返回连接对象，如：

```
conn=MySQLdb.connect(host="localhost",user="root",passwd="sa",db="mytable",port=3306)
```

比较常用的参数包括:

host:数据库主机名.默认是用本地主机.
user:数据库登陆名.默认是当前用户.
passwd:数据库登陆的秘密.默认为空.
db:要使用的数据库名.没有默认值,如果在这里设置了 db,则连接时直接连接到 Mysql 的 db 设置的数据库中
port:MySQL服务使用的TCP端口.默认是3306.

注：connect 中的 host、user、passwd 等可以不写，只有在写的时候按照 host、user、passwd、db(可以不写)、port 顺序写就可以，注意端口号 port=3306 还是不要省略的为好，如果没有 db 在 port 前面，直接写 3306 会报错

连接成功后，如需切换该用户的其他数据库，使用以下语句：conn.select_db('mysql')形式切换数据库

```
>>> con=MySQLdb.connect('localhost','root','123456',port=3306)
>>> con.select_db('mysql')
>>> cur=con.cursor()
>>> cur.execute('show tables')
24L
>>> cur.fetchall()
(('columns_priv',), ('db',), ('event',), ('func',), ('general_log',), ('help_category',), ('help_keyword',), ('help_relation',), ('help_to
```

第 1 行：连接数据库
第 2 行：选择连接 mysql 这个数据库
第 3 行以下是获取数据库表，语法后面会讲

1. 怎么操作数据库，MySQLdb 用游标（指针）cursor 的方式操作数据库

因该模块底层其实是调用 C API 的，所以，需要先得到当前指向数据库的指针

```
>>> cur=con.cursor()
```

1. 数据库的操作和结果显示

我们利用 cursor 提供的方法来进行操作，方法主要是 1.执行命令 2.接收结果

ursor 用来执行命令的方法:
execute(query, args):执行单条 sql 语句,接收的参数为 sql 语句本身和使用的参数列表,返回值为受影响的行数
executemany(query, args):执行单挑 sql 语句,但是重复执行参数列表里的参数,返回值为受影响的行数

cursor 用来接收返回值的方法:
fetchall(self):接收全部的返回结果行.
fetchmany(size=None):接收 size 条返回结果行.如果 size 的值大于返回的结果行的数量,则会返回 cursor.arraysize 条数据.
fetchone():返回一条结果行.
scroll(value, mode='relative'):移动指针到某一行.如果 mode='relative',则表示从当前所在行移动 value 条,如果 mode='absolute',则表示从结果集的第一行移动 value 条.

先来看一下 execute 的增删改查的操作

```
 #创建数据库 51ctotest

>>> cur.execute('create database 51ctotest')
 #选择数据库 51ctotest

>>>con.select_db('51ctotest')
 #创建表 51cto,id 自增

>>>cur.execute('create table if not exists 51cto(id int(11) PRIMARY KEY AUTO_INCREMENT,name varchar(20),age int(3
 #插入一行数据,只给 name、age 赋值，让 id 自增

 #使用 sql 语句,这里要接收的参数都用 %s 占位符.要注意的是,无论你要插入的数据是什 # 么类型,占位符永远都要用 %s，后面的数值

>>>cur.execute("insert into 51cto(name,age) values(%s,%s)",('fan',25))
 #插入多行数据，用 executemany，它会循环插入后面元组中的所有值
```

```
>>> cur.executemany("insert into 51cto(name,age) values(%s,%s)",(('te',25),('fei',26),('musha',25)))
3L
 #查询

>>> cur.execute('select * from 51cto')
5L
 #我们使用了 fetchall 这个方法.这样,cds 里保存的将会是查询返回的全部结果.每条结果都是一个 tuple 类型的数据,这些 tuple 组成了

>>> cur.fetchall()
((1L, 'fan', 25L), (2L, 'fan', 25L), (3L, 'te', 25L), (4L, 'fei', 26L), (5L, 'musha', 25L))
 #再次查询，会看到查询不到结果，因为无论 fetchone、fetchall、fetchmany 指针是会发生移动的。所以，若不重置指针，那么使用

>>> cur.fetchall()
()
>>> cur.scroll(1,'absolute')
>>> for i in cur.fetchall():
...     print i
...
(2L, 'fan', 25L)
(3L, 'te', 25L)
(4L, 'fei', 26L)
(5L, 'musha', 25L)
```

这里有必要说一下 scroll：

```
cur.scroll(int,parm)
```

这里参数含义为：

int：移动的行数，整数；在相对模式下，正数向下移动，负值表示向上移动。

parm：移动的模式，默认 是 relative，相对模式；可接受 absoulte，绝对模式。

```
 #fetchone一次只取一行，指针下移  fetchmany（size）一次去 size 行

>>> cur.scroll(1,'absolute')
>>> cur.fetchone()
(2L, 'fan', 25L)
>>> cur.fetchmany(2)
((3L, 'te', 25L), (4L, 'fei', 26L))
 #普通取出是元组的形式，再从里面取值不好取，那怎么取成字典的格式呢，MySQLdb 中有 DictCursor，要做到这点也很简单，那就

>>> cur = con.cursor(cursorclass=MySQLdb.cursors.DictCursor)
>>> cur.execute('select * from 51cto')
5L
>>> for i in cur.fetchall():
```

```
...     print i
...
{'age': 25L, 'id': 2L, 'name': 'fan'}
{'age': 25L, 'id': 3L, 'name': 'te'}
{'age': 26L, 'id': 4L, 'name': 'fei'}
{'age': 25L, 'id': 5L, 'name': 'musha'}
 #更新，习惯 %s 的用法

>>> cur.execute('update 51cto set name=%s where id=3',('Mus'))
>>> cur.scroll(2,'absolute')
>>> cur.fetchone()
{'age': 25L, 'id': 3L, 'name': 'Mus'}
 #在执行完插入或删除或修改操作后,需要调用一下 conn.commit() 方法进行提交.这样,数据才会真正保 存在数据库中

>>> con.commit()
 #最后关闭游标，关闭连接

>>> cur.close()
>>> con.close()
```

14

python 监控文件或目录变化

我们经常会遇到监控一个文件或目录的变化，如果有变化，把文件上传备份至备份主机，并且我们还要监控上传过程是否有问题等，根据此需求，查阅了相关的一些材料，编写如下脚本实现这个功能：

```
#!/usr/bin/env python

#coding=utf-8

######################

#

#Status wd gs/ccs sql file changed

#date:2013-08-26  王伟

#文件有变化上传至备份主机，上传之后验证文件是否正确

#

######################

import paramiko,os,sys,datetime,time,MySQLdb
from pyinotify import WatchManager, Notifier, ProcessEvent, IN_DELETE, IN_CREATE,IN_MODIFY
'''
CREATE TABLE `wddel_log.status_sql` (
  `ip` varchar(16) NOT NULL COMMENT '机器IP',
  `tar_name` varchar(50) NOT NULL COMMENT '备份文件名字',
  `md5` varchar(50) NOT NULL COMMENT '备份文件 MD5',
  `flag` int(2) NOT NULL COMMENT '0:成功;1:失败',
  `error_log` varchar(100) NOT NULL COMMENT '错误日志',
  `uptime` datetime NOT NULL COMMENT '更新时间',
  KEY `ip` (`ip`),
  KEY `uptime` (`uptime`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8'''#日志表创建脚本
GM_path='/home/asktao/'
center_hostname='192.168.1.100'
center_username='root'
center_password='123456'
center_port=63008
def log2db(ip,tar_name,md5,flag,error='0'):#删除日志入库
    try:
        tar_name = os.path.split(tar_name)[1]
        now  = time.strftime("%Y-%m-%d %H:%M:%S")
        conn = MySQLdb.connect(host = '192.168.1.104',user = 'root',passwd = '1q2w3e4r',charset='utf8',connect_timeout=20
        cursor = conn.cursor()
```

```
        sql = "SELECT ip FROM wddel_log.status_sql WHERE ip='%s'" % ip
        cursor.execute(sql)
        res = cursor.fetchall()
        if len(res)==0:
            inster_sql = "insert into wddel_log.status_sql VALUES('%s','%s','%s',%s,'%s','%s')" % (ip,tar_name,md5,flag,error_
            cursor.execute(inster_sql)
            conn.commit()
        else:
            update_sql = "UPDATE wddel_log.status_sql SET md5='%s',flag='%s',error_log='%s',uptime='%s' WHERE ip='%s
            cursor.execute(update_sql)
            conn.commit()
        cursor.close()
        conn.close()
    except Exception,e:
        print e
def find_ip():#获取本地 eth0 的 IP 地址
    ip = os.popen("/sbin/ip a|grep 'global eth0'").readlines()[0].split()[1].split("/")[0]
    if "192.168." in ip:
        ip = os.popen("/sbin/ip a|grep 'global eth1'").readlines()[0].split()[1].split("/")[0]
    return ip
def md5sum(file_name):#验证 sql 打包文件的 MD5
    if os.path.isfile(file_name):
        f = open(file_name,'rb')
        py_ver = sys.version[:3]
        if py_ver == "2.4":
            import md5 as hashlib
        else:
            import hashlib
            md5 = hashlib.md5(f.read()).hexdigest()
            f.close()
            return md5
    else:
        return 0
def center_md5(file_name):#上传至备份中心的文件的 MD5
    try:
        s=paramiko.SSHClient()
        s.set_missing_host_key_policy(paramiko.AutoAddPolicy())
        s.connect(hostname = center_hostname,port=center_port,username=center_username, password=center_passwo
        conm = "/usr/bin/md5sum %s" % file_name
        stdin,stdout,stderr=s.exec_command(conm)
        result = stdout.readlines()[0].split()[0].strip()
        s.close()
        return result
    except Exception,e:
        return e
```

```
def back_file(ip,tar_name,tar_md5):#上传文件到备份中心
    remote_dir='/data/sql'
    file_name=os.path.join(remote_dir,os.path.split(tar_name)[1])
    try:
        t=paramiko.Transport((center_hostname,center_port))
        t.connect(username=center_username,password=center_password)
        sftp=paramiko.SFTPClient.from_transport(t)
        sftp.put(tar_name,file_name)
        t.close()
        #print "%s back_file OK" % tar_name
        os.remove(tar_name)
        remot_md5=center_md5(file_name)
        if remot_md5 == tar_md5:
            log2db(ip,tar_name,tar_md5,0)
        else:
            log2db(ip,tar_name,tar_md5,1,'remot_md5!=tar_md5')
    except Exception,e:
        #print "connect error!"
        log2db(ip,tar_name,tar_md5,1,e)
        os.remove(tar_name)
def back_sql():#执行备份
    ip = find_ip()
    tar_name = "/tmp/%s.tar.gz" % ip
    sql_conn = "/usr/bin/find %s -type f  -name '*.sql'|/usr/bin/xargs /bin/tar zcvPf %s" % (GM_path,tar_name)
    sql_tar = os.popen(sql_conn).readlines()
    tar_md5 = md5sum(tar_name)
    if tar_md5 != 0:
        back_file(ip,tar_name,tar_md5)
    else:
        error_log =  "%s not find" % tar_name
        log2db(ip,tar_name,tar_md5,0,error_log)
class PFilePath(ProcessEvent):#文件变化的触发
    def process_IN_CREATE(self, event):
        if os.path.splitext(event.name)[1] == ".sql":
            text = "Create file: %s " % os.path.join(event.path, event.name)
            #print text
            back_sql()
    def process_IN_MODIFY(self, event):
        if os.path.splitext(event.name)[1] == ".sql":
            text = "Modify file: %s " % os.path.join(event.path, event.name)
            #print text
            back_sql()
def FSMonitor():#主监控函数
    back_sql()#运行脚本先备份 sql 文件
    wm = WatchManager()
```

```
    mask = IN_CREATE |IN_MODIFY
    notifier = Notifier(wm, PFilePath())
    wdd = wm.add_watch(GM_path, mask, rec=True)
    print 'now starting monitor %s' % (GM_path)
    while True:
        try :
            notifier.process_events()
            if notifier.check_events():
                notifier.read_events()
        except KeyboardInterrupt:
            notifier.stop()
            break
if __name__ == "__main__":
    FSMonitor()
```

此脚本中主要用到 paramiko 和 pyinotify 模块，关于 paramiko 的讲解可以参见：http://wangwei007.blog.51cto.com/68019/1058726一文，pyinotify 的用法可以参见官方文档：https://github.com/seb-m/pyinotify/wiki/Events-types

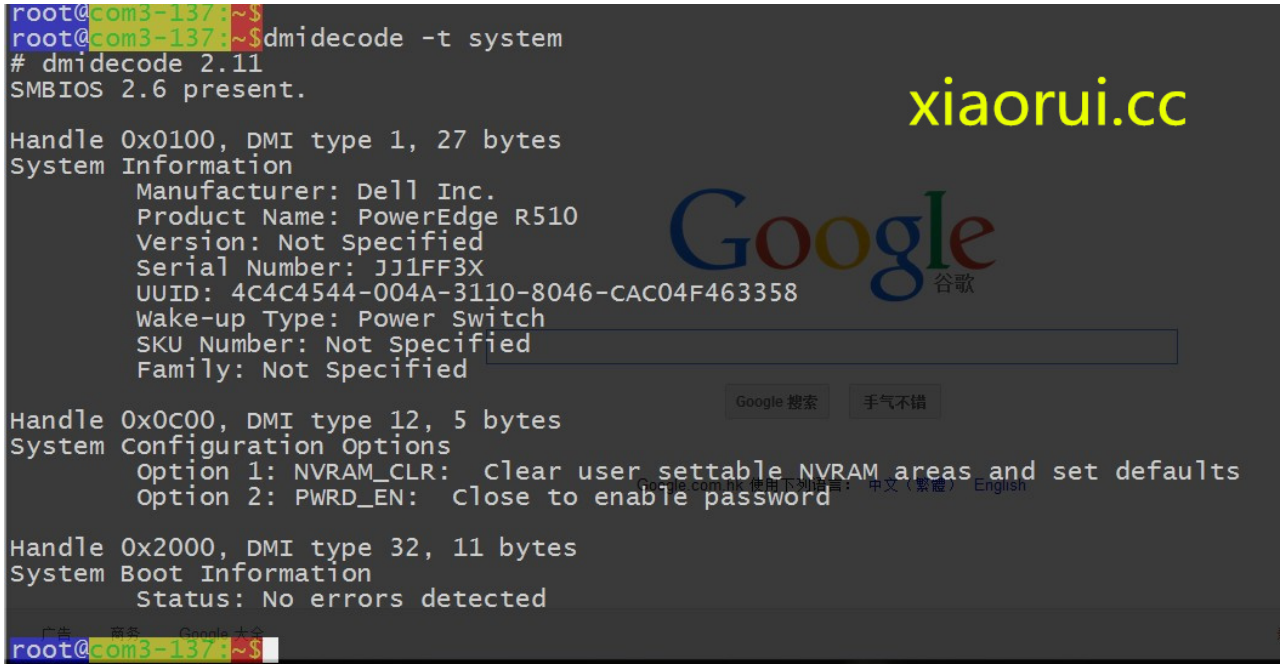
15

Mongodb 千万级数据在 python 下的综合压力测试及应用探讨

曾经在收集数据的项目中，用过 mongodb 的数据存储，但是当数据很大的时候，还是比较的吃力。很可能当时的应用水平不高，也可以是当时的服务器不是很强。 所以这次能力比以前高点了，然后服务器比以前也高端了很多，好嘞 ~再测试下。
（更多的是单机测试，没有用复制分片的测试 ~ ）！

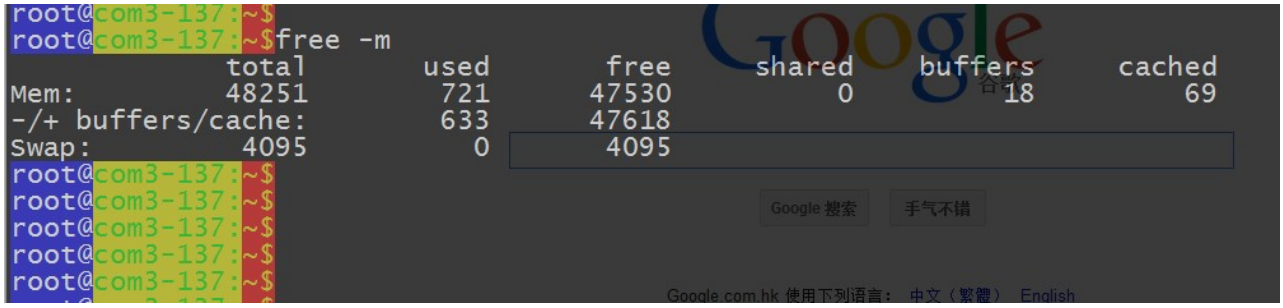
相比较 MySQL，**MongoDB 数据库更适合那些读作业较重的任务模型**。MongoDB 能充分利用机器的内存资源。如果机器的内存资源丰富的话，MongoDB 的查询效率会快很多。

这次测试的服务器是 dell 的 r510!



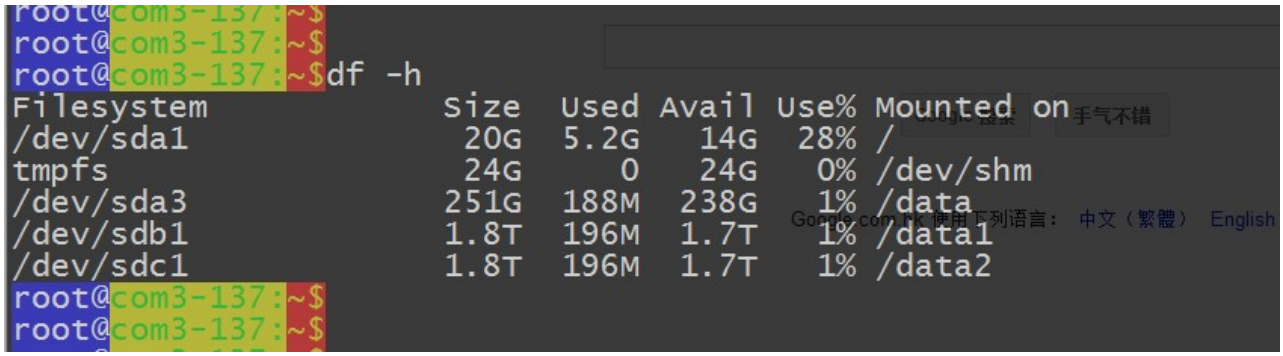
图片 15.1 pic

内存还行，是 48 G 的，本来想让同事给加满，但是最终还是没有说出口 ~



图片 15.2 pic

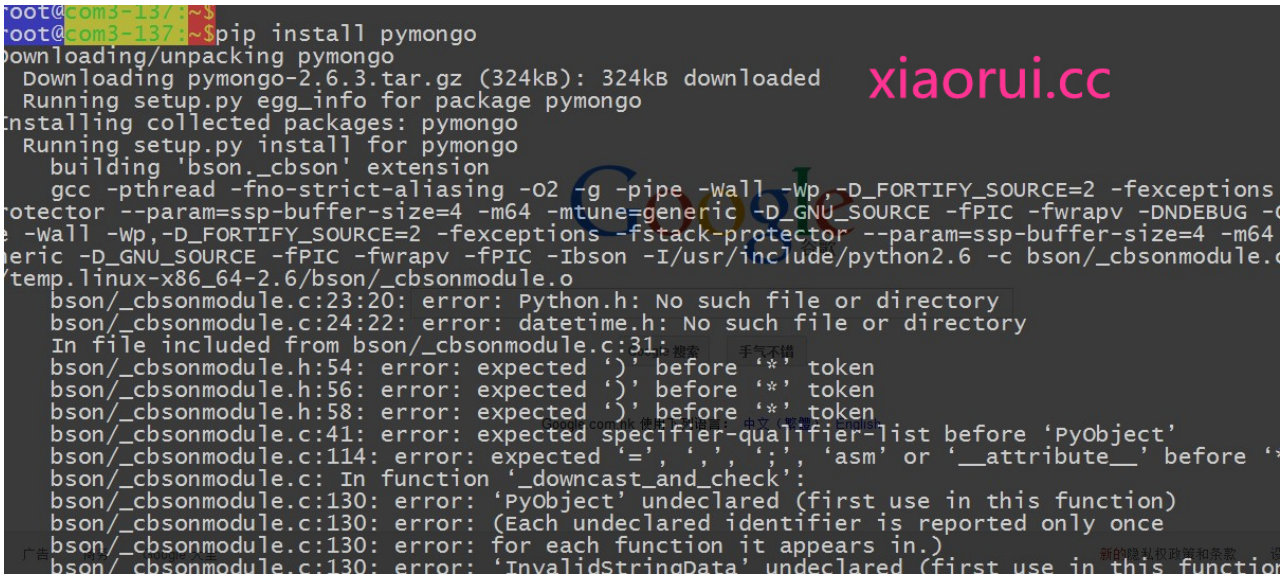
磁盘是 10 个 2T 的，但是因为格式化的时间太久了，哥们直接把其他的硬盘给拔出来了，就用了三个盘。。。data 目录没有做 raid，是为了让他们体现更好的硬盘速度。



图片 15.3 pic

既然说好了是在 python 下的应用测试，那就需要安装 mongodb python 下的模块！ 对了，不知道 mongodb-server 的安装要不要说下?

```
cat /etc/yum.repos.d/10.repo
[10gen]
name=10gen Repository
baseurl=http://downloads-distro.mongodb.org/repo/redhat/os/x86_64
gpgcheck=0
```



图片 15.4 pic

Pymongo 的基本用法

```
from pymongo import * # 导包
con = Connection(...) # 链接
db = con.database # 链接数据库
db.authenticate('username', 'password') # 登录
db.drop_collection('users') #删除表
db.logout() # 退出
db.collection_names() # 查看所有表
```

```
db.users.count() # 查询数量
db.users.find_one({'name' : 'xiaoming'}) # 单个对象
db.users.find({'age' : 18}) # 所有对象
db.users.find({'id':64}, {'age':1,'_id':0}) # 返回一些字段 默认_id总是返回的 0不返回 1返回
db.users.find({}).sort({'age': 1}) # 排序
db.users.find({}).skip(2).limit(5) # 切片
```

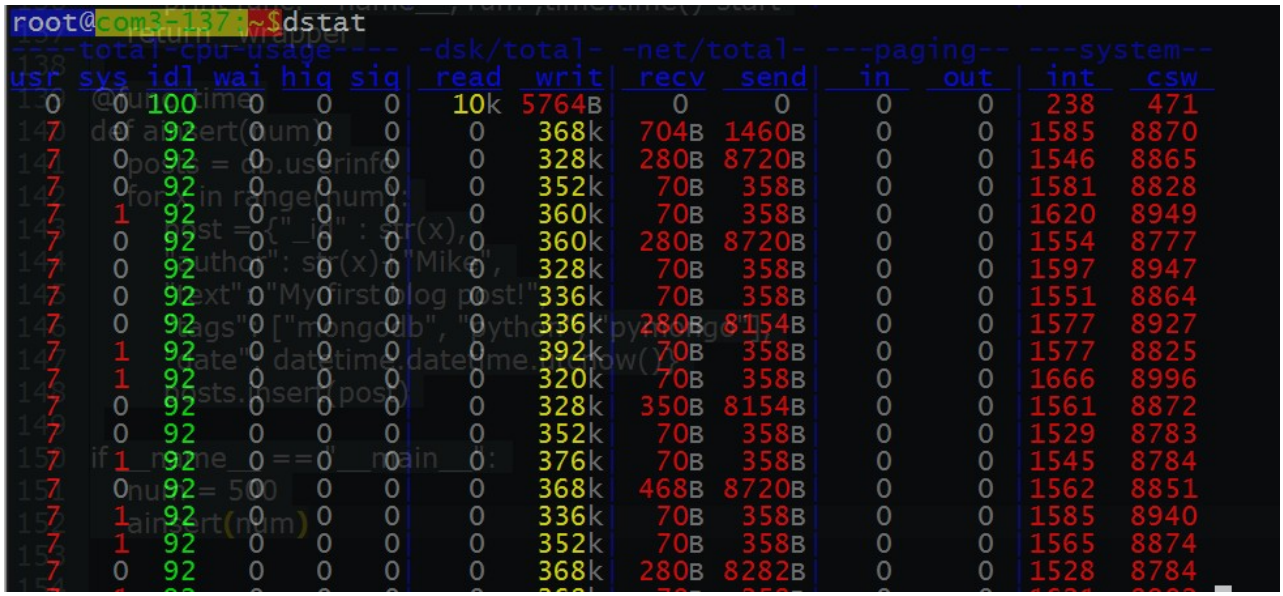
测试的代码：

```
 #!/usr/bin/env python

from pymongo import Connection
import time,datetime
import os,sys
connection = Connection('127.0.0.1', 27017)
db = connection['xiaorui']
def func_time(func):
    def _wrapper(*args,**kwargs):
        start = time.time()
        func(*args,**kwargs)
        print func.__name__,'run:',time.time()-start
    return _wrapper
@func_time
def ainsert(num):
    posts = db.userinfo
    for x in range(num):
        post = {"_id" : str(x),
        "author": str(x)+"Mike",
        "text": "My first blog post!",
        "tags": ["xiaorui", "xiaorui.cc", "rfyiamcool.51cto"],
        "date": datetime.datetime.utcnow()}
        posts.insert(post)
if __name__ == "__main__":
    num = sys.argv[1]
    ainsert(int(num))
```

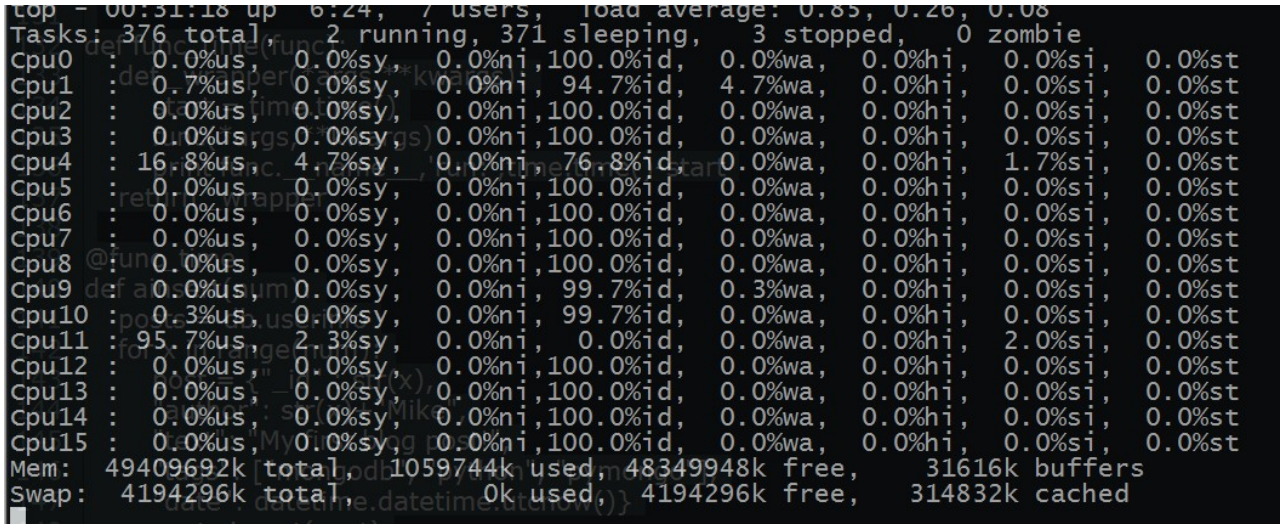
咱们就先来个百万的数据做做测试~

综合点的数据：



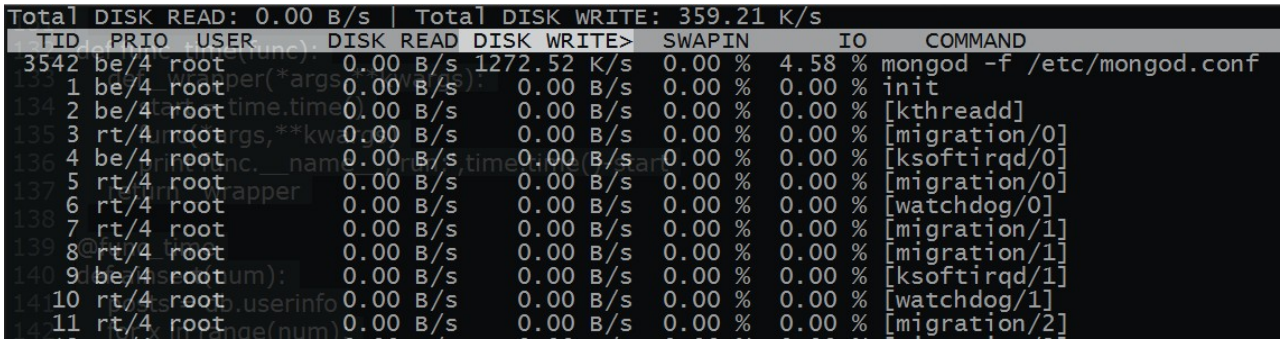
图片 15.5 pic

在 top 下看到的程序占用资源的情况 ~ 我们看到的是有两个进程的很突出，对头！ 正是 mongodb 的服务和我们正在跑的 python 脚本!



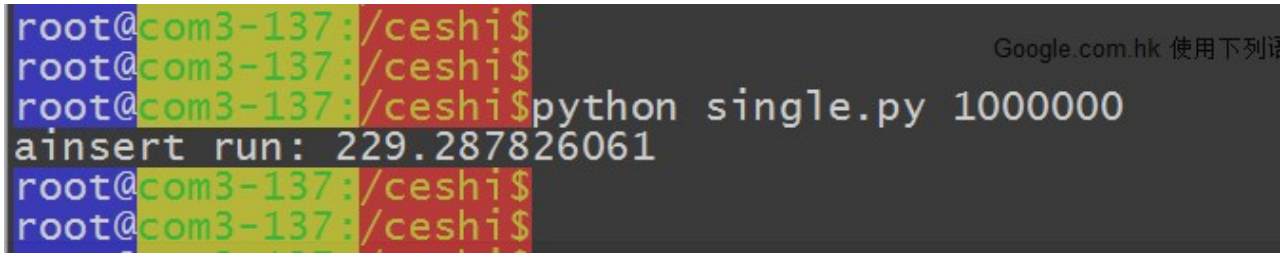
图片 15.6 pic

看下服务的 io 的情况 ~



图片 15.7 pic

脚本运行完毕，总结下运行的时间 ~



图片 15.8 pic

查看 mongodb 的状态~

他的 insert 也不到 5k ~ 插入量也就 800k 左右 ~

它的输出有以下几列：

inserts/s 每秒插入次数

query/s 每秒查询次数

update/s 每秒更新次数

delete/s 每秒删除次数

getmore/s 每秒执行 getmore 次数

command/s 每秒的命令数，比以上插入、查找、更新、删除的综合还多，还统计了别的命令

flushs/s 每秒执行 fsync 将数据写入硬盘的次数。

mapped/s 所有的被 mmap 的数据量，单位是 MB，

vsize 虚拟内存使用量，单位 MB

res 物理内存使用量，单位 MB

faults/s 每秒访问失败数（只有 Linux 有），数据被交换出物理内存，放到 swap。不要超过 100，否则就是机器内存太小，造成频繁 swap 写入。此时要升级内存或者扩展

locked % 被锁的时间百分比，尽量控制在 50% 以下吧

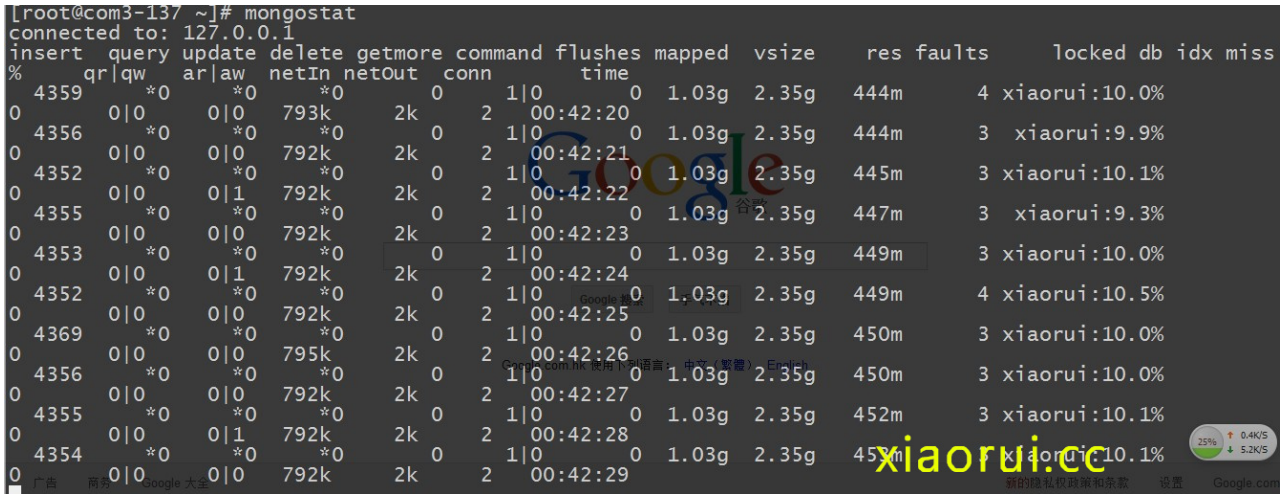
idx miss % 索引不命中所占百分比。如果太高的话就要考虑索引是不是少了

q t|r|w 当 Mongodb 接收到太多的命令而数据库被锁住无法执行完成，它会将命令加入队列。这一栏显示了总共、读、写 3 个队列的长度，都为 0 的话表示 mongo 毫无压力。高并发时，一般队列值会升高。

conn 当前连接数

time 时间戳

瞅下面的监控数据！



图片 15.9 pic

然后我们在测试下在一千万的数据下的消耗时间情况 ~

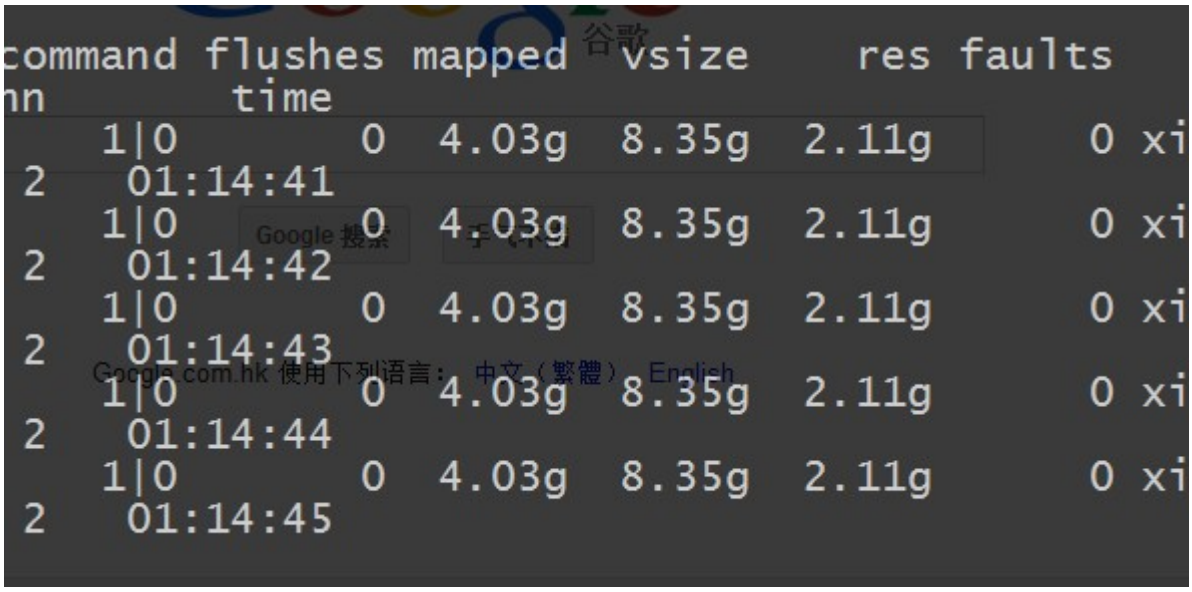
共用了 2294 秒，每秒插入 4359 个数据 ~



图片 15.10 pic

看看他的内存的使用情况：

虚拟内存在 8 gb 左右，真实内存在 2 gb 左右



图片 15.11 pic

再换成多线程的模式跑跑 ~ 个人不太喜欢用多线程，这东西属于管你忙不忙，老大说了要公平，我就算抢到了，但是没事干，我也不让给你。。。属于那种蛮干的机制 ~

nima，要比单个跑的慢呀 ~ 线程这东西咋会这么不靠谱呀 ~
应该是没有做线程池 pool，拉取队列。导致线程过多导致的。不然不可能比单进程都要慢~
还有就是像这些涉及到 IO 的东西，交给协程的事件框架更加合理点！！！

```
def goodinsert(a):
    posts.insert(a)
def ainsert(num):
    for x in range(num):
        post = {"_id" : str(x),
        "author": str(x)+"Mike",
        "text": "My first blog post!",
        "tags": ["mongodb", "python", "pymongo"],
        "date": datetime.datetime.utcnow()}
#       goodinsert(post)

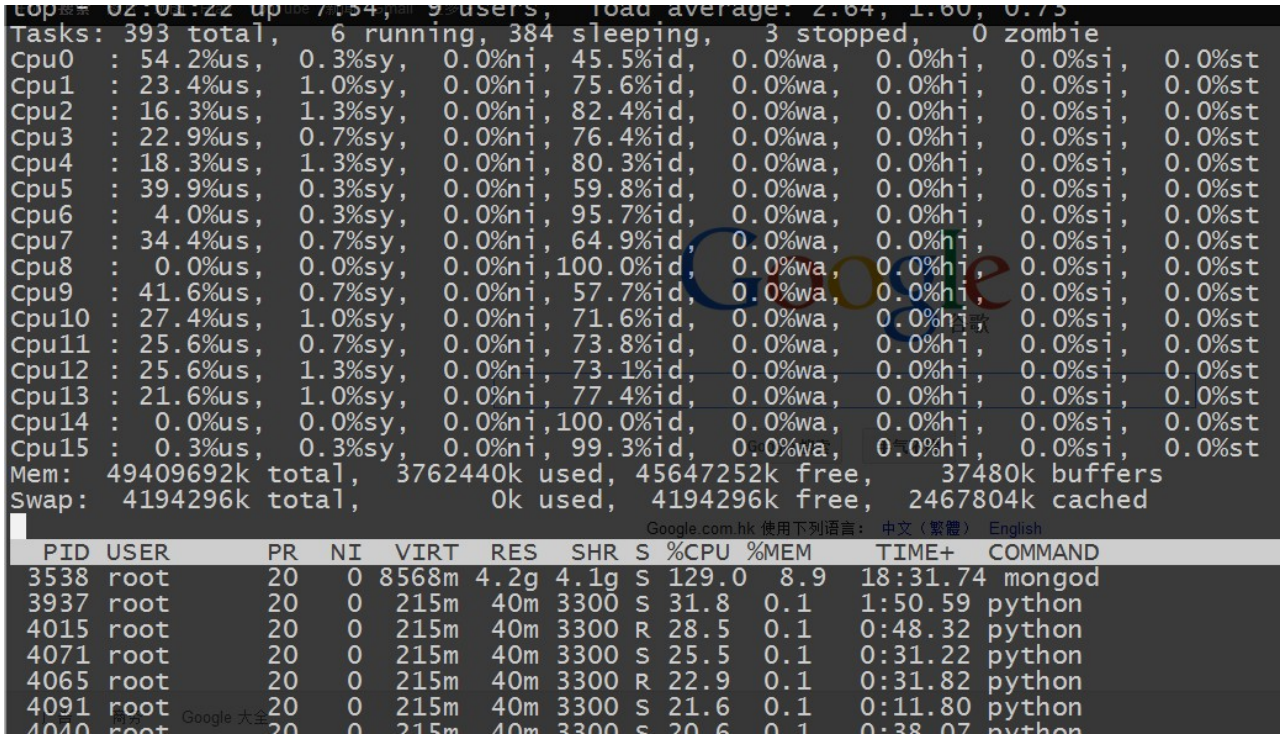
        a=threading.Thread(target=goodinsert,args=(post,))
        a.start()
```



图片 15.12 pic

python 毕竟有 gil 的限制，虽然 multiprocess 号称可以解决多进程的。但是用过的朋友知道，这个东西更不靠谱 ~ 属于坑人的东西 ~

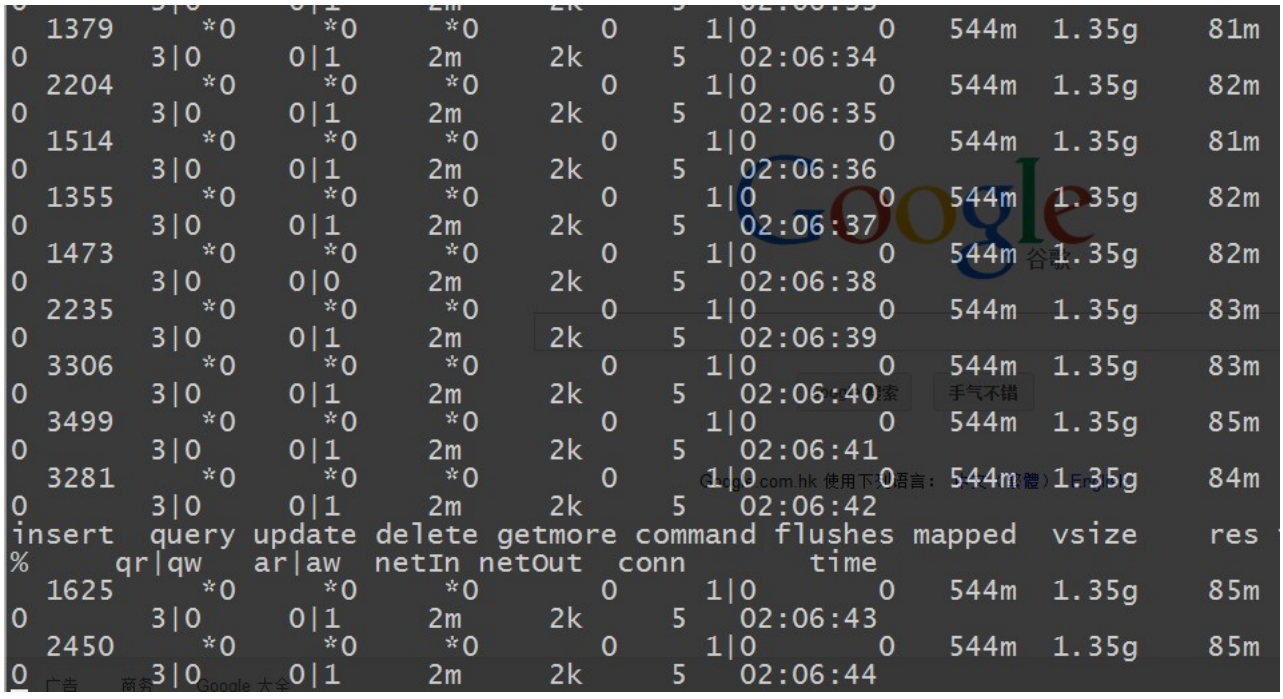
要是有朋友怀疑是 python 的单进程的性能问题，那咱们就用 supervisord 跑了几个后台的 python 压力脚本 ~ supervisord 的配置我就不说了，我以前的文章里面有详述的 ~



图片 15.13 pic

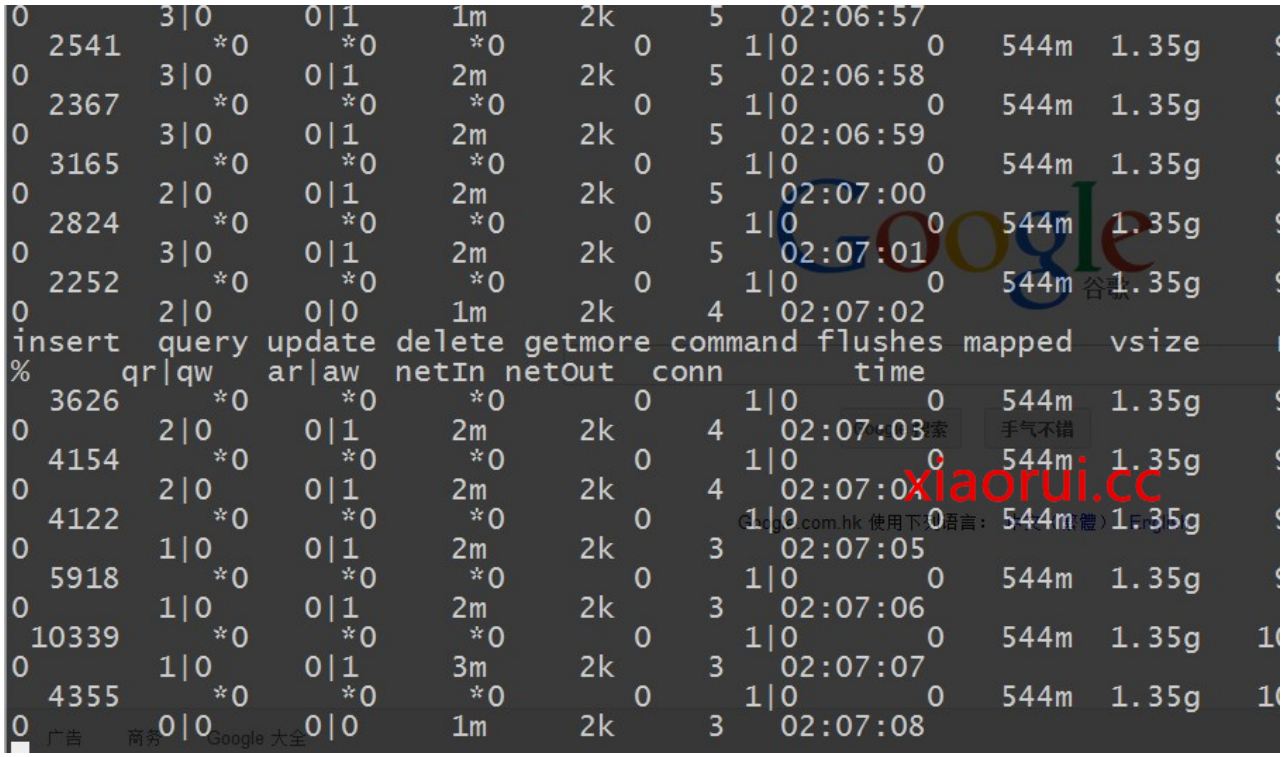
cpu 方面是跑的有点均匀了，但是 mongodb 那边的压力总是上不去

当加大到 16 个后台进程做压力测试的时候 ~ 大家会发现 insert 很不稳定。 看来他的极限也就是 2 MB 左右的数据 ~



图片 15.14 pic

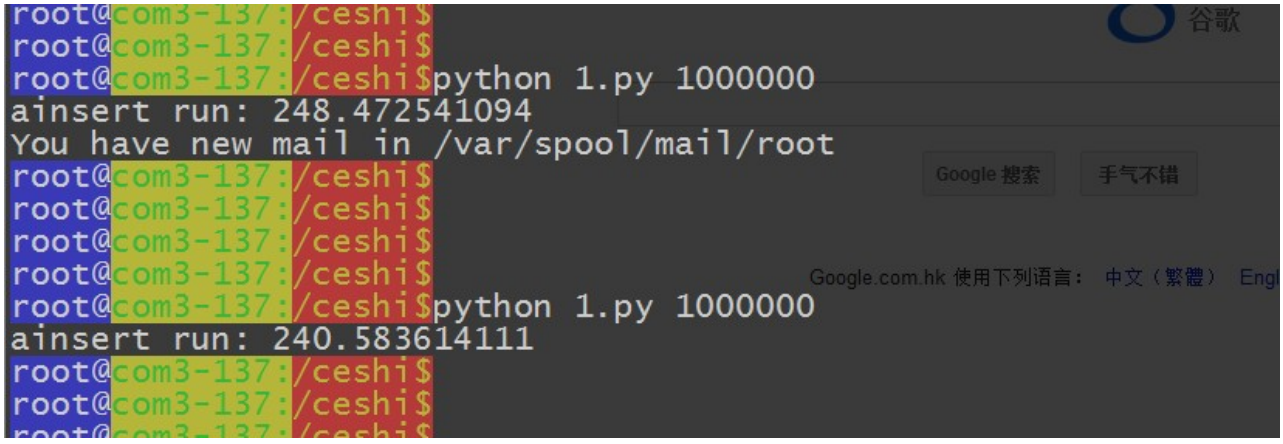
当减少到 8 个压力进程的时候 ~ 我们发现他的 insert 慢慢的提供到正常了，也就是说 他真的是 2 MB 的极限 ~



图片 15.15 pic

脚本里面是有做有序的 id 插入的，我们试试把 id 的插入给去掉，看看有没有提升~

结果和不插入 id 差不多的结果 ~



图片 15.16 pic

调优之后 ~ 再度测试

ulimit 的优化

```
cat /etc/security/limits.conf
*     soft   nofile      102400
*     hard   nofile      102400
```

内核的 tcp 优化

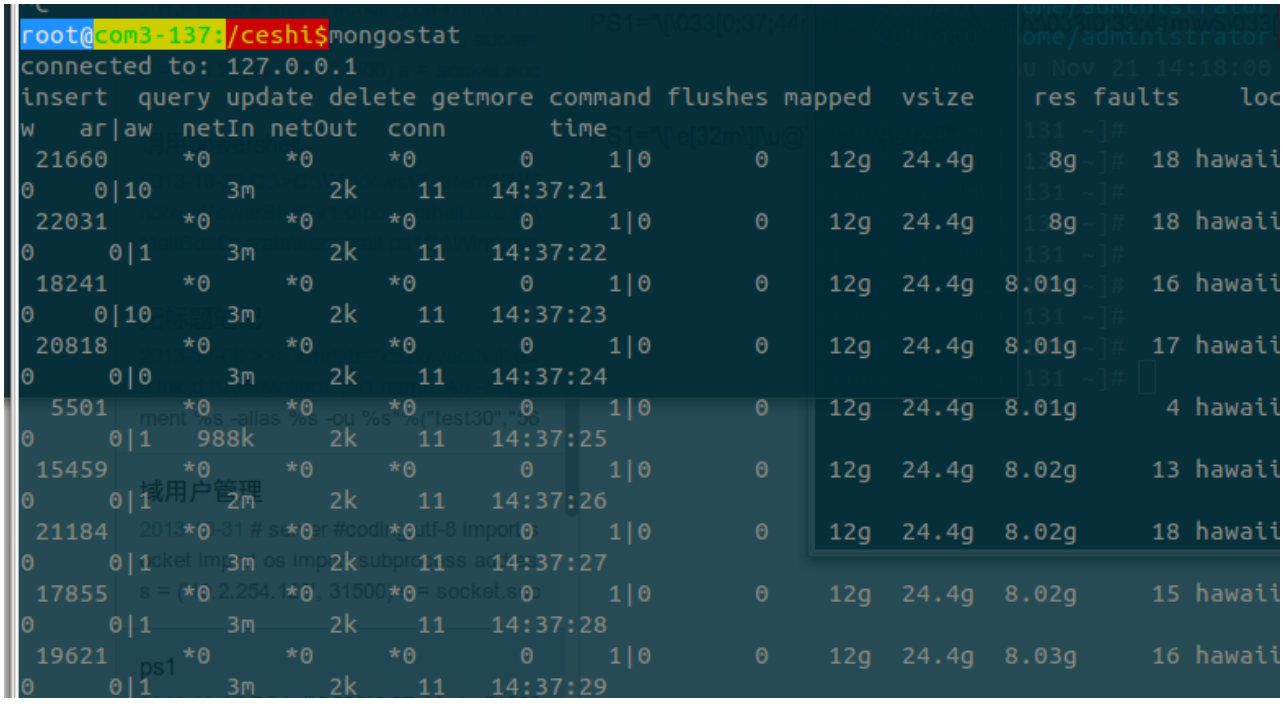
```
cat /etc/sysctl.conf
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_tw_recycle = 1
net.ipv4.tcp_timestsmps = 0
net.ipv4.tcp_synack_retries = 2
net.ipv4.tcp_syn_retries = 2
net.ipv4.tcp_wmem = 8192 436600 873200
net.ipv4.tcp_rmem = 32768 436600 873200
net.ipv4.tcp_mem = 94500000 91500000 92700000
net.ipv4.tcp_max_orphans = 3276800
net.ipv4.tcp_fin_timeout = 30
 #直接生效

/sbin/sysctl -p
```

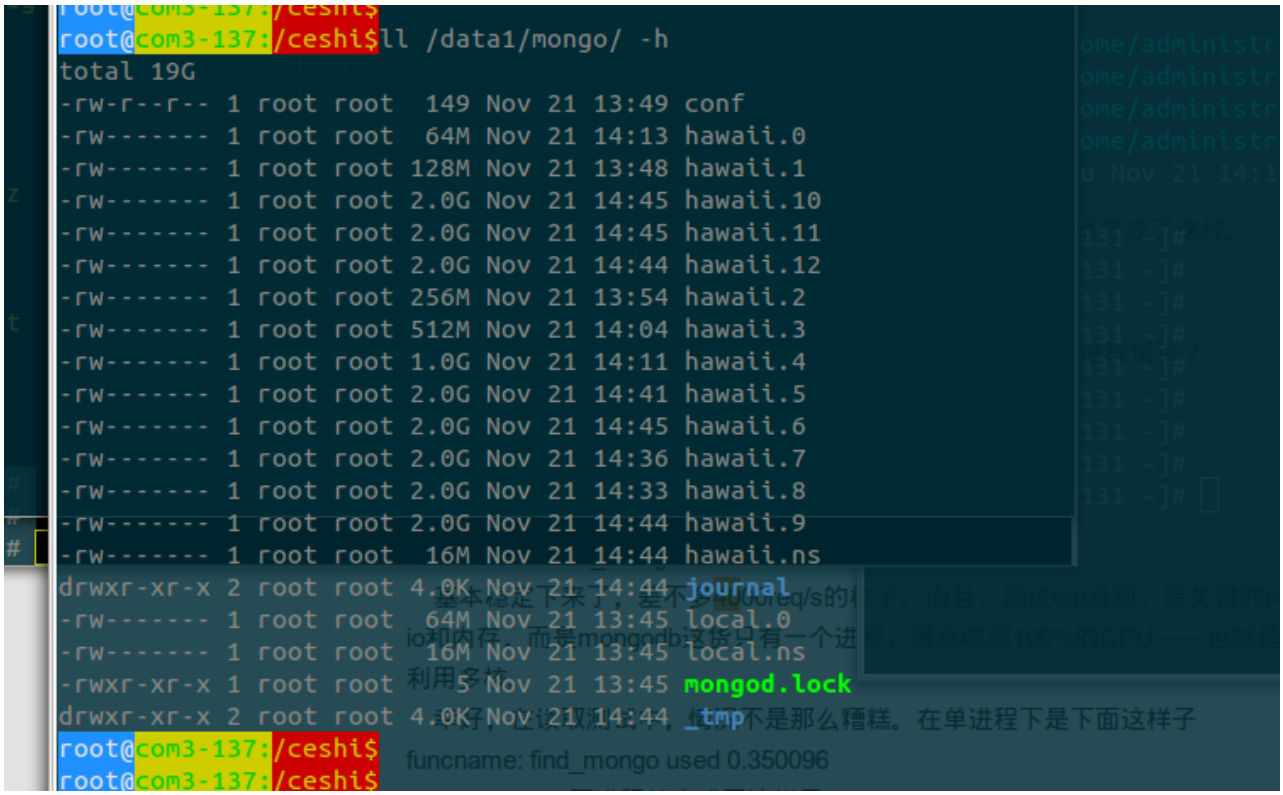
启动的时候，加上多核的优化参数

```
多核问题可以在启动时加入启动参数： numactl --interleave=all
```

insert 的频率已经到了 2 w 左右 ~ 内存占用了 8 G 左右 ~



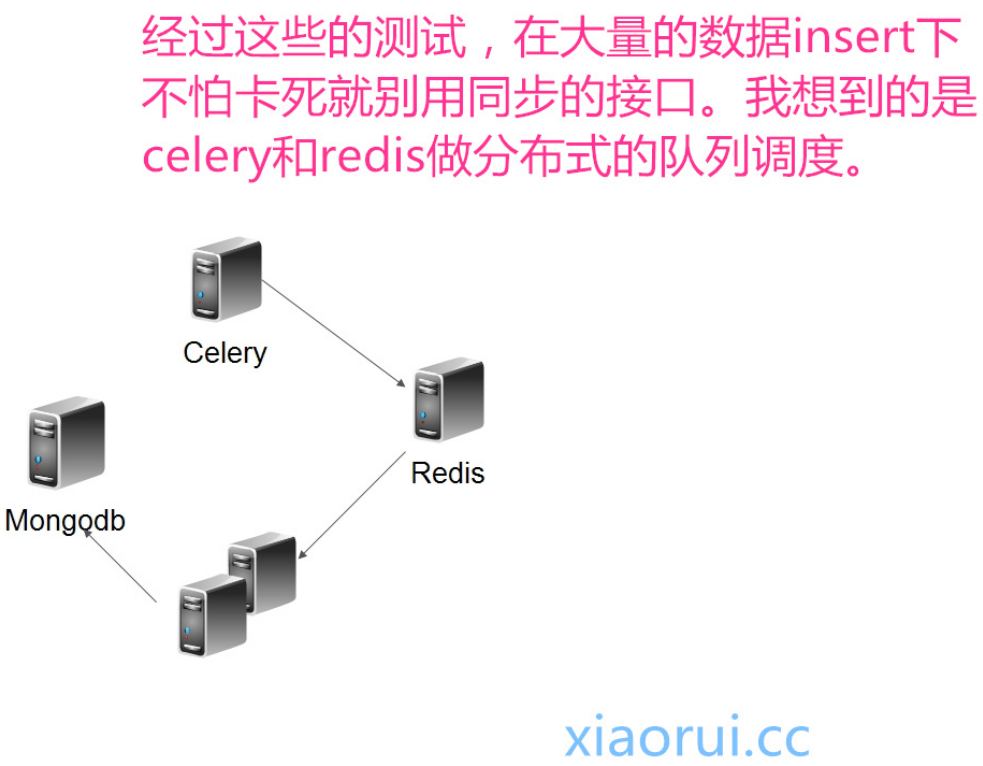
图片 15.17 pic



图片 15.18 pic

我想到的一个方案：

当然不用非要 celery，就算咱们用 socket 写分发，和 zeromq 的 pub sub 也可以实现这些的。这是 celery 的调度更加专业点。

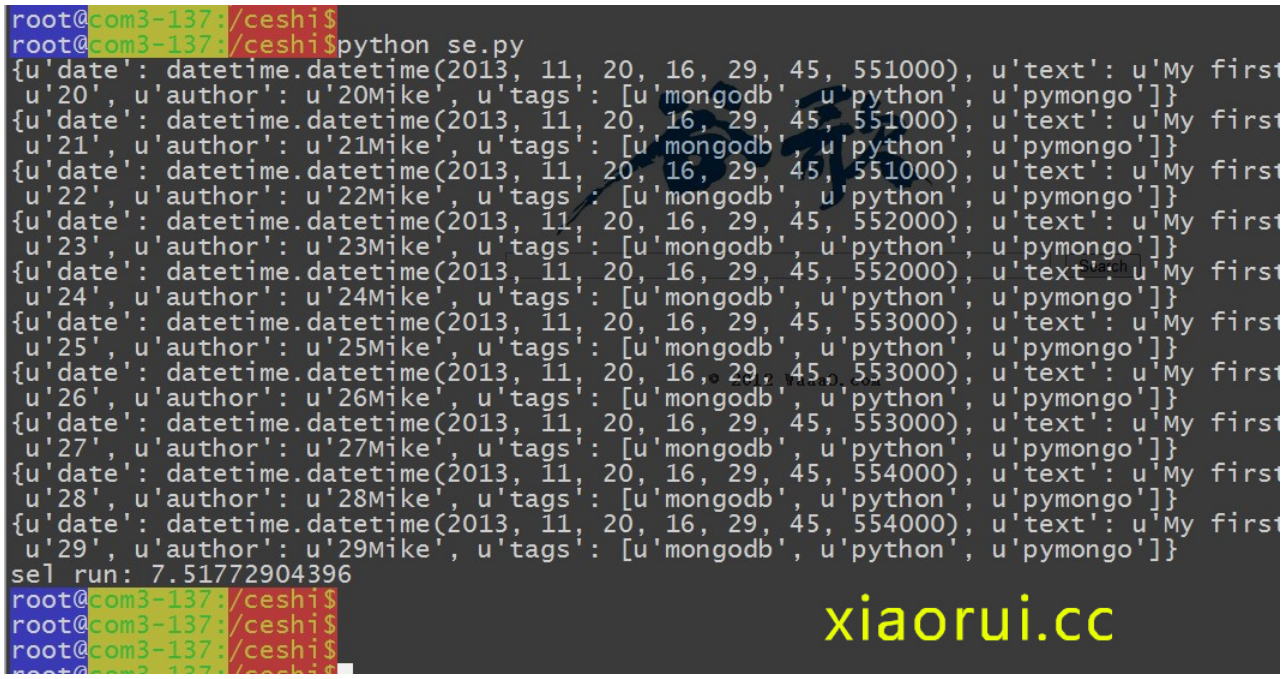


图片 15.19 pic

刚才我们测试的都是insert，现在我们再来测试下在千万级别数据量下的查询如何：

查询正则的，以2开头的字符

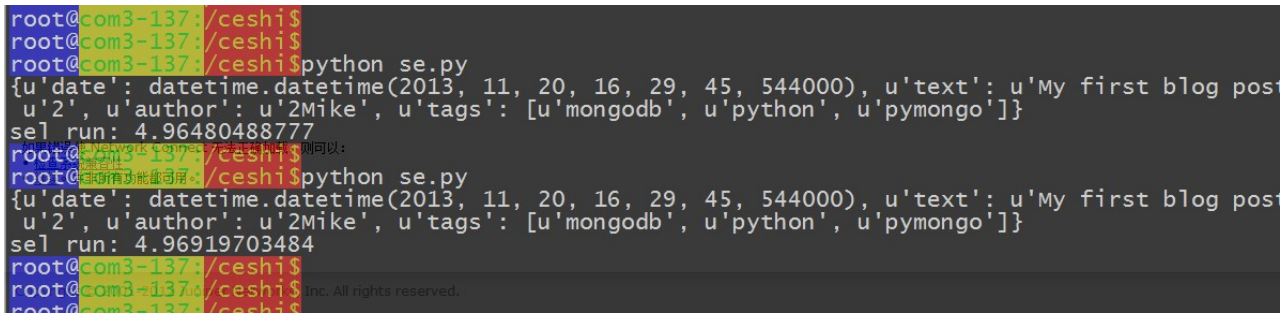
```
posts = db.userinfo
for i in posts.find({"author":re.compile('^2.Mike')}):
    print i
```



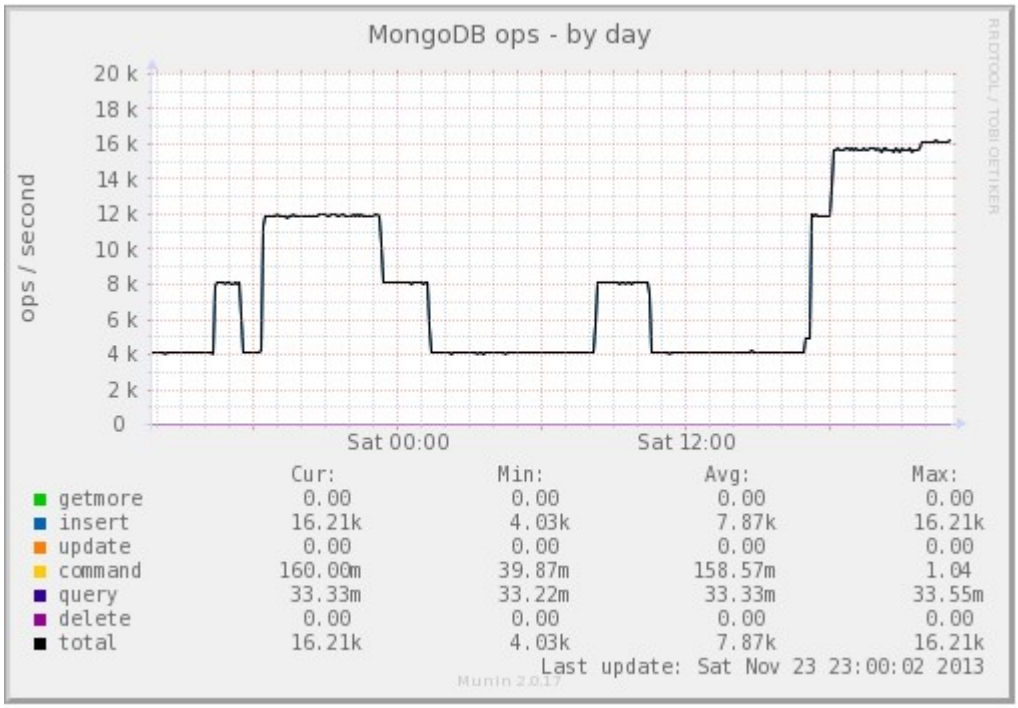
图片 15.20 pic

精确的查询：

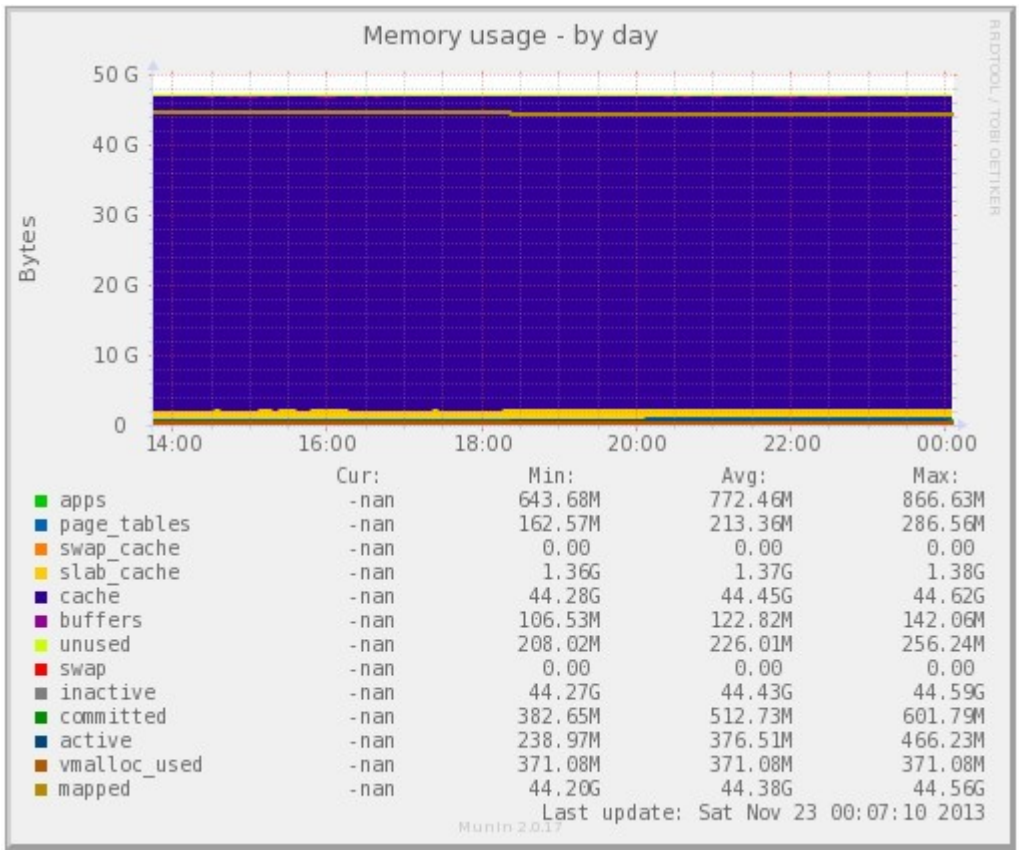
查询在 5s 左右 ~



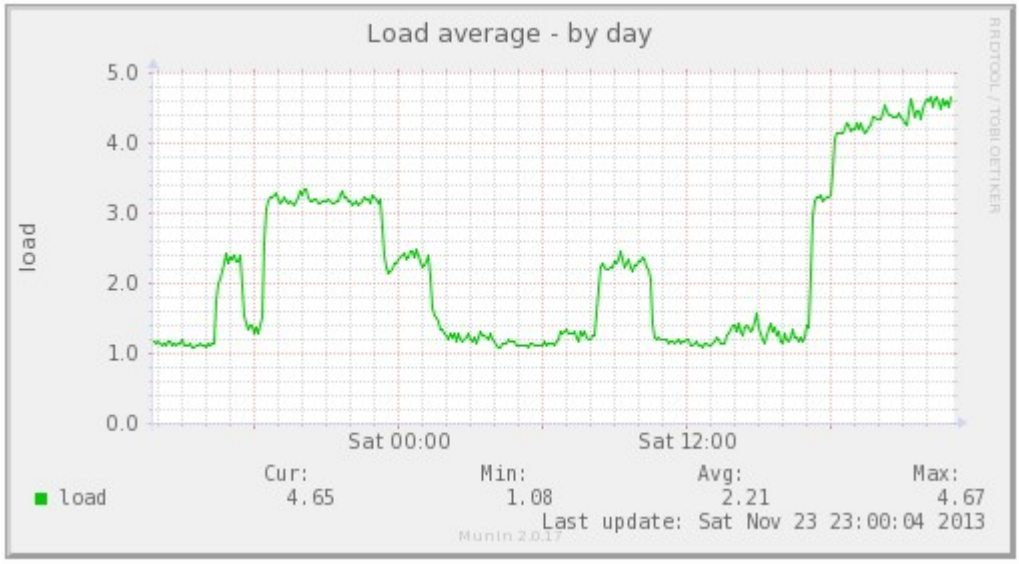
图片 15.21 pic



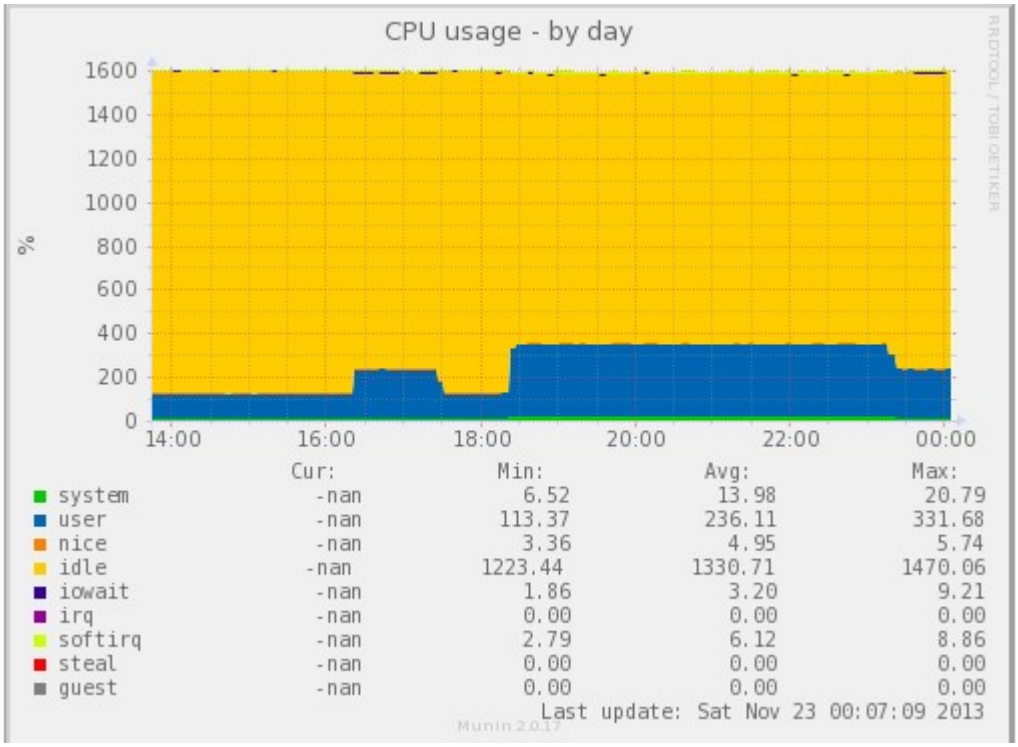
图片 15.22 pic



图片 15.23 pic



图片 15.24 pic



图片 15.25 pic

总结:

典型的高读低写数据库!

本文出自 “峰云，就她了。” 博客，谢绝转载!

16

通过 memcached 实现领号排队功能及 python 队列实例

前言：

前段时间写的那个域用户平台，要做大量的新功能运维测试，据说要抄 IT 组，让那帮人到搞，跑一下！ 尼玛，这可吓坏了我了。 因为平台要和 windows 做大量的交互，那边 powershell 又很不给力，改成多线程版本后，出现莫名的问题，很让人闹心。现在的状态是，client 给 server 端可以同时推送两片信息，要是多的话，powershell 实在处理不了，我只能放到 queue 队列里面了。

现在很多的堵塞都是在 windows 那边，我这边因为用的是 tornado，对于用户访问是无压力的，但是 windows 那边不能同时运行多了，不然会提示 bug。。。

ad 的信息我暂时还没有批量的同步过来，所以只能单点搞了 ~

一直在想咋才能不出丑。所以做了好多的限制，比如短信接口的 token 机制，用户更新接口的次数的限制。 现在唯一的节点就是和 win 那边的交互。别等到了周一的时候，一帮人把 获取手机号码、修改密码、更新用户信息的接口给点爆了。

突然想到 12306 那个渣渣，可以用排队呀。。。 这样的话，最少能看起来很高端的样子。



图片 16.1 pic

我的前端实现 ~

用户点击的时候，我会从后端的 api 查看队列中的数目，以及有谁排在我的前面 ~

```
    $("#dialog").hide();
    $("#mailname").focus();
    $("#service").click(function(){
            $.ajax({
             type: "POST",
             url: "/queue",
             data : $("#form_service").serialize(),
             dataType: "html",
              timeout:5000,
             error: function(){
                 alert('nima，超时了');
             },
             success: function(data,status){
               if( data=="ok"){
                 <!--  alert (data+status+"成功了");-->
                   var a=$("input[name=mailname]").val();
                   window.location.href="/mailpost?mailname="+a; }
               else{
                    $('#myModal').modal();
                   }
               }
            });
    });
});
```

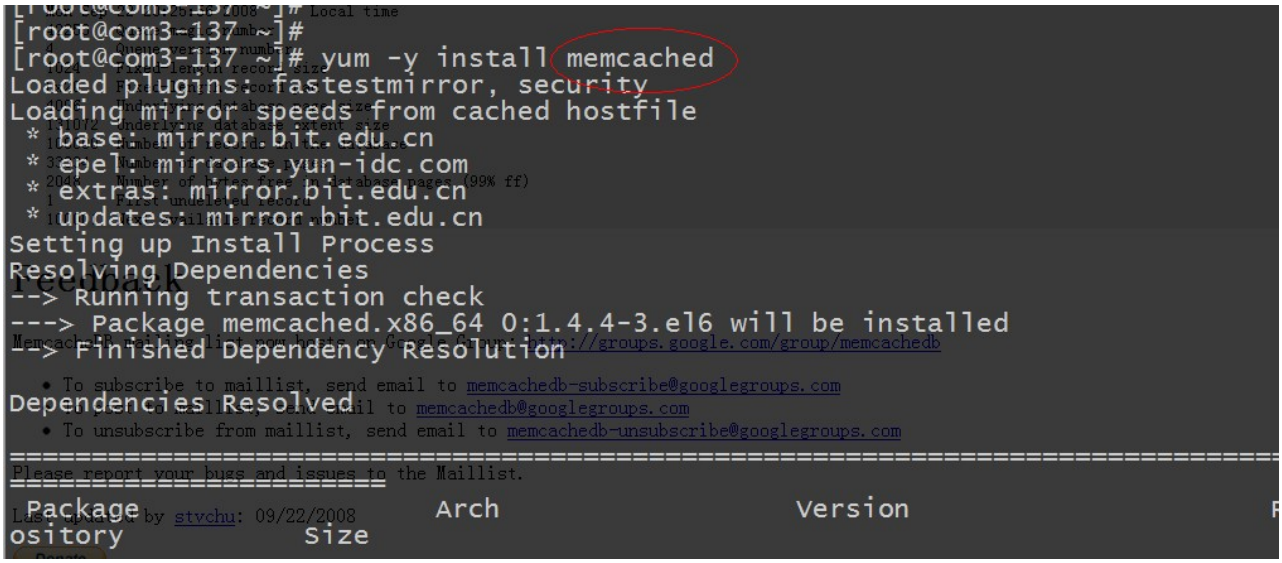
后端的实现~

不用 redis 做队列的原因是，python 调用队列的时候总是莫名的关闭，卸载安装了好多遍。。。怪事 ~ 和 powershell 多线程一样都很怪~

安装配置 memcached 环境，简单点直接 yum ~

需要编译安装的朋友，用下面的脚本~

```
wget https://github.com/downloads/libevent/libevent-2.0.21-stable.tar.gz
tar xzf libevent-2.0.21-stable.tar.gz
cd libevent-2.0.21-stable
./configure
make
make install
wget http://memcached.googlecode.com/files/memcached-1.4.15.tar.gz
tar vxzf memcached-1.4.15.tar.gz
cd memcached-1.4.15
./configure --prefix=/usr/local/webserver/memcached
make
make install
```



图片 16.2 pic

启动 memcached 命令是： /usr/local/memcached/bin/memcached -d -m 100 -c 1000 -u root -p 11211

-d 选项是启动一个守护进程
-m 是分配给 Memcache 使用的内存数量，单位是 MB，默认 64 MB
-M return error on memory exhausted (rather than removing items)
-u 是运行 Memcache 的用户，如果当前为 root 的话，需要使用此参数指定用户
-l 是监听的服务器 IP 地址，默认为所有网卡
-p 是设置 Memcache 的 TCP 监听的端口，最好是 1024 以上的端口
-c 选项是最大运行的并发连接数，默认是 1024
-P 是设置保存 Memcache 的 pid 文件
-f chunk size growth factor (default: 1.25)
-I Override the size of each slab page. Adjusts max item size(1.4.2 版本新增)

有朋友可能没有接触过 memcached，也有没有用 python 操作 memcached 的。 我在这里就简单操作下，让大家瞅瞅哈~

python 操作 memcached 需要安装 python-memcached 模块

pip install python-memcached

```
import memcache
 mc=memcache.Client(['127.0.0.1:11211'],debug=0)
 mc.set(“xiaorui.cc”,”fengyun”)
 value=mc.get(“xiaorui.cc”)
 mc.set(“another_key”,3)
 mc.delete(“another_key)
 mc.set(“key”,”1″) #用于自动增量/减量的必须是字符串
 mc.incr(“key”)
 mc.decr(“key”)
```

```
标准的使用 memcache 作为数据库缓存的方法如下:
key=derive_key(obj)
obj=mc.get(key)
if not obj:
obj=backend_api.get(…)
mc.set(obj)
#现在可以操作 obj
```

构造函数

delete(key,time=0)

删除某个键。time 的单位是秒，确保特定时间内的 set/update 操作会失败。返回 1 成功，0 失败。

incr(key,delta=1)

给自增量变量加上 delta，默认为 1。

decr(key,delta=1)

给自减量变量减去 delta，默认为 1。

add(key,val,time=0,min_compress_len=0)

添加一个键值对，内部调用 _set() 方法。

replace(key,val,time=0,min_compress_len=0)

替换值，内部调用_set() 方法。

set(key,val,time=0,min_compress_len=0)

无条件的设置键值对。time 设置超时，单位是秒。min_compress_len 用于设置 zlib 压缩。内部调用_set() 方法。

set_multi(mapping,time=0,key_prefix=”,min_compress_len=0)

设置多个键值对。

get(key)

获取值。出错则返回 None。

get_multi(keys,key_prefix=”)

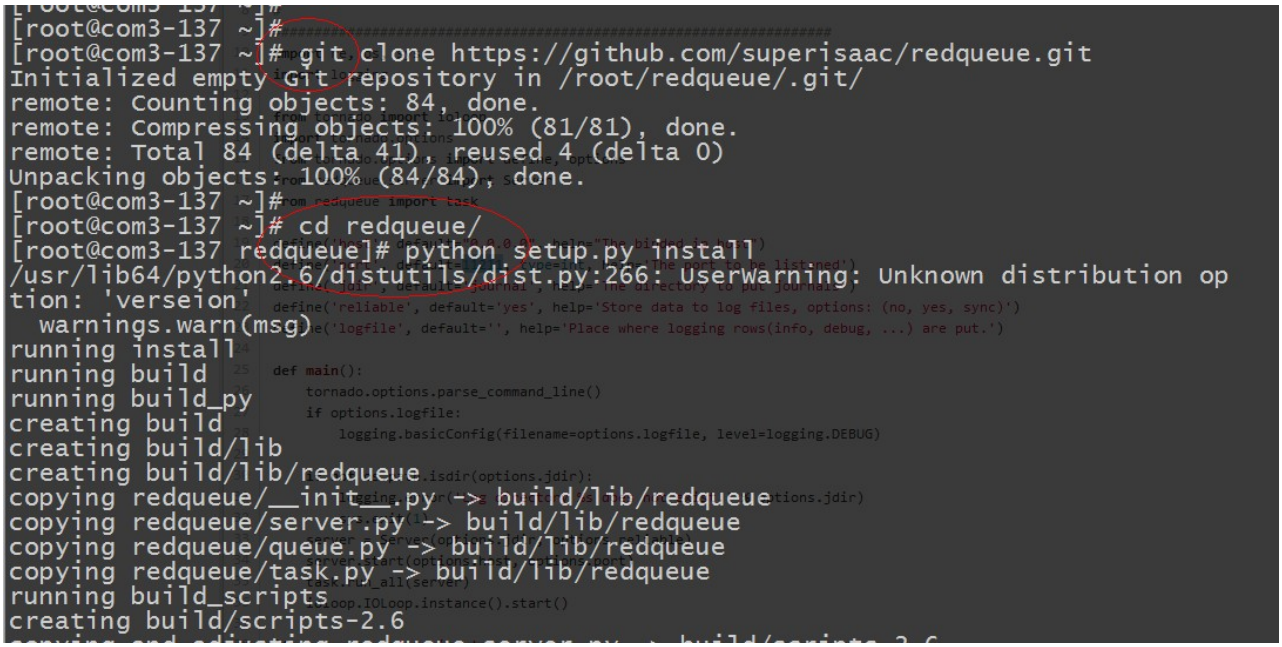
获取多个键的值，返回字典。keys 为健明列表。key_prefix 是键名前缀，可以最终构成 key_prefix+key 的完整键名。与 set_multi 中一样。

Memcached 本身没有的实现的，但是高手还是多呀，有个高手开源了一个 memcached 队列的 python 实现方案。

RedQueue 参考了 github 开源项目 starling(ruby 写的)， twitter 曾经使用伊做队列服务，后来改成了用 scala 写的 scaling(kestrol) . Redqueue 用 python 的高性能框架 tornado 写成。支持 memcache 协议, 也就是说伪装成一个 memcache server，由于许多语言都有了 memcache 库，也就有了应用 redqueue 的土壤。

redqueue 是可以持久化的，使用日志文件记录所有的操作，当系统重启的时候，可以恢复没有处理的未超时任务重新处理。 这样对于 server 端的容错性有好处。更进一步的是，redqueue 具有客户端容错性，客户通过 ge

t 命令从队列中得到一个任务，使用 delete 删除这个任务，如果没有 delete 而因某种原因退出了，则该任务会被 server 重新塞入队列等待处理。



图片 16.3 pic

关于 redqueue 的 python 应用小 demo ~

```
 # 引入 memcache 模块

import memcache
 #初始化客户端

mc = memcache.Client(['127.0.0.1:12345'])  # 假设 redqueue server 守候在 localhost 的 12345 端口
 # 发布一个项目到 key myqueue 中, 值为"Hello world"

mc.set("xiaorui", "good")

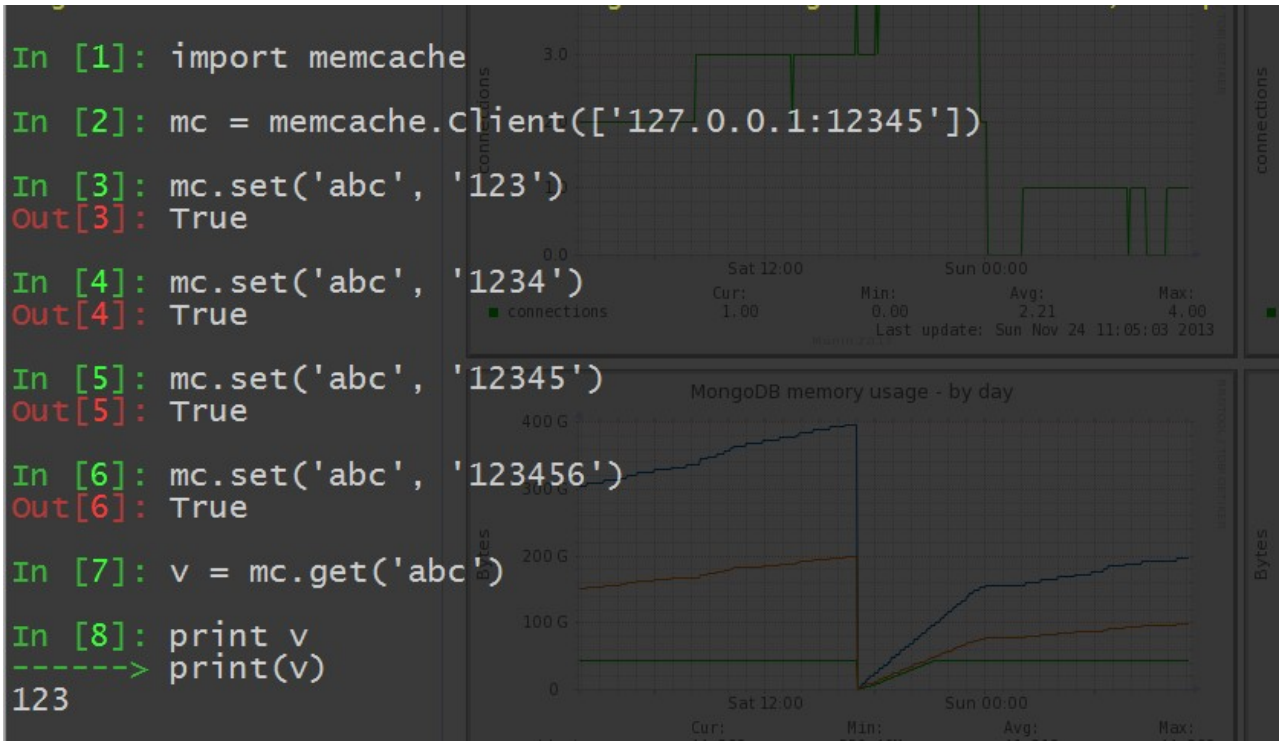
 # 消费者从 queue server 中取出一个任务， 并打印

print mc.get("xiaorui")  # 应该是 good

 # 删除一个任务，必须做，否则 server 会认为客户端异常发生了，而重新队列处理该任务

 # 什么时候客户端确认该任务已经确保执行了，就可以 delete 掉。在这之间，任务不会被其他客户端执行。

mc.delete("xiaorui")
```



图片 16.4 pic

这个是作者给的过程：

```
== Install and Run
 Install tornado and (optional) python-memcached for client testing

 Get the source from
 git@github.com:superisaac/redqueue.git

 Install
 % python setup.py install
 Make the log dir
 % mkdir -p log
 Run the server
 % redqueue_server.py

 For more options please run
 % redqueue_server.py --help
== Reserve/delete mode
  Reserve/delete mode is currently the sole mode, once an item is fetched, a delete request must be send later to mark th
  >>> mc.set('abc', '123')
  >>> v = mc.get('abc')
  >>> if v is not None:
  >>>     mc.delete('abc')
```

现在队列有了，我给大家说下，我那边是咋实现排队的~

当用户访问页面下一步的时候，我会判断队列，要是他前面有人在进行，我会给他重定向到最初的页面。当别人搞完了，他才可以的。

但是这样的话，还有个问题，那就是要是有 5 个人同时进了队列里面了，我给他们已经排序了，要是老大和老二，他不在进行了，老三的话，咋办。。。 这时候就需要配置队列里面的值和 kv 的一个值做时间的生效。 也就是说 老大和老二要是在指定的时间内没有完成的话，我会把他们踢出去，这样老三就成老大了。

本文出自 “峰云，就她了。” 博客，谢绝转载!

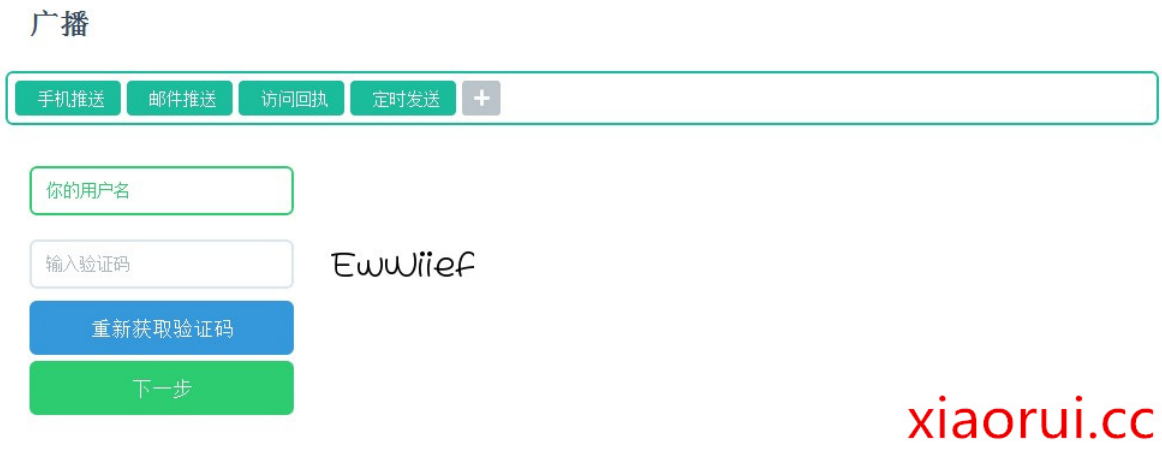
17

python 之利用 PIL 库实现页面的图片验证码及缩略图

前言：

在域用户平台里面集成了一个信使的功能，但是怕别人无脑的乱发，又怕别人乱调用接口，加了一个图片验证码的功能~

实现起来比较简单，用 python 的 PIL 库就可以实现了。



图片 17.1 pic

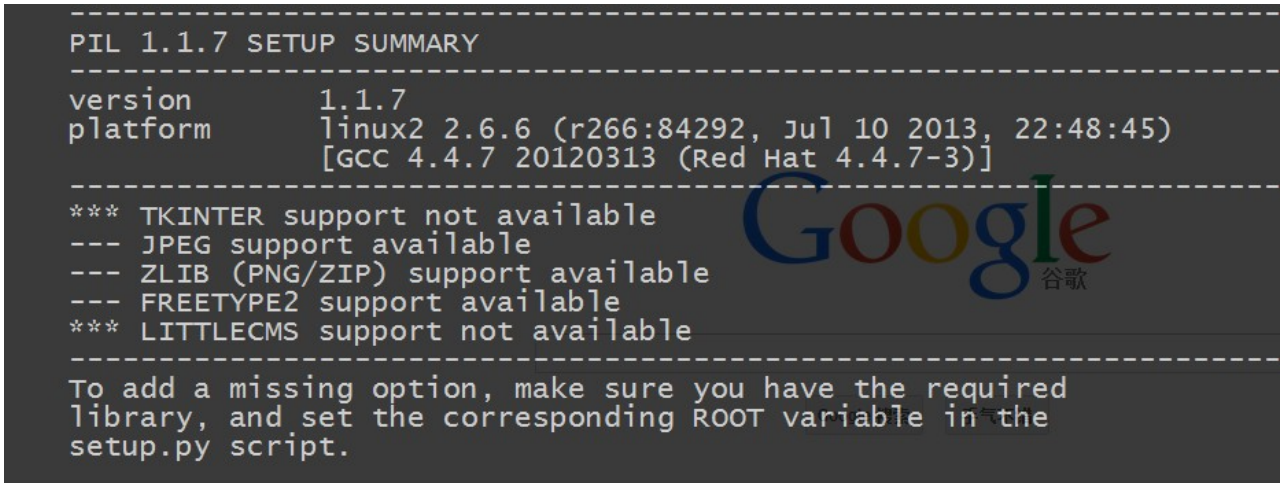
```
yum install -y libjpeg-devel
yum install freetype-*
```

安装 pil 库 ~

```
[root@102 ~]# pip install PIL
Downloading/unpacking PIL
  Downloading PIL-1.1.7.tar.gz (506kB): 506kB downloaded
  Running setup.py egg_info for package PIL
    WARNING: '' not a valid package name; please use only.-separated pac
Installing collected packages: PIL
  Running setup.py install for PIL
    WARNING: '' not a valid package name; please use only.-separated pac
    building '_imaging' extension
    gcc -pthread -fno-strict-aliasing -O2 -g -pipe -Wall -Wp,-D_FORTIFY_
rotector --param=ssp-buffer-size=4 -m64 -mtune=generic -D_GNU_SOURCE -fP
e -Wall -Wp,-D_FORTIFY_SOURCE=2 -fexceptions -fstack-protector --param=s
neric -D_GNU_SOURCE -fPIC -fwrapv -fPIC -DHAVE_LIBZ -IlibImaging -I/usr/
/usr/include/python2.6 -c _imaging.c -o build/temp.linux-x86_64-2.6/_ima
    _imaging.c:3017: warning: initialization from incompatible pointer t
    _imaging.c:3077: warning: initialization from incompatible pointer t
    gcc -pthread -fno-strict-aliasing -O2 -g -pipe -Wall -Wp,-D_FORTIFY_
rotector --param=ssp-buffer-size=4 -m64 -mtune=generic -D_GNU_SOURCE -fP
```

图片 17.2 pic

能不能用，要看下面是否支持 jpeg，不然图片没法写入的

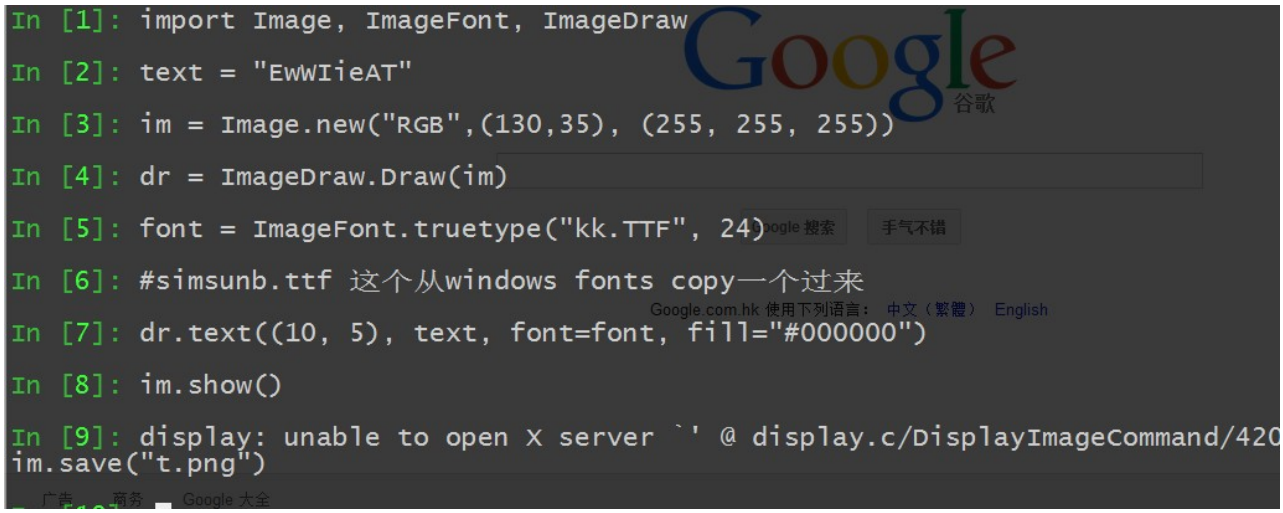


图片 17.3 pic

好嘞，咱们可以跑一个 demo 了~

```
import Image, ImageFont, ImageDraw
text = "EwWIieAT"
im = Image.new("RGB",(130,35), (255, 255, 255))
dr = ImageDraw.Draw(im)
font = ImageFont.truetype("kk.TTF", 24)
 #simsunb.ttf 这个从windows fonts copy一个过来

dr.text((10, 5), text, font=font, fill="#000000")
im.show()
im.save("t.png")
```



图片 17.4 pic

因为用的是终端，没法看，传到 windows 上，看下结果~



图片 17.5 pic

PIL 可以做很多的事情，比如缩略图的实现~

```
from PIL import Image
img = Image.open('god.jpg')
img = img.resize((250, 156), Image.ANTIALIAS)
img.save('sharejs_small.jpg')
```



图片 17.6 pic

对于验证码来说，上面的有点太正派了，很容易被破解了，所以咱们把验证码的图片给伪装下。

```
 #!/usr/bin/env python

 #coding=utf-8

import random
from PIL import Image, ImageDraw, ImageFont, ImageFilter
_letter_cases = "abcdefghjkmnpqrstuvwxy" # 小写字母，去除可能干扰的 i，l，o，z
_upper_cases = _letter_cases.upper() # 大写字母
_numbers = ''.join(map(str, range(3, 10))) # 数字
init_chars = ''.join((_letter_cases, _upper_cases, _numbers))
def create_validate_code(size=(120, 30),
                         chars=init_chars,
                         img_type="GIF",
                         mode="RGB",
                         bg_color=(255, 255, 255),
                         fg_color=(0, 0, 255),
                         font_size=18,
                         font_type="kk.TTF",
                         length=4,
                         draw_lines=True,
                         n_line=(1, 2),
                         draw_points=True,
                         point_chance = 2):
```

```
'''
@todo: 生成验证码图片
@param size: 图片的大小，格式（宽，高），默认为(120, 30)
@param chars: 允许的字符集合，格式字符串
@param img_type: 图片保存的格式，默认为 GIF，可选的为 GIF，JPEG，TIFF，PNG
@param mode: 图片模式，默认为 RGB
@param bg_color: 背景颜色，默认为白色
@param fg_color: 前景色，验证码字符颜色，默认为蓝色#0000FF
@param font_size: 验证码字体大小
@param font_type: 验证码字体，默认为 ae_AlArabiya.ttf
@param length: 验证码字符个数
@param draw_lines: 是否划干扰线
@param n_lines: 干扰线的条数范围，格式元组，默认为(1, 2)，只有 draw_lines 为 True 时有效
@param draw_points: 是否画干扰点
@param point_chance: 干扰点出现的概率，大小范围[0, 100]
@return: [0]: PIL Image 实例
@return: [1]: 验证码图片中的字符串
'''
width, height = size # 宽， 高
img = Image.new(mode, size, bg_color) # 创建图形
draw = ImageDraw.Draw(img) # 创建画笔
def get_chars():
    '''生成给定长度的字符串，返回列表格式'''
    return random.sample(chars, length)
def create_lines():
    '''绘制干扰线'''
    line_num = random.randint(*n_line) # 干扰线条数
    for i in range(line_num):
        # 起始点
        begin = (random.randint(0, size[0]), random.randint(0, size[1]))
        #结束点
        end = (random.randint(0, size[0]), random.randint(0, size[1]))
        draw.line([begin, end], fill=(0, 0, 0))
def create_points():
    '''绘制干扰点'''
    chance = min(100, max(0, int(point_chance))) # 大小限制在[0, 100]

    for w in xrange(width):
        for h in xrange(height):
            tmp = random.randint(0, 100)
            if tmp > 100 - chance:
                draw.point((w, h), fill=(0, 0, 0))
def create_strs():
    '''绘制验证码字符'''
    c_chars = get_chars()
```

```
        strs = ' %s ' % ' '.join(c_chars) # 每个字符前后以空格隔开

        font = ImageFont.truetype(font_type, font_size)
        font_width, font_height = font.getsize(strs)
        draw.text(((width - font_width) / 3, (height - font_height) / 3),
                  strs, font=font, fill=fg_color)

        return ''.join(c_chars)
    if draw_lines:
        create_lines()
    if draw_points:
        create_points()
    strs = create_strs()
    # 图形扭曲参数
    params = [1 - float(random.randint(1, 2)) / 100,
              0,
              0,
              0,
              1 - float(random.randint(1, 10)) / 100,
              float(random.randint(1, 2)) / 500,
              0.001,
              float(random.randint(1, 2)) / 500
              ]
    img = img.transform(size, Image.PERSPECTIVE, params) # 创建扭曲
    img = img.filter(ImageFilter.EDGE_ENHANCE_MORE) # 滤镜，边界加强（阈值更大）
    return img, strs
if __name__ == '__main__':
    code_img = create_validate_code()
    code_img[0].save("xiaorui.cc.gif", "GIF")
```

看下验证码效果 ~



图片 17.7 pic

要是在 web 框架里面的话，可以直接 return 一个验证码图片

比如：

tornado 的实现方式

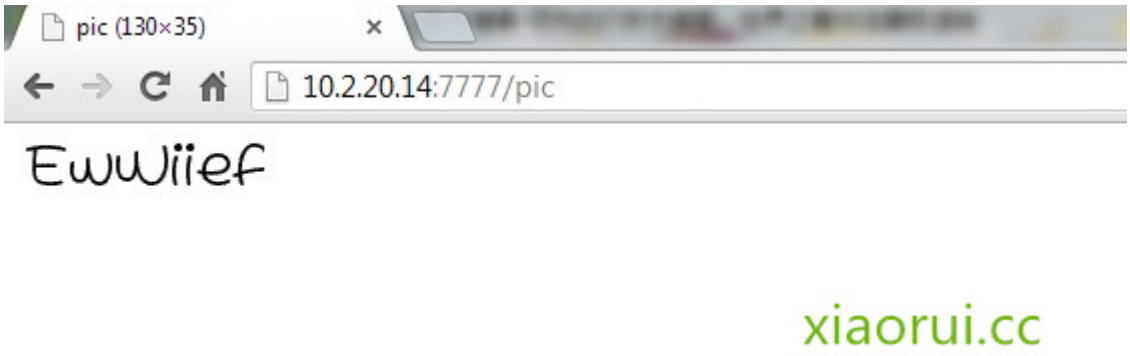
```
output = StringIO()
im.save(output, 'JPEG', quality = 95)
img_data = output.getvalue()
output.close()
 #在浏览器现实图片
```

```
set_header('Content-Type','image/jpeg')
write(img_data)
```

我这里用的是 flask，代码实现是

```
@app.route('/pic')
def showPic():
    import Image, ImageFont, ImageDraw
    import StringIO
    output = StringIO.StringIO()
    text = "EwWlieAT"
    im = Image.new("RGB",(130,35), (255, 255, 255))
    dr = ImageDraw.Draw(im)
    font = ImageFont.truetype("kk.TTF", 24)
    #simsunb.ttf 这个从windows fonts copy一个过来
    dr.text((10, 5), text, font=font, fill="#000000")
 #    im.show()

    im.save(output,"GIF")
    img_data = output.getvalue()
    output.close()
    response = make_response(img_data)
    response.headers['Content-Type'] = 'image/gif'
    return response
```



图片 17.8 pic

本文出自 “峰云，就她了。” 博客，谢绝转载!

18

如何将 Mac OS X10.9 下的 Python2.7 升级到最新的 Python3.3

Mac OS X10.9 默认带了 Python2.7，不过现在 Python3.3.3 出来了，如果想使用最新版本，赶紧升级下吧。基本步骤如下。

第 1 步：下载 Python3.3

下载地址如下：

Python3.3

这里面有 windows 和 mac os x 下的安装程序，下载那个 64 位的安装程序（估计现在没有用 32 位的 mac os x 的吧）

第 2 步：

安装下载的 img 文件，安装完后的目录如下：

```
/Library/Frameworks/Python.framework/Versions/3.3
```

第 3 步：移动 python 的安装目录

原来的安装目录见第 2 步，不过所有的 python 都在

```
/System/Library/Frameworks/Python.framework/Versions
```

目录中，所以最好使用下面的命令移动一下，当然不移动也可以。但后面步骤中的某些路径需要修改下。

```
sudo mv /Library/Frameworks/Python.framework/Versions/3.3 /System/Library/Frameworks/Python.framework/Versions
```

第 4 步：改变 Python 安装目录的用户组为 wheel

```
sudo chown -R root:wheel /System/Library/Frameworks/Python.framework/Versions/3.3
```

python2.7 的用户组就是 wheel，3.3 也照葫芦画瓢吧！

第 5 步：修改 Python 当前安装目录的符号链接

在 /System/Library/Frameworks/Python.framework/Versions/ 目录下有一个 Current，这是一个目录符号链接，指向当前的 Python 版本。原来指向 2.7 的，现在指向 3.3。所以应先删除 Current。然后重新建立 Current 符号链接，命令如下：

```
sudo rm /System/Library/Frameworks/Python.framework/Versions/Current
sudo ln -s /System/Library/Frameworks/Python.framework/Versions/3.3 /System/Library/Frameworks/Python.framework
```

第 6 步：删除旧的命令符号链接

在 /usr/bin 目录下有 4 个 python 命令的符号链接，使用下面的命令先删除

```` sudo rm /usr/bin/pydoc sudo rm /usr/bin/python sudo rm /usr/bin/pythonw sudo rm /usr/bin/python-config

第 7 步：重新建立新的命令符号链接

将第 6 步删除的符号链接重新使用下面命令建立，它们都指向 Python3.3 了。

sudo ln -s /System/Library/Frameworks/Python.framework/Versions/3.3/bin/pydoc3.3 /usr/bin/pydoc sudo ln -s /System/Library/Frameworks/Python.framework/Versions/3.3/bin/python3.3 /usr/bin/python sudo ln -s /System/Library/Frameworks/Python.framework/Versions/3.3/bin/pythonw3.3 /usr/bin/pythonw sudo ln -s /System/Library/Frameworks/Python.framework/Versions/3.3/bin/python3.3m-config /usr/bin/python-config

第 8 步：更新 /root/.bash_profile 文件中的路径

cd ~ vim .bash_profile

在.bash_profile 插入下面的内容即可

# Setting PATH for Python 3.3

# The orginal version is saved in .bash_profile.pysave

PATH="/System/Library/Frameworks/Python.framework/Versions/3.3/bin:${PATH}" export PATH

ok，现在重新启动一下 Console，然后执行 python --version，得到的就是 Python 3.3.3。如果在程序中，需要使用下面代码获取

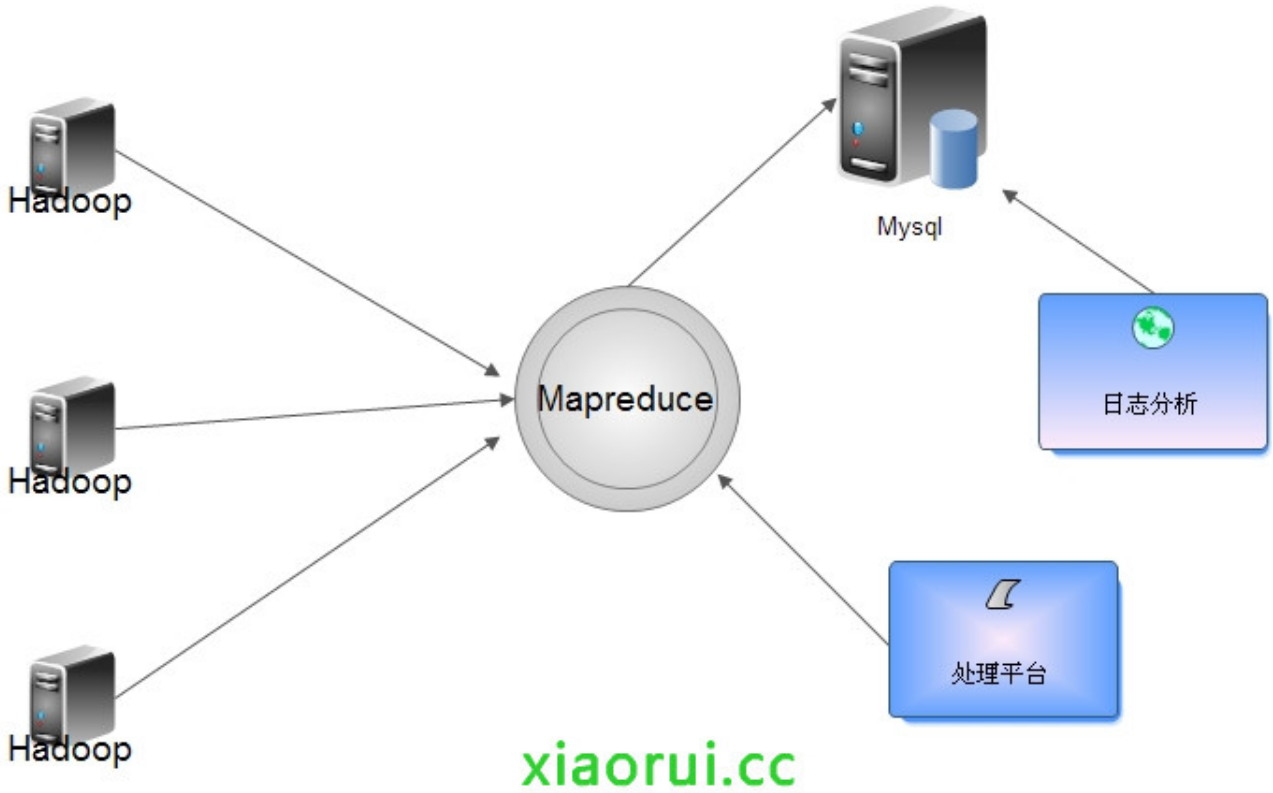
import platform print(platform.python_version()) ```

如果还是用了如 PyDev 等 IDE，仍然需要更新一下相关的路径。

现在可以使用最新的 Python3.3.3 了。
````

19

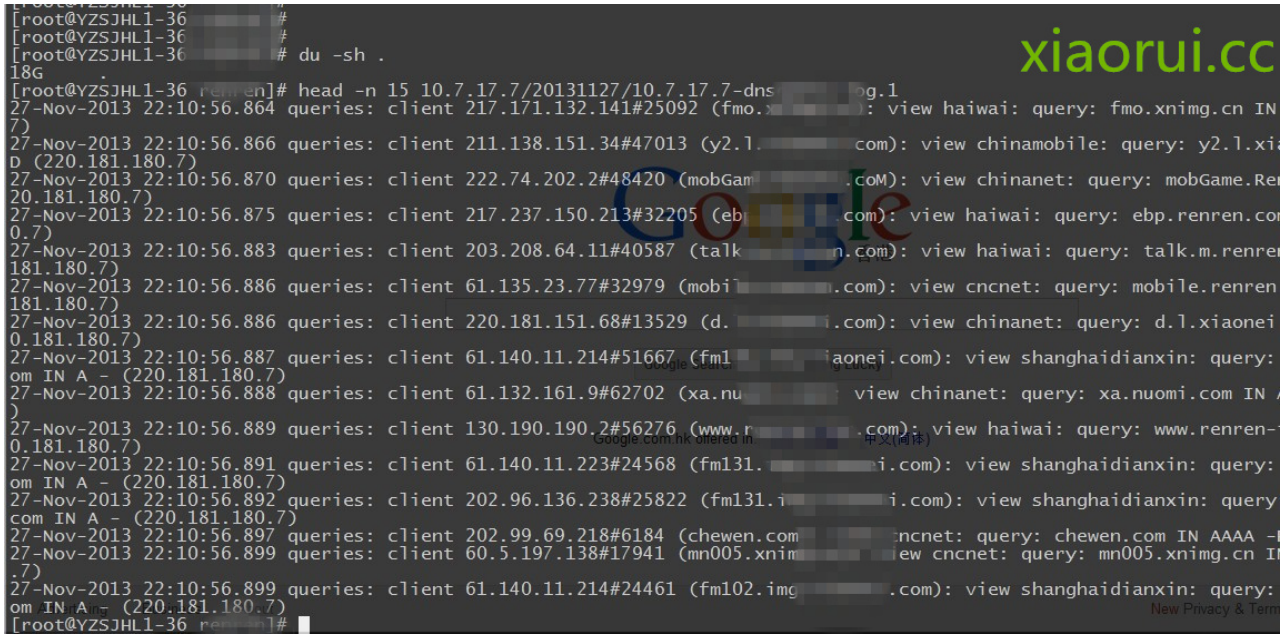
使用 python 构建基于 hadoop 的 mapreduce 日志分析平台



图片 19.1 pic

流量比较大的日志要是直接写入 Hadoop 对 Namenode 负载过大，所以入库前合并，可以把各个节点的日志凑并成一个文件写入 HDFS。根据情况定期合成，写入到 hdfs 里面。

咱们看看日志的大小，200 G 的 dns 日志文件，我压缩到了 18 G，要是用 awk perl 当然也可以，但是处理速度肯定没有分布式那样的给力。



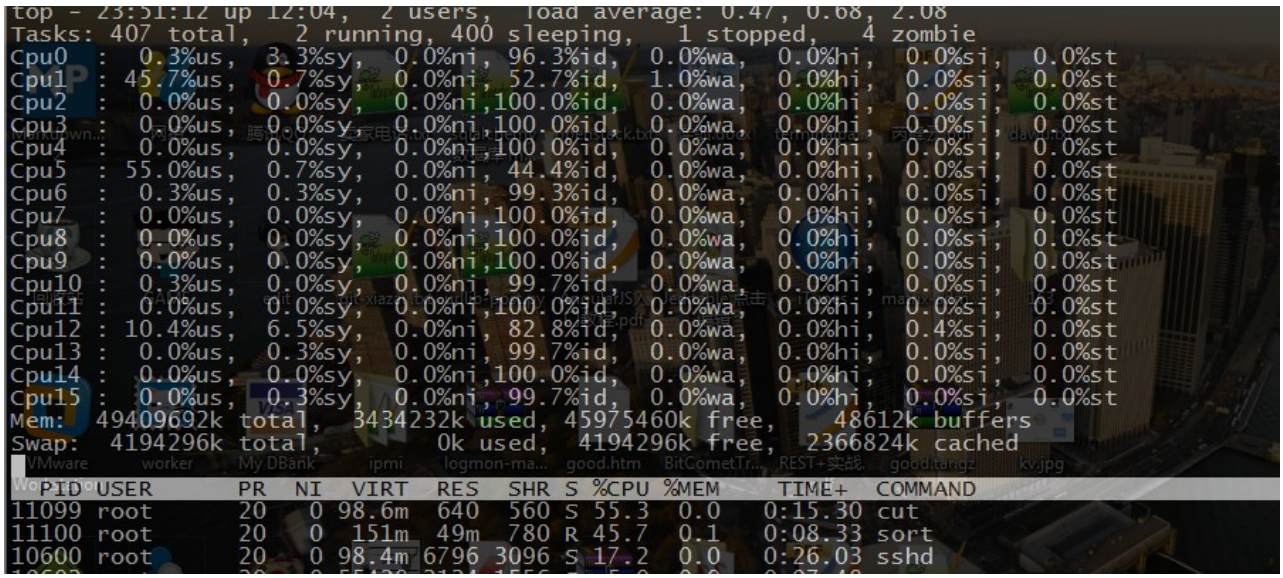
图片 19.2 pic

Hadoop Streaming 原理

mapper 和 reducer 会从标准输入中读取用户数据，一行一行处理后发送给标准输出。Streaming 工具会创建 MapReduce 作业，发送给各个 tasktracker，同时监控整个作业的执行过程。

任何语言，只要是方便接收标准输入输出就可以做 mapreduce~

再搞之前我们先简单测试下 shell 模拟 mapreduce 的性能速度~



图片 19.3 pic

看下他的结果，350 M 的文件用时 35 秒左右。



图片 19.4 pic

这是 2 G 的日志文件，居然用了 3 分钟。 当然和我写的脚本也有问题，我们是模拟 mapreduce 的方式，而不是调用 shell 下牛逼的 awk，gawk 处理。

```
[root@101 data2]#
[root@101 data2]# time cat 10.7.17.7-dnsquery.log.2 |cut -d ' ' -f 5|sort|uniq -c|sort -rn|head -n 20
  41147 125.76.192.170
  36507 61.50.244.69
  30669 117.34.127.40
  24180 211.137.32.177
  23655 211.137.32.179
  21260 61.135.23.61
  20668 61.135.23.59
  19749 61.135.23.58
  19152 124.160.105.198
  18886 222.74.126.232
  18524 61.135.23.62
  18170 61.50.246.117
  17607 61.135.23.90
  17316 61.135.23.93
  17194 61.135.23.60
  17147 61.135.23.29
  16575 202.106.195.90
  16538 61.139.113.166
  16461 61.139.113.162
  16055 61.135.23.109

real    3m12.635s
user    3m15.782s
sys     0m4.142s
[root@101 data2]#
[root@101 data2]# du -sh 10.7.17.7-dnsquery.log.2
2.0G    10.7.17.7-dnsquery.log.2
[root@101 data2]#
```

Search
xiaorui.cc

图片 19.5 pic

awk 的速度！果然很霸道，处理日志的时候，我也很喜欢用 awk，只是学习的难度有点大，不像别的 shell 组件那么灵活简单。

```
#!/usr/bin/python2.6
#coding=utf8
import os
import os.path
import re
def cassandra_file(url_dir):
        if not os.path.isdir(url_dir):
                return False
        elif os.path.isdir(url_dir):
                for dirpath,dirname,filenames in os.walk(url_dir):
                        for file in filenames:
                                filepath = os.path.join(dirpath,file)
                                aa = os.path.splitext(file)
                                bb = "".join(aa)
                                if "-ib-" not in str(bb):  #排除某些文件
                                        print "要处理文件名为:" + bb                    # 获取所有文件名
                                        cc = bb.split('-')
                                        dd = '-'.join(cc[1:])
                                        print "重命名后的文件名为:" + dd        #获取最后的文件名名
                                        old_url= os.path.split(filepath)[0]
                                        print "旧目录路径为:" + old_url
                                        old_file_url= os.path.join(old_url,dd)
                                        print "旧文件路径为:" + old_file_url
                                        os.rename(filepath,old_file_url)
                                        if len(url_dir.split('/')) == 3:
                                                dddd=filepath.split("/")[1:4]    #
                                                dsc_dir= "/" + "/".join(dddd)
                                                print "需要挪到这个路径下:"   dsc_dir # 获取要挪的目的路径
                                                eeee = os.path.join(dsc_dir,dd)
                                                print "新文件路径为:" + eeee
                                                os.rename(old_file_url,os.path.join(dsc_dir,dd))
                                        elif len(url_dir.split('/')) == 4:
                                                dddd=filepath.split("/")[1:5]    #
                                                dsc_dir= "/" + "/".join(dddd)
                                                print "需要挪到这个路径下:" + dsc_dir # 获取要挪的目的路径
                                                eeee = os.path.join(dsc_dir,dd)
                                                print "新文件路径为:" + eeee
                                                os.rename(old_file_url,os.path.join(dsc_dir,dd))
cassandra_file("/cassandra/data")
```

51CTO.com
技术博客 Blog

图片 19.6 pic

这是官方的提供的两个 demo ~

map.py

```
#!/usr/bin/env python

"""A more advanced Mapper, using Python iterators and generators."""
import sys
def read_input(file):
    for line in file:
        # split the line into words
        yield line.split()
def main(separator='\t'):
    # input comes from STDIN (standard input)
    data = read_input(sys.stdin)
    for words in data:
        # write the results to STDOUT (standard output);
        # what we output here will be the input for the
        # Reduce step, i.e. the input for reducer.py
        #
        # tab-delimited; the trivial word count is 1
        for word in words:
            print '%s%s%d' % (word, separator, 1)
if __name__ == "__main__":
    main()
```

reduce.py 的修改方式

```
#!/usr/bin/env python

"""A more advanced Reducer, using Python iterators and generators."""
from itertools import groupby
from operator import itemgetter
import sys
def read_mapper_output(file, separator='\t'):
    for line in file:
        yield line.rstrip().split(separator, 1)
def main(separator='\t'):
    # input comes from STDIN (standard input)
    data = read_mapper_output(sys.stdin, separator=separator)
    # groupby groups multiple word-count pairs by word,
    # and creates an iterator that returns consecutive keys and their group:
    #   current_word - string containing a word (the key)
    #   group - iterator yielding all ["<current_word>", "<count>"] items
    for current_word, group in groupby(data, itemgetter(0)):
        try:
            total_count = sum(int(count) for current_word, count in group)
```

```
            print "%s%s%d" % (current_word, separator, total_count)
        except ValueError:
            # count was not a number, so silently discard this item
            pass
if __name__ == "__main__":
    main()
```

咱们再简单点：

```
 #!/usr/bin/env python

import sys
for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
        print '%s\t%s' % (word, 1)
```

```
 #!/usr/bin/env python



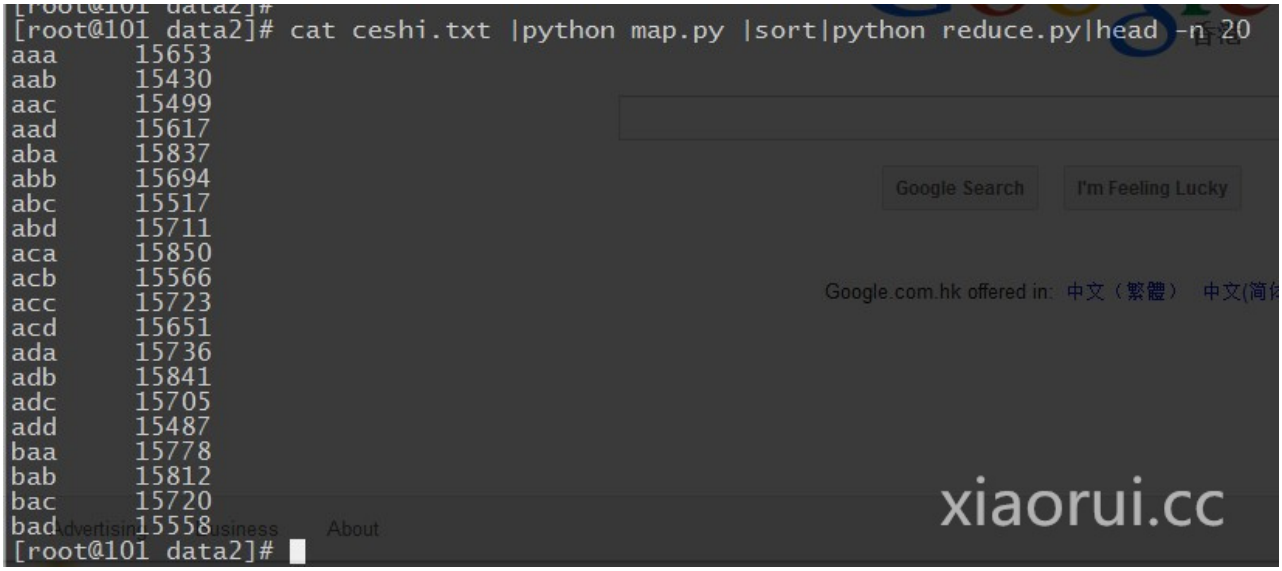
from operator import itemgetter
import sys

current_word = None
current_count = 0
word = None

for line in sys.stdin:
    line = line.strip()
    word, count = line.split('\t', 1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print '%s\t%s' % (current_word, current_count)
        current_count = count
        current_word = word

if current_word == word:
    print '%s\t%s' % (current_word, current_count)
```

咱们就简单模拟下数据，跑个测试



图片 19.7 pic

剩下就没啥了，在 hadoop 集群环境下，运行 hadoop 的 steaming.jar 组件，加入 mapreduce 的脚本，指定输出就行了. 下面的例子我用的是 shell 的成分。

```
[root@101 cron]#$HADOOP_HOME/bin/hadoop  jar $HADOOP_HOME/contrib/streaming/hadoop-*-streaming.jar \
-input myInputDirs \
-output myOutputDir \
-mapper cat \
-reducer wc
```

详细的参数，对于咱们来说提供性能可以把 tasks 的任务数增加下，根据情况自己测试下，也别太高了，增加负担。

（1）-input：输入文件路径
（2）-output：输出文件路径
（3）-mapper：用户自己写的 mapper 程序，可以是可执行文件或者脚本
（4）-reducer：用户自己写的 reducer 程序，可以是可执行文件或者脚本
（5）-file：打包文件到提交的作业中，可以是 mapper 或者 reducer 要用的输入文件，如配置文件，字典等。
（6）-partitioner：用户自定义的 partitioner 程序
（7）-combiner：用户自定义的 combiner 程序（必须用 java 实现）
（8）-D：作业的一些属性（以前用的是-jonconf），具体有：

1）mapred.map.tasks：map task 数目
2）mapred.reduce.tasks：reduce task 数目
3）stream.map.input.field.separator/stream.map.output.field.separator： map task 输入/输出数 据的分

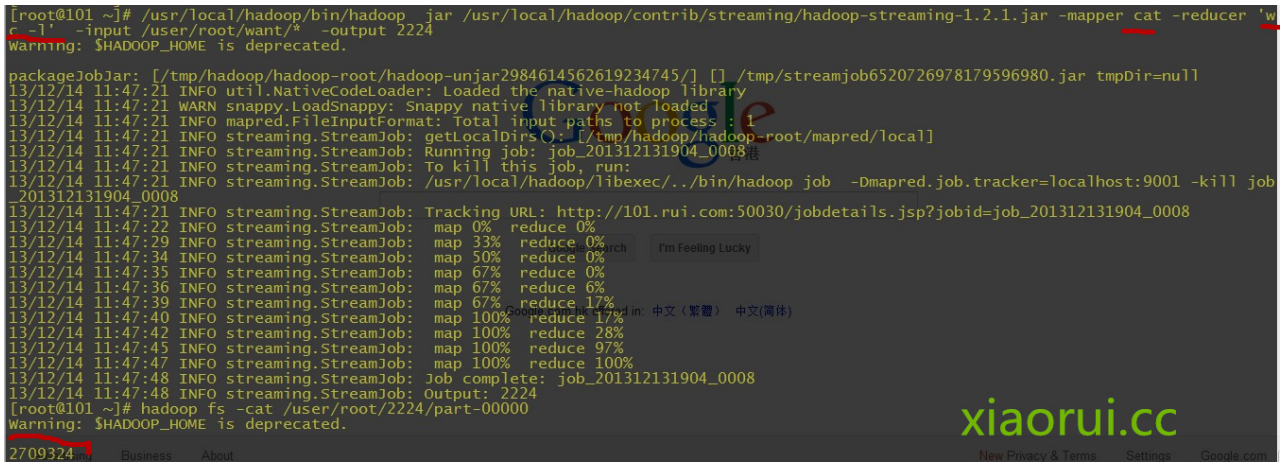
隔符,默认均为 \t。

4）stream.num.map.output.key.fields：指定 map task 输出记录中 key 所占的域数目

5）stream.reduce.input.field.separator/stream.reduce.output.field.separator：reduce task 输入/输出数据的分隔符，默认均为 \t。

6）stream.num.reduce.output.key.fields：指定 reduce task 输出记录中 key 所占的域数目

这里是统计 dns 的日志文件有多少行 ~



图片 19.8 pic

在 mapreduce 作为参数的时候，不能用太多太复杂的 shell 语言，他不懂的~

可以写成 shell 文件的模式;

```
#! /bin/bash

while read LINE; do
 #  for word in $LINE

 #  do

 #    echo "$word 1"

       awk '{print $5}'
  done
done
```

```
#! /bin/bash

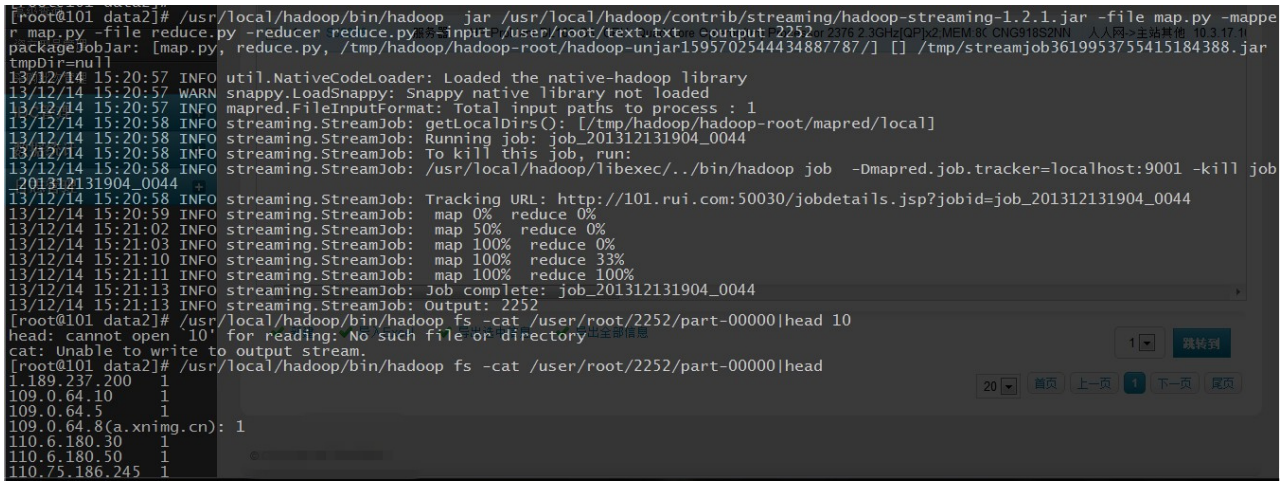
count=0
started=0
word=""
while read LINE;do
```

```
  goodk=`echo $LINE | cut -d ' '  -f 1`
  if [ "x" == x"$goodk" ];then
    continue
  fi
  if [ "$word" != "$goodk" ];then
   [ $started -ne 0 ] && echo -e "$word\t$count"
   word=$goodk
   count=1
   started=1
  else
   count=$(( $count + 1 ))
  fi
done
```

有时候会出现这样的问题，好好看看自己写的 mapreduce 程序 ~

```
13/12/14 13:26:52 INFO streaming.StreamJob: Tracking URL: http://101.rui.com:50030/jobdetails.jsp?jobid=job_201312131
13/12/14 13:26:53 INFO streaming.StreamJob:  map 0%  reduce 0%
13/12/14 13:27:16 INFO streaming.StreamJob:  map 100%  reduce 100%
13/12/14 13:27:16 INFO streaming.StreamJob: To kill this job, run:
13/12/14 13:27:16 INFO streaming.StreamJob: /usr/local/hadoop/libexec/../bin/hadoop job  -Dmapred.job.tracker=localhos
13/12/14 13:27:16 INFO streaming.StreamJob: Tracking URL: http://101.rui.com:50030/jobdetails.jsp?jobid=job_201312131
13/12/14 13:27:16 ERROR streaming.StreamJob: Job not successful. Error: # of failed Map Tasks exceeded allowed limit.
13/12/14 13:27:16 INFO streaming.StreamJob: killJob...
Streaming Command Failed!
```

python 做为 mapreduce 执行成功后，结果和日志一般是放在你指定的目录下的，结果是在 part-00000 文件里面~



图片 19.9 pic

下面咱们谈下，如何入库和后台的执行

本文出自 “峰云，就她了。” 博客，谢绝转载!

20

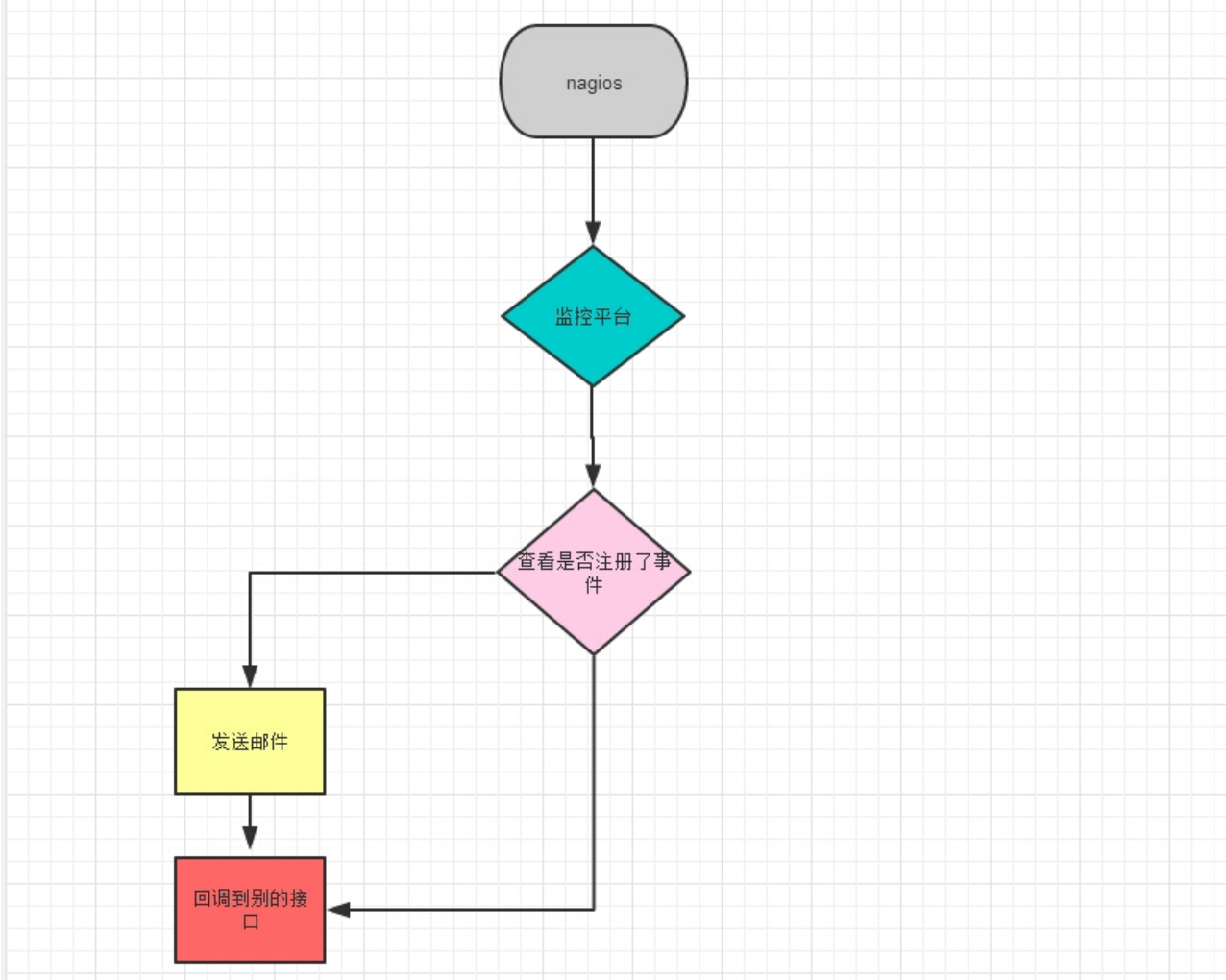
报警监控平台扩展功能 url 回调的设计及应用 [python 语言]

前言：

这个国内也有一些第三方的厂商在用，比如 dnspod 的 url 回调和监控宝的 url 回调!

有人开源了一个脚本，监控宝的 url 回调，可以联合 dnspod 的 api 接口。可以处理当 ip-A 的 web 死掉的时候，dns 记录切换到 ip-B 上。 当然这只是个小应用罢了，但不能不说，这个想法确实不错。 我这边也实现了类似方式。

所谓的 URL 回调功能，您可以让告警通知发送到您指定的 URL，使你能更加灵活处理告警消息。 打个比方，有个服务器的 nginx 进程死掉了，这个时候 nagios 监控到了这个情况，然后调用了我这边的接口，我这边接到的 post 数据，不仅发邮件，而且会根据注册事件的情况，进行处理。 如果注册了一个远程 nginx 重启的事项，我这边就远程 paramiko 或者是 saltstack 过去重启该进程！！！



图片 20.1 pic

怎么个灵活法：

每个业务部门其实都想自己统计 error 情况，但是监控平台一般是在基础监控部门手里掌控者，又不太方便做部署，这个时候，url 回调是个好方法。我会把每次告警的信息不仅推到你的 mail 和手机上，而且会给你的 url 地址做 webhook。你服务端接受认证后的 url 地址后，会有相应的措施，比如调用 saltsatck 来进行处理特定的主机，比如插入到库里面，自己做报表统计，根据来着的信息做自动化处理。

关于触发式的处理：

只是个人的想法而已 ~

在监控系统的体系下，比如有 nagios，zabbix 专业监控系统。 咱们还是用例子说话： 监控 mysql 从是否高延迟，严重不同步问题的时候，咱们一般是在 nagios 里加载监控获取判断从延迟的脚本，以及在某个节点上做处理脚本【 脚本的内容是 while get 每个 mysql 从情况，高延迟的那台在负载群里面踢出去 】，这样算的话是两个脚本了。

如果利用 url 回调，可以用处理脚本，这个脚本也只是当触发 url 回调的时候，才执行才处理的。避免了处理脚本没完没了的去判断和获取状态。要是监控一些统计压力大的服务，那就有点悲催了。

当然这样也会有些问题的，比如 web 死掉的话，他无法接受 url 回调，另一方面 开发部也不想调用系统层面的外部命令，毕竟责任是个问题。

下面是我写的 url 回调的 demo，等有机会上线供大神们测试下。

第一版的时候，没有定义 post 的方式，以及回调结果的查看。



图片 20.2 pic

第二版做了，get 和 post 的方式，返回结果的验证。



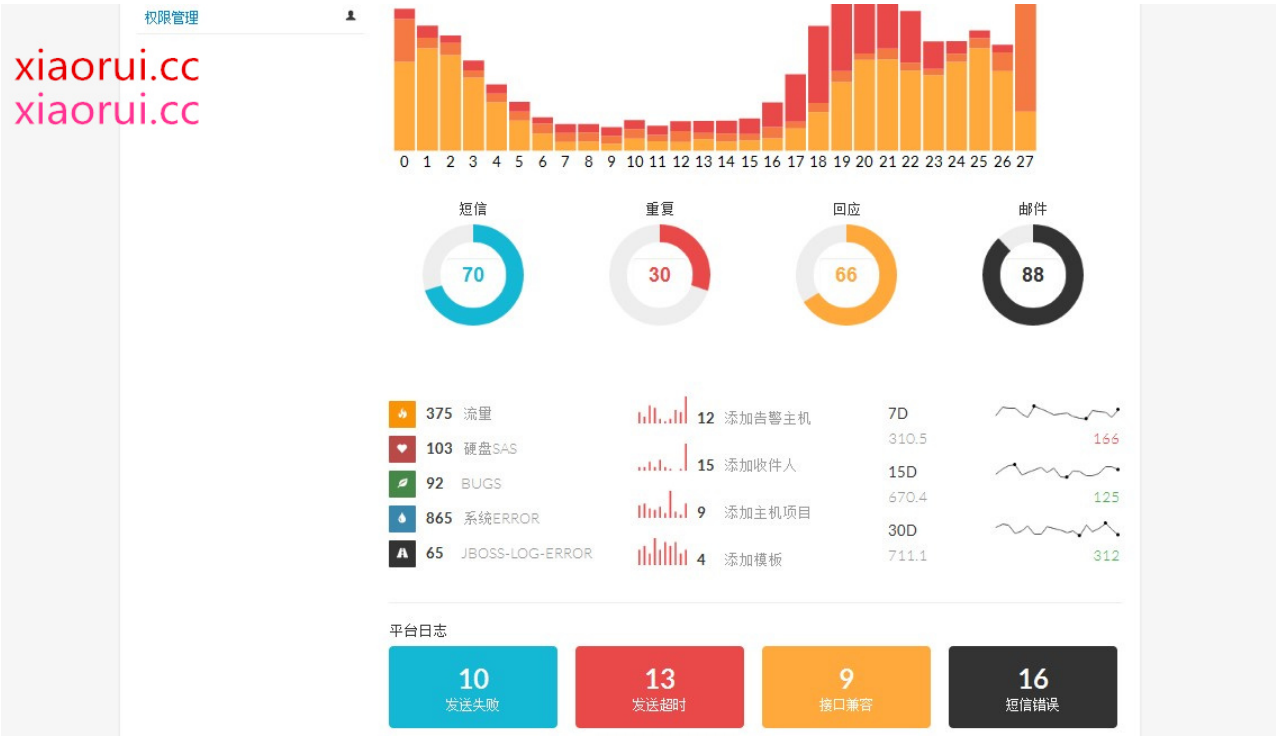
图片 20.3 pic

下面是平台的 demo ~ 我想说的是，现在好多公司的告警信息都没有统计，随意的调用 smtp 发邮件，而不知道发送成功了没有，每个月发送了几次，发送都是啥内容。当然这些东西在 nagios zabbix 也大体可以看到，但是个人觉得还是综合到一个管理系统下，管理系统更加直观。

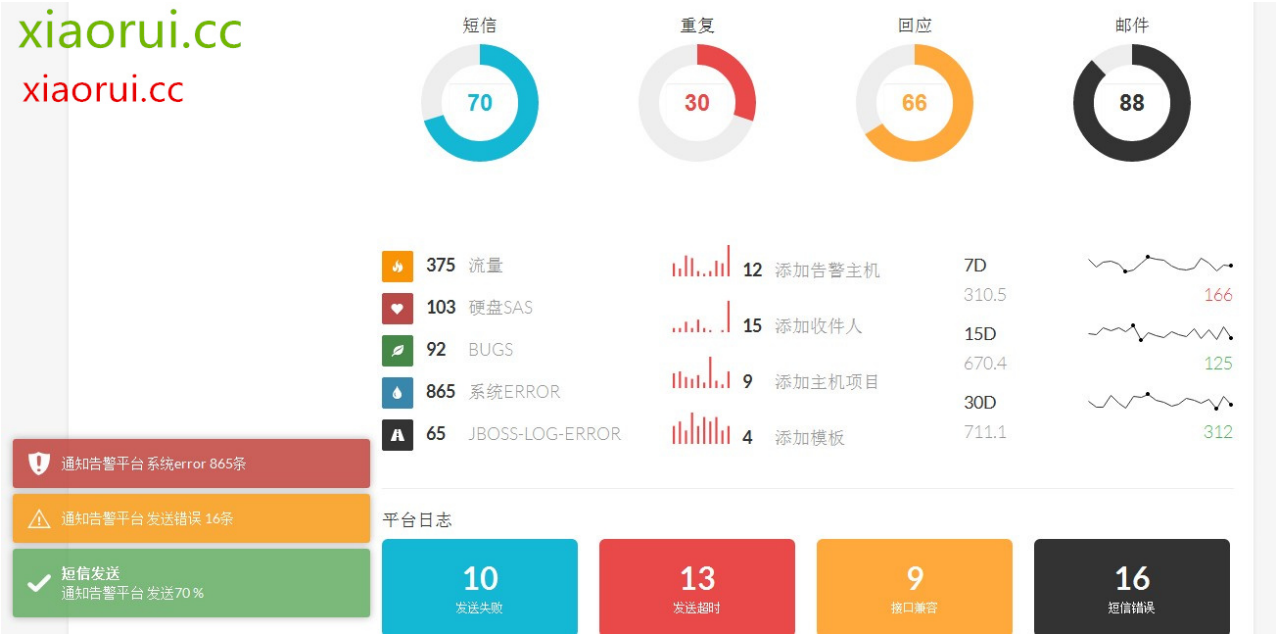
也有想这么搞的朋友直接提问题就行，我会第一时间给大家解答~

框架：

nginx tornado jquery



图片 20.4 pic



图片 20.5 pic

此文接上文： http://rfyiamcool.blog.51cto.com/1030776/1332160

有后文，会补上的~

本文出自 “峰云，就她了。” 博客，谢绝转载!

21

服务端 socket 开发之多线程和 gevent 框架并发测试[python 语言]

测试下多线程和 gevent 在 socket 服务端的小包表现能力，测试的方法不太严谨，有点属于自娱自乐，要是有问题之处，请大家喷之！

每个连接都特意堵塞了 0.5 秒钟！



图片 21.1 pic

在大批量 tcp 测试下，threading 的开销越来越大，所以造成了在并发数加大的情况下，出现 threading 崩溃的情况！gevent 是 libevent 和协程的融合，一个线程里面都可以跑超多的协程！ 利用 libevent 做 io 堵塞的调度，gevent 体系下，同一时间只有一个任务在运行！

先来测试下多线程： 我们就不加线程池了

```
#!/usr/bin/env python

# -*- coding: utf-8 -*-

#xiaorui.cc

import sys
import socket
import time
import threading
#xiaorui.cc

def threads(port):
    s = socket.socket()
    s.bind(('0.0.0.0', port))
    s.listen(500)
    while True:
        cli, addr = s.accept()
        t = threading.Thread(target=handle_request, args=(cli, time.sleep))
        t.daemon = True
        t.start()
def handle_request(s, sleep):
    try:
```

```
        s.recv(1024)
        sleep(0.5)
        s.send('''http/1.0 200 OK
                Hello World! ''')
        s.shutdown(socket.SHUT_WR)
        print '.',
    except Exception, ex:
        print ex
    finally:
        sys.stdout.flush()
        s.close()
if __name__ == '__main__':
    threads(4444)
```

用 threading 跑 socket，每个连接堵塞的时间是 0.5

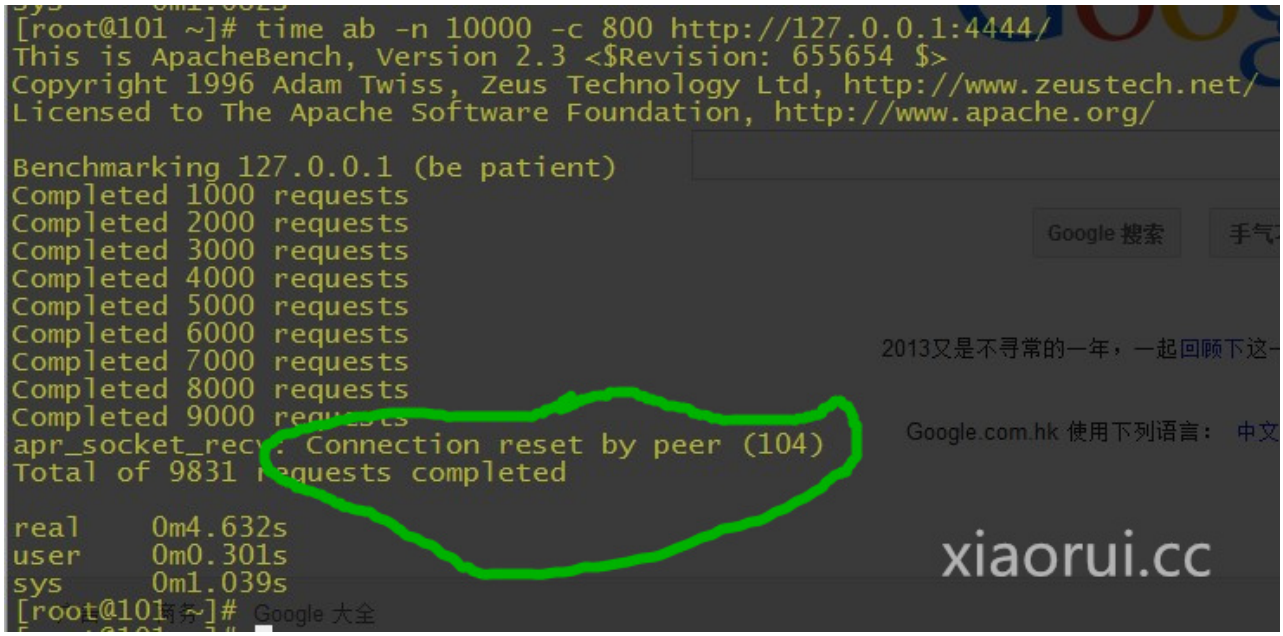
```
time ab -n 10000 -c 500 http://127.0.0.1:4444/
This is ApacheBench, Version 2.3 <$Revision: 655654 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
Benchmarking 127.0.0.1 (be patient)
Completed 1000 requests
Completed 2000 requests
Completed 3000 requests
Completed 4000 requests
Completed 5000 requests
Completed 6000 requests
Completed 7000 requests
Completed 8000 requests
Completed 9000 requests
Completed 10000 requests
Finished 10000 requests
Server Software:
Server Hostname:        127.0.0.1
Server Port:            4444
Document Path:          /
Document Length:        0 bytes
Concurrency Level:      500
Time taken for tests:   11.123 seconds
Complete requests:      10000
Failed requests:        0
Write errors:           0
Total transferred:      470000 bytes
HTML transferred:       0 bytes
Requests per second:    899.01 [#/sec] (mean)
Time per request:       556.166 [ms] (mean)
```

```
Time per request:       1.112 [ms] (mean, across all concurrent requests)
Transfer rate:          41.26 [Kbytes/sec] received
Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0   33 177.0      0    1000
Processing:   500  508  33.9    501    1132
Waiting:      500  508  33.9    501    1132
Total:        500  541 201.8    501    2132
Percentage of the requests served within a certain time (ms)
  50%    501
  66%    501
  75%    502
  80%    505
  90%    522
  95%    532
  98%   1534
  99%   1722
 100%   2132 (longest request)
real    0m11.145s
user    0m0.210s
sys     0m0.961s
```



图片 21.2 pic

加到 800 的时候~



图片 21.3 pic

gevent：

```
 #xiaorui.cc

import sys
import socket
import time
import gevent
from gevent import socket
def server(port):
    s = socket.socket()
    s.bind(('0.0.0.0', port))
    s.listen(500)
    while True:
        cli, addr = s.accept()
        gevent.spawn(handle_request, cli, gevent.sleep)
def handle_request(s, sleep):
    try:
        data=s.recv(1024)
        sleep(0.5)
        s.send('''http/1.0 200 OK
                Hello World! this is xiaorui.cc !!!''')
        print data
        request_string = "GET %s HTTP/1.1\r\nHost: %s\r\n\r\nServer: xiaorui.cc\n" %('index.html', '127.0.0.1')
        s.send(request_string)
        s.shutdown(socket.SHUT_WR)
        print '.', ‘be killed’
```

```
    except Exception, ex:
        print ex
    finally:

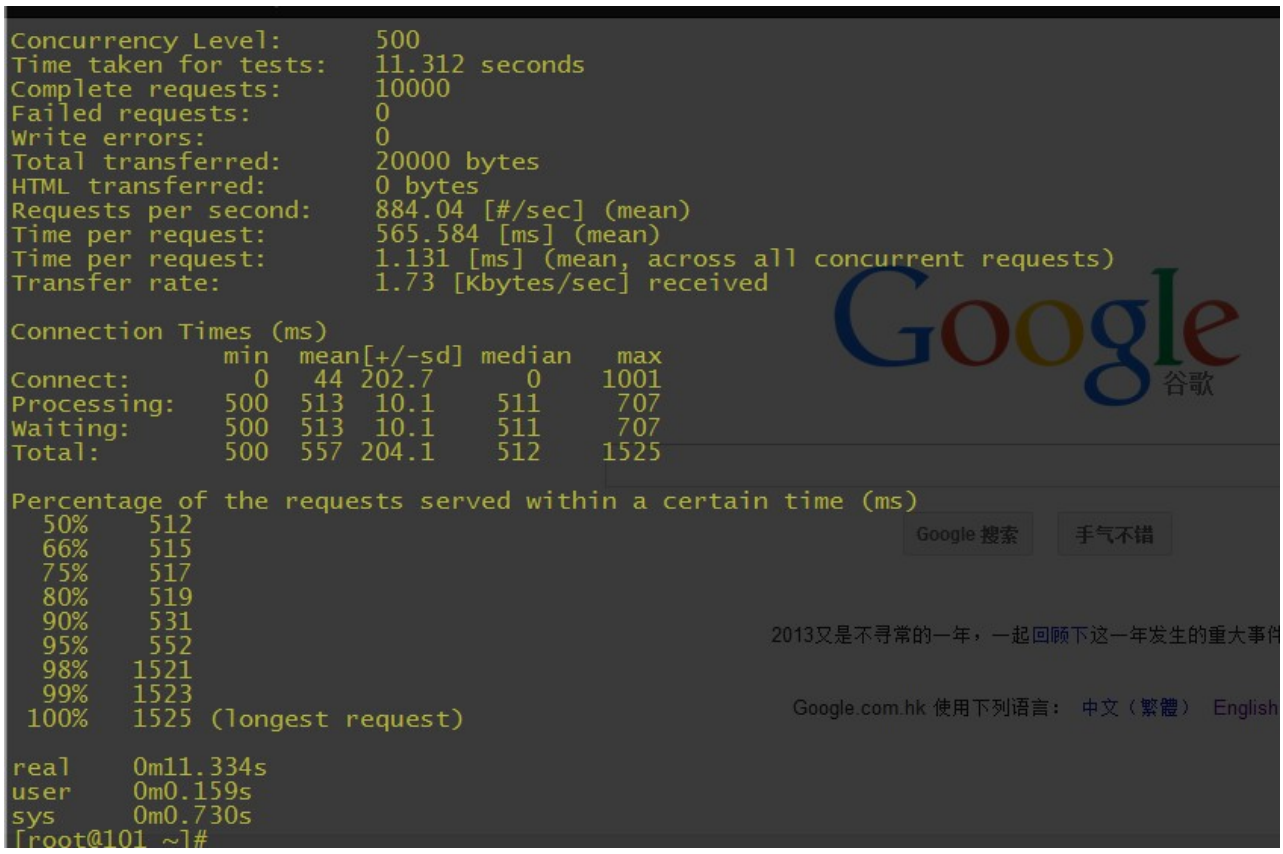
        s.close()
if __name__ == '__main__':
    server(7777)
```

gevent 跑 socket 服务：

并发数值是 500 的时候！

```
time ab -n 10000 -c 500 http://127.0.0.1:7777/
This is ApacheBench, Version 2.3 <$Revision: 655654 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
Benchmarking 127.0.0.1 (be patient)
Completed 1000 requests
Completed 2000 requests
Completed 3000 requests
Completed 4000 requests
Completed 5000 requests
Completed 6000 requests
Completed 7000 requests
Completed 8000 requests
Completed 9000 requests
Completed 10000 requests
Finished 10000 requests
Server Software:
Server Hostname:        127.0.0.1
Server Port:            7777
Document Path:          /
Document Length:        0 bytes
Concurrency Level:      500
Time taken for tests:   11.312 seconds
Complete requests:      10000
Failed requests:        0
Write errors:           0
Total transferred:      20000 bytes
HTML transferred:       0 bytes
Requests per second:    884.04 [#/sec] (mean)
Time per request:       565.584 [ms] (mean)
Time per request:       1.131 [ms] (mean, across all concurrent requests)
Transfer rate:          1.73 [Kbytes/sec] received
Connection Times (ms)
              min  mean[+/-sd] median   max
```

```
Connect:       0   44 202.7      0    1001
Processing:   500  513  10.1    511     707
Waiting:      500  513  10.1    511     707
Total:        500  557 204.1    512    1525
Percentage of the requests served within a certain time (ms)
  50%    512
  66%    515
  75%    517
  80%    519
  90%    531
  95%    552
  98%   1521
  99%   1523
 100%   1525 (longest request)
real    0m11.334s
user    0m0.159s
sys     0m0.730s
```



图片 21.4 pic

服务端看到的信息都是正常的!



图片 21.5 pic

并发是 1000 的时候：

```
time ab -n 10000 -c 1000 http://127.0.0.1:7777/
This is ApacheBench, Version 2.3 <$Revision: 655654 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
Benchmarking 127.0.0.1 (be patient)
Completed 1000 requests
Completed 2000 requests
Completed 3000 requests
Completed 4000 requests
Completed 5000 requests
Completed 6000 requests
Completed 7000 requests
Completed 8000 requests
Completed 9000 requests
Completed 10000 requests
Finished 10000 requests
Server Software:
Server Hostname:       127.0.0.1
Server Port:           7777
Document Path:         /
Document Length:       0 bytes
Concurrency Level:     1000
Time taken for tests:   7.406 seconds
```

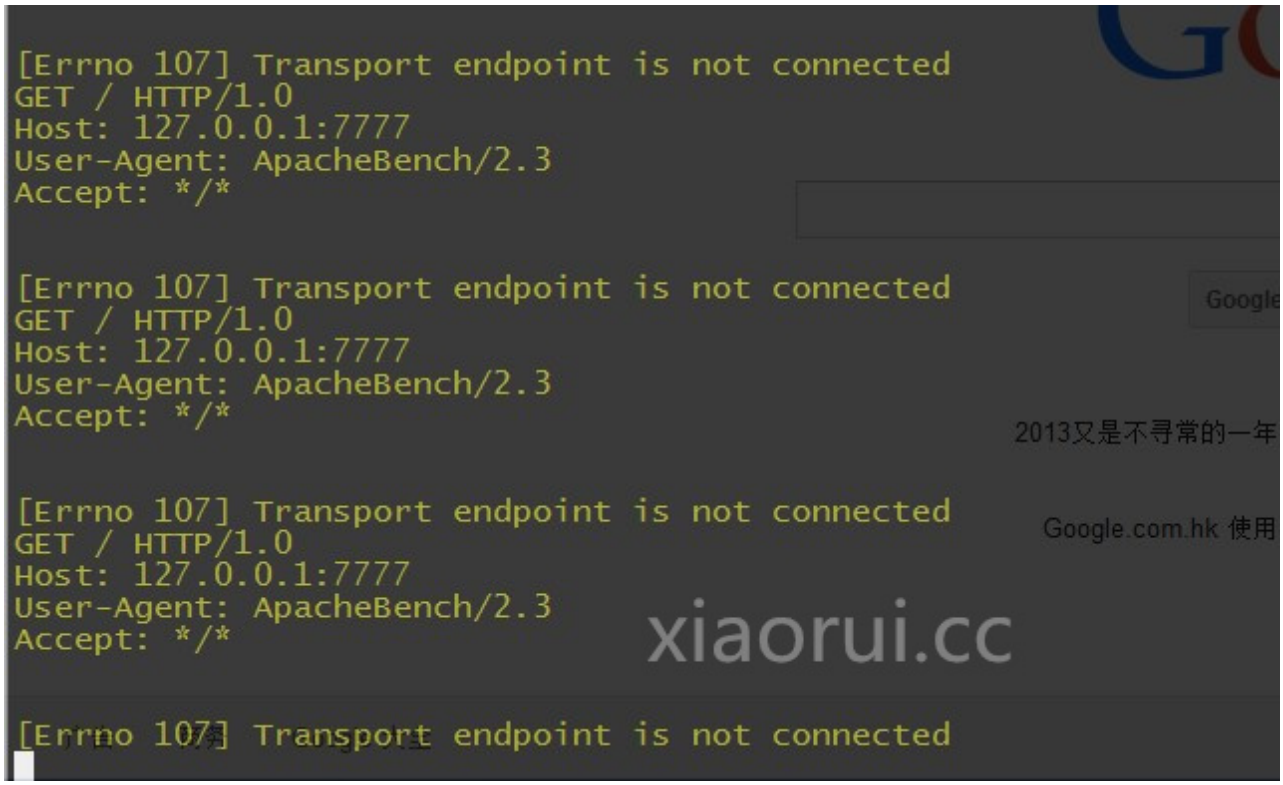
```
Complete requests:      10000
Failed requests:        0
Write errors:           0
Total transferred:      20000 bytes
HTML transferred:       0 bytes
Requests per second:    1350.22 [#/sec] (mean)
Time per request:       740.623 [ms] (mean)
Time per request:       0.741 [ms] (mean, across all concurrent requests)
Transfer rate:          2.64 [Kbytes/sec] received
Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0  175 491.7      0    3000
Processing:   500  520  17.7    515     707
Waiting:      500  520  17.7    515     707
Total:        500  695 492.5    517    3521
Percentage of the requests served within a certain time (ms)
  50%    517
  66%    523
  75%    538
  80%    569
  90%   1515
  95%   1530
  98%   1539
  99%   3514
 100%   3521 (longest request)
real    0m7.428s
user    0m0.208s
sys     0m0.741s
```

当并发到 1500 的时候：

```
time ab -n 10000 -c 1500 http://127.0.0.1:7777/
This is ApacheBench, Version 2.3 <$Revision: 655654 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/
Benchmarking 127.0.0.1 (be patient)
Completed 1000 requests
Completed 2000 requests
Completed 3000 requests
Completed 4000 requests
Completed 5000 requests
Completed 6000 requests
Completed 7000 requests
Completed 8000 requests
Completed 9000 requests
Completed 10000 requests
```

```
Finished 10000 requests
Server Software:
Server Hostname:        127.0.0.1
Server Port:            7777
Document Path:          /
Document Length:        0 bytes
Concurrency Level:      1500
Time taken for tests:   5.290 seconds
Complete requests:      10000
Failed requests:        0
Write errors:           0
Total transferred:      20000 bytes
HTML transferred:       0 bytes
Requests per second:    1890.27 [#/sec] (mean)
Time per request:       793.536 [ms] (mean)
Time per request:       0.529 [ms] (mean, across all concurrent requests)
Transfer rate:          3.69 [Kbytes/sec] received
Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0  214 404.9      1    1003
Processing:   500  522  23.0    514     716
Waiting:      500  522  23.0    514     716
Total:        500  736 406.7    520    1712
Percentage of the requests served within a certain time (ms)
  50%    520
  66%    558
  75%    602
  80%   1506
  90%   1526
  95%   1531
  98%   1535
  99%   1548
 100%   1712 (longest request)
real    0m5.313s
user    0m0.275s
sys     0m0.763s
```

出现了少量的报错：



图片 21.6 pic

gevent 可以加个队列，来限制协程的数目，但是数目限制了，虽然稳定了，但是并发数上不去。

```
from gevent.pool import Pool
pool = Pool(N)
```

这里测试有点简单，虽然已经安排了连接的堵塞，但是毕竟不符合业务。 有时间把后端的任务改成才 mongodb 取数据！

本文出自 “峰云，就她了。” 博客，谢绝转载!

22

利用 pypy 提高 python 脚本的执行速度及测试性能

啥是 pypy

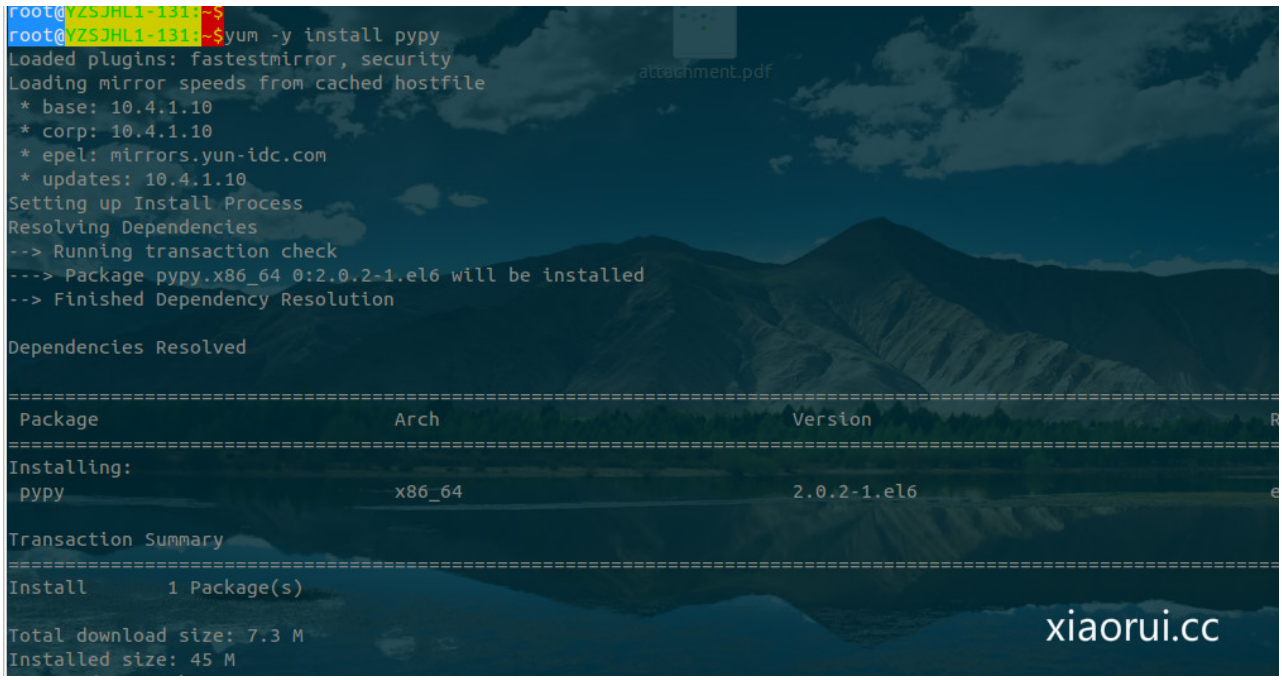
简单地说, PyPy 是用 Python 语言写了一个工具, 将 Python 代码成 C, .NET, Java 等语言和平台的代码. PHP 也有类似的项目 – HipHop-PHP, 把 PHP 代码转成 C++ 代码.

为什么要将一种语言转成另一种语言? 首先是目标语言可能在性能(如 C 语言)和/或跨平台(如 .NET, Java)等方面具有优势. 其次, 在转换的过程, 可以进行代码优化或加入新技术, 比如 PyPy 应用的 Just-in-Time(JIT) 技术, 能让 Python (事实上是转换后的目标代码)的执行速度更快.

反正是性能很好的东西，但是也有不少的局限性。

咱们就不用源码安装了，最新的版本是 2.2.1，但是看老外那边用 2.0 的还是比较的多。 有需要的朋友可以下载 2.2.1

地址：http://pypy.org/download.html



图片 22.1 pic

简单的跑一下:

```
import time
 #xiaorui.cc

 #rfyiamcool@163.com
```

```
def test(n,m):
    m=m
    vals = []
    keys = []
    for i in xrange(m):
        vals.append(i)
        keys.append('a%s'%i)
    d = None
    for i in xrange(n):
        d = dict(zip(keys, vals))
    return d
if __name__ == '__main__':
    st = time.time()
    print test(1000000,100)
    print 'use:', time.time() - st
```

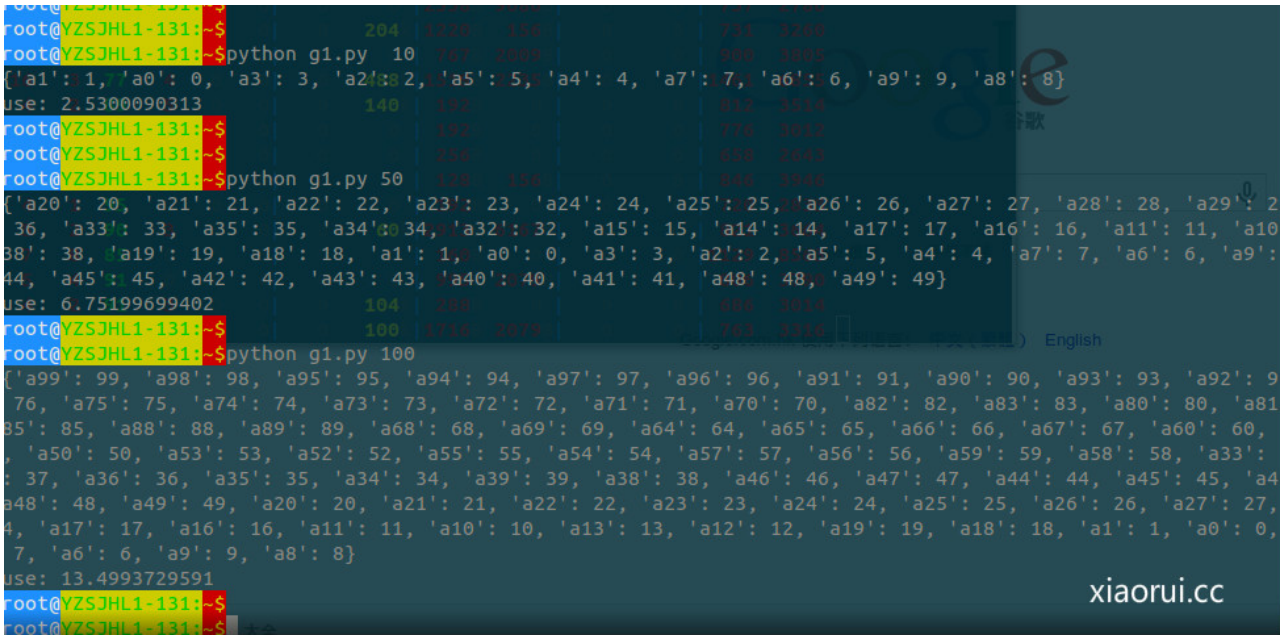
看看 pypy 和纯 python 执行的效率比较!

发现一个小规律，在小数据的时候，貌似 pypy 比率很大，但是运算多了后，貌似结果差距不算大。



图片 22.2 pic

这是用纯 python 执行的结果。



图片 22.3 pic

```
import time
from time import clock
 #xiaorui.cc

def check(num):
    a = list(str(num))
    b = a[::-1]
    if a == b:
        return True
    return False


def main():
    all = range(1,10**7)
    for i in all:
        if check(i):
            if check(i**2):
                print(i,i**2)


if __name__ == '__main__':
    start = clock()
    main()
    end = clock()
    print (end-start)
```



图片 22.4 pic

结果打出来是这样的 ~

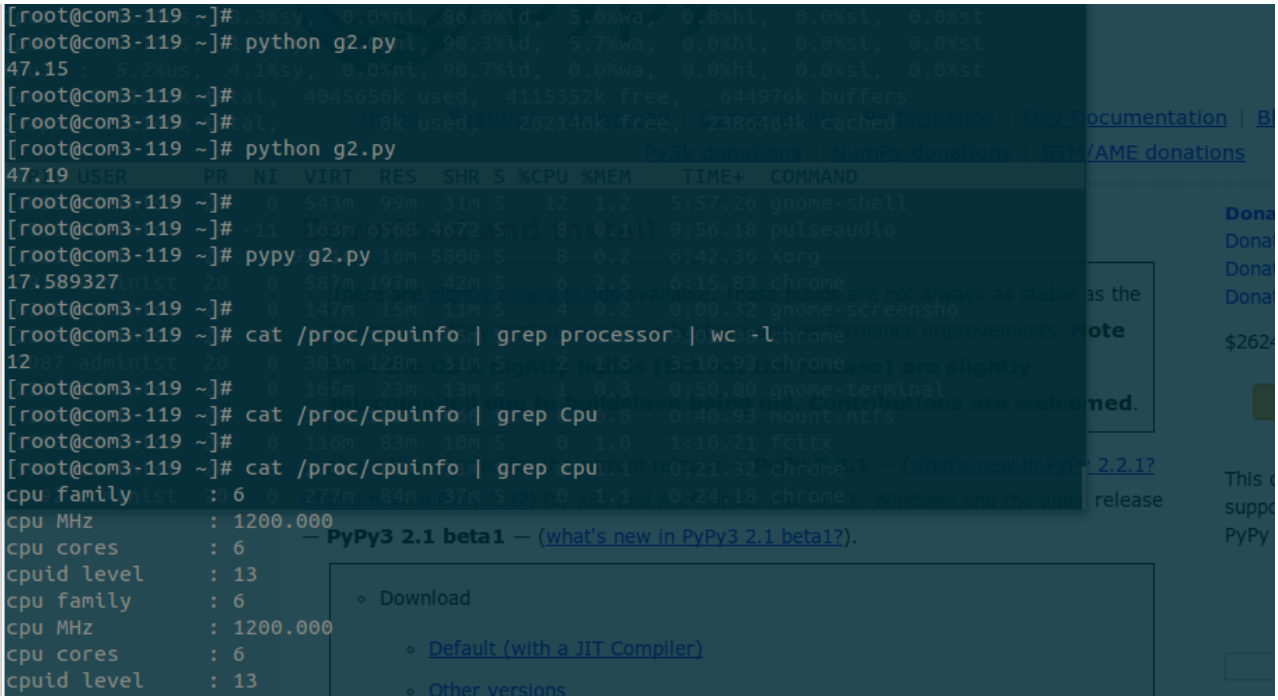
```
root@YZSJHL1-131:~$python g2.py
(1, 1)
(2, 4)
(3, 9)
(11, 121)
(22, 484)
(101, 10201)
(111, 12321)
(121, 14641)
(202, 40804)
(212, 44944)
(1001, 1002001)
(1111, 1234321)
(2002, 4008004)
(10001, 100020001)
(10101, 102030201)
(10201, 104060401)
(11011, 121242121)
(11111, 123454321)
(11211, 125686521)
(20002, 400080004)
(20102, 404090404)
(100001, 10000200001)
(101101, 10221412201)
(110011, 12102420121)
(111111, 12345654321)
(200002, 40000800004)
(1000001, 1000002000001)
(1001001, 1002003002001)
(1002001, 1004006004001)
(1010101, 1020304030201)
```

```
(1011101, 1022325232201)
(1012101, 1024348434201)
(1100011, 1210024200121)
(1101011, 1212225222121)
(1102011, 1214428244121)
(1110111, 1232346432321)
(1111111, 1234567654321)
(2000002, 4000008000004)
(2001002, 4004009004004)
```

不知道是不是服务器的性能不够高，找了台所谓高性能的服务器再跑跑：（公司刚买了一堆的华为服务器，挺好奇的，华为居然也出服务器了，找了台没上线的服务器测试下）

我晕，真垃圾的呀！看来这个机型适合做杂七杂八的业务的机型！

python 的因为是单核跑的，所以和 cpu 的主频有关联的！



图片 22.5 pic

有老外说 gevent 和 pypy 可以更好的提高效率，在 pypy 层次下也可以调用协程。等有结果在分享给大家！

pypy 对我来说，最大的缺点就是他的库支持的还是少，尤其是 socket 级别的包，不少的 bug。就算是最广泛的 Django 也是存在兼容的！

```
root@YZSJHL1-131:~$pypy myapp.py
Traceback (most recent call last):
  File "app_main.py", line 72, in run_toplevel
  File "myapp.py", line 2, in <module>
```

```
    from flask import Flask
ImportError: No module named flask
root@YZSJHL1-131:~$
```

好，先说到这里！

本文出自 “峰云，就她了。” 博客，谢绝转载！

23

python 实现 select 和 epoll 模型 socket 网络编程

这里简单搞搞 select 和 eopll 的接口开发 ~

select 目前几乎在所有的平台上支持，其良好跨平台支持也是它的一个优点，事实上从现在看来，这也是它所剩不多的优点之一，现在其实更多的人用 epoll，在 python 下 epoll 文档有点少，就先讲究搞搞 select ~

select 的一个缺点在于单个进程能够监视的文件描述符的数量存在最大限制，在 Linux 上一般为 1024，不过可以通过修改宏定义甚至重新编译内核的方式提升这一限制。

说点我的理解，要是用烦了多线程的网络编程，可以试试 select 的模型。

传递给 select 的参数是几个列表，分别表示读事件、写事件和错误事件。select 方法返回三个列表，其中包含满足条件的对象（读、写和异常）。

服务端的代码：

```
 #coding:utf-8

import socket,select
import time
import os
 #xiaorui.cc

host = "localhost"
port = 50000
s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.bind((host,port))
s.listen(5)
while 1:
    infds,outfds,errfds = select.select([s,],[],[],5)
    if len(infds) != 0:
      clientsock,clientaddr = s.accept()
      buf = clientsock.recv(8196)
      if len(buf) != 0:
         print (buf)
         os.popen('sleep 10').read()
      clientsock.close()
 #    print "no data coming"
```

客户端的代码：

```
 #coding:utf-8

import socket,select
 #xiaorui.cc
```

```
host = "localhost"
port = 50000
s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect((host,port))

s.send("coming from select client")
s.close()
```



图片 23.1 pic

一个完成的 select 的例子:

这里有队列的概念

```
 #

import select
import socket
import Queue
import time
import os
 #创建 socket 套接字

server = socket.socket(socket.AF_INET,socket.SOCK_STREAM)
server.setblocking(False)
 #配置参数

server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR  , 1)
server_address= ('192.168.0.101',9999)
server.bind(server_address)
server.listen(10)
inputs = [server]
outputs = []
message_queues = {}
 #timeout = 20
```

```
while inputs:
    print "waiting for next event"
#    readable , writable , exceptional = select.select(inputs, outputs, inputs, timeout) 最后一个是超时，当前连接要是超过这个

    readable , writable , exceptional = select.select(inputs, outputs, inputs)

    # When timeout reached , select return three empty lists
    if not (readable or writable or exceptional) :
        print "Time out ! "
        break;
    for s in readable :
        if s is server:
            #通过 inputs 查看是否有客户端来
            connection, client_address = s.accept()
            print "    connection from ", client_address
            connection.setblocking(0)
            inputs.append(connection)
            message_queues[connection] = Queue.Queue()
        else:
            data = s.recv(1024)
            if data :
                print " received " , data , "from ",s.getpeername()
                message_queues[s].put(data)
                # Add output channel for response
                if s not in outputs:
                    outputs.append(s)
            else:
                #Interpret empty result as closed connection
                print "  closing", client_address
                if s in outputs :
                    outputs.remove(s)
                inputs.remove(s)
                s.close()
                #清除队列信息
                del message_queues[s]
    for s in writable:
        try:
            next_msg = message_queues[s].get_nowait()
        except Queue.Empty:
            print " " , s.getpeername() , 'queue empty'
            outputs.remove(s)
        else:
            print " sending " , next_msg , " to ", s.getpeername()
            os.popen('sleep 5').read()
            s.send(next_msg)
```

```
    for s in exceptional:
        print " exception condition on ", s.getpeername()
        #stop listening for input on the connection
        inputs.remove(s)
        if s in outputs:
            outputs.remove(s)
        s.close()
        #清除队列信息
        del message_queues[s]
```

关于 epoll 的方面，大家可以看看这个老外的文档，写不错 ~

select 是轮询、epoll 是触发式的，所以 epoll 的效率高。

参考的文档地址：http://scotdoyle.com/python-epoll-howto.html

下面是用 epoll 实现一个服务端 ~

blog from xiaorui.cc

```
import socket, select
EOL1 = b'\n\n'
EOL2 = b'\n\r\n'
response  = b'HTTP/1.0 200 OK\r\nDate: Mon, 1 Jan 1996 01:01:01 GMT\r\n'
response += b'Content-Type: text/plain\r\nContent-Length: 13\r\n\r\n'
response += b'Hello, world!'
serversocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
serversocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
serversocket.bind(('0.0.0.0', 8080))
serversocket.listen(1)
serversocket.setblocking(0)
epoll = select.epoll()
epoll.register(serversocket.fileno(), select.EPOLLIN)
try:
  connections = {}; requests = {}; responses = {}
  while True:
    events = epoll.poll(1)
    for fileno, event in events:
      if fileno == serversocket.fileno():
        connection, address = serversocket.accept()
        connection.setblocking(0)
        epoll.register(connection.fileno(), select.EPOLLIN)
        connections[connection.fileno()] = connection
        requests[connection.fileno()] = b''
        responses[connection.fileno()] = response
```

```
        elif event & select.EPOLLIN:
            requests[fileno] += connections[fileno].recv(1024)
            if EOL1 in requests[fileno] or EOL2 in requests[fileno]:
                epoll.modify(fileno, select.EPOLLOUT)
                connections[fileno].setsockopt(socket.IPPROTO_TCP, socket.TCP_CORK, 1)
                print('-'*40 + '\n' + requests[fileno].decode()[:-2])
        elif event & select.EPOLLOUT:
            byteswritten = connections[fileno].send(responses[fileno])
            responses[fileno] = responses[fileno][byteswritten:]
            if len(responses[fileno]) == 0:
                connections[fileno].setsockopt(socket.IPPROTO_TCP, socket.TCP_CORK, 0)
                epoll.modify(fileno, 0)
                connections[fileno].shutdown(socket.SHUT_RDWR)
        elif event & select.EPOLLHUP:
            epoll.unregister(fileno)
            connections[fileno].close()
            del connections[fileno]
finally:
    epoll.unregister(serversocket.fileno())
```

Epoll 的最大好处是不会随着FD的数目增长而降低效率，在 select 中采用轮询处理，每个 fd 的处理情况，而 epoll 是维护一个队列，直接看队列是不是空就可以了。

在这里也推荐大家用 epoll 写服务端的东西，当然我自己理解的不够好，咱们多交流！！！

本文出自 “峰云，就她了。” 博客，谢绝转载!

24

对 Python-memcache 分布式散列和调用的实现

煮酒品茶：对 python-memcache 进行学习，把分布式 HASH 算法加进去，不说线上自己玩玩的程序可以加到里面去。memcached 读存数据就这些东西，看着补。

分布式一致性 HASH 算法：memcache_ring.py

```
 #coding:utf8

import hash_ring
import memcache
memcache_servers = [
       '127.0.0.1:11211']
weights = {
       '127.0.0.1:11211':1}
ring = hash_ring.HashRing(memcache_servers,weights)
 #if value is null then get else set

def mc(key,value="Null-0"):
    server_node = ring.get_node(key)
    mc = memcache.Client([server_node],debug=1)
    if value == "Null-0":
        return mc.get(key)
    else:
        return mc.set(key,value)
```

一致性 hash 读取数据：

```
In [14]: from memcache_ring import mc

In [15]: mc("key","I'm here")
Out[15]: True

In [16]: mc("key")
Out[16]: "I'm here"

In [17]: key = "Testdict"

In [18]: value = {"test":"dict","test2":"dict2"}

In [19]: mc(key,value)
Out[19]: True

In [20]: a = mc(key)

In [21]: type(a)
Out[21]: dict
```

图片 24.1 pic

从数据库中读取数据

sql:

```
#Hash
mysql> use test;
mysql> select * from zwhset;
+----+------+
| id | name |
+----+------+
|  1 | a    |
|  2 | b    |
|  3 | d    |
|  4 | e    |
|  5 | f    |
+----+------+
5 rows in set (0.00 sec
```

图片 24.2 pic

```
In [32]: from memcache_ring import mc

In [33]: import MySQLdb
ll()
mc(key,value)
mc(key)
In [34]: conn = MySQLdb.connect(host="localhost",user="root",passwd="[redacted]",db="test",chars
et="utf8")

In [35]: cursor = conn.cursor()

In [36]: sql = "select * from zwhset"

In [37]: cursor.execute(sql)
Out[37]: 5L

In [38]: key = str(hash(sql))

In [39]: value = cursor.fetchall()

In [40]: mc(key,value)
Out[40]: True

In [41]: mc(key)
Out[41]: ((1L, u'a'), (2L, u'b'), (3L, u'd'), (4L, u'e'), (5L, u'f'))
```

图片 24.3 pic

可利用起来的程序，稍改动加一些 try 之类的就可以用到自己的程序玩了。else下key =str(hash(sql))可以去掉。不知道为啥不好册。

```
#coding:utf8

from memcache_ring import mc
import MySQLdb
```

```
 #如果在 memcache 中就不查数据库，不在就取出来并存一份

sql = "select * from zwhset where id=100"
key = str(hash(sql))
 #查数据库

def select_sql(sql):
    conn = MySQLdb.connect(host="localhost",user="root",passwd="",db="test",charset="utf8")
    cursor = conn.cursor()
    cursor.execute(sql)
    value = cursor.fetchall()
    #如果没有查到数据，则原值返回
    if not value:
        return value
    else:
        key = str(hash(sql))
        #把存储的结果给调用程序
        return mc(key,value)
 #读数据，先看 memcached里有没有，有就直接返回 memcached 查的值，没有就查数据库，

 #如果数据库也返回空的话原值返回，如果有值就写 memcached,然后把 value 返回

if not mc(key):
    select_sql(sql)
else:
    mc(key)
```

25

Parallel Python 实现程序的并行多 cpu 多核利用【pp 模块】

为啥要这个模块：

Python 是解释型的语言,而 Python 解释器使用 GIL(全局解 释器锁)来在内部禁止并行执行,正是这个 GIL 限制你在多核处理器上同一时间也只能执行一条字节码指令. 听朋友说 python 3.0 里面已经改进, 默认有了多处理器编程的库了. Python2.XX 暂时还不支持。

Parallel Python 这个库,正是为支持 smp 多路多核多 cpu 而设计的, 而且它不仅可以多核处理器协同工作,还可以通过网络集群运行。

官网：
http://www.parallelpython.com/

pp 模块的简介

PP 是一个 Python 模块，提供了在 SMP(多 CPU 或多核)和集群(通过网络连接的多台计算机)上并行执行 Python 代码的机制。轻量级，易于安装，并 集成了其他软件。PP 也是一个用纯 Python 代码实现的跨平台，开放源码模块。

下面是看起来很高端的功能介绍!

- 在 SMP 和集群上并行执行 Python 代码
- 易于理解和实现的基于工作的并行机制，便于把穿行应用转换成并行的
- 自动构造最佳配置(默认时工作进程数量等同于系统处理器数量)
- 动态处理器分配(允许运行时改变工作处理器数量)
- 函数的工作缓存(透明的缓存机制确保后续调用降低负载)
- 动态负载均衡(任务被动态的分配到各个处理器上)
- 基于 SHA 的连接加密认证
- 跨平台移植(Windows/Linux/Unix)
- 开放源代码

有些朋友可能对并发和并行理解有点乱，在这里梳理下哈:

当有多个线程在操作时,如果系统只有一个 CPU,则它根本不可能真正同时进行一个以上的线程,它只能把 CPU 运行时间划分成若干个时间段,再将时间 段分配给各个线程执行,在一个时间段的线程代码运行时,其它线程处于挂起状态.这种方式我们称之为并发(Concurrent).
当系统有一个以上 CPU 时,则线程的操作有可能非并发.当一个 CPU 执行一个线程时,另一个 CPU 可以执行另一个线程,两个线程互不抢占 CPU 资源,可以同时进行,这种方式我们称之为并行(Parallel)

地址：http://rfyiamcool.blog.51cto.com/1030776/1357112

多线程在并发和并行环境中的不同作用

在并发环境时,多线程不可能真正充分利用 CPU,节约运行时间,它只是以”挂起->执行->挂起”的方式以很小的时间片分别运行各个线程,给用户以每个线程都在运行的错觉.在这种环境中,多线程程序真正改善的是系统的响应性能和程序的友好性.
在并行环境中, 一个时刻允许多个线程运行,这时多线程程序才真正充分利用了多 CPU 的处理能力, 节省了整体的运行时间.在这种环境中,多线程程序能体现出它的四大优势:充分利用 CPU,节省时间,改善响应和增加程序的友好性.

并发和并行的区别就是一个处理器同时处理多个任务和多个处理器或者是多核的处理器同时处理多个不同的任务。

前者是逻辑上的同时发生（simultaneous），而后者是物理上的同时发生.

来个比喻：并发和并行的区别就是一个人同时吃三个馒头和三个人同时吃三个馒头。

咱们跑一下:

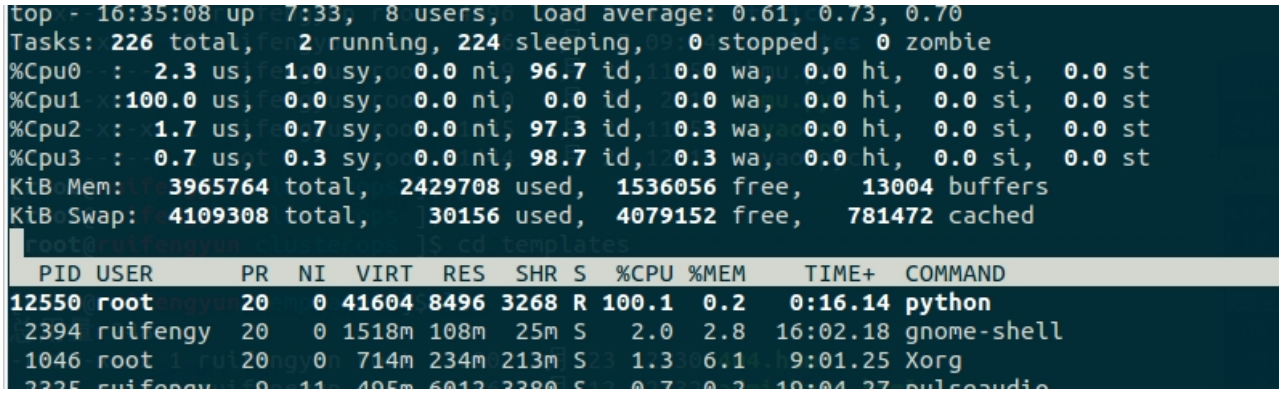
安装 pp 模块



图片 25.1 pic

好，测试

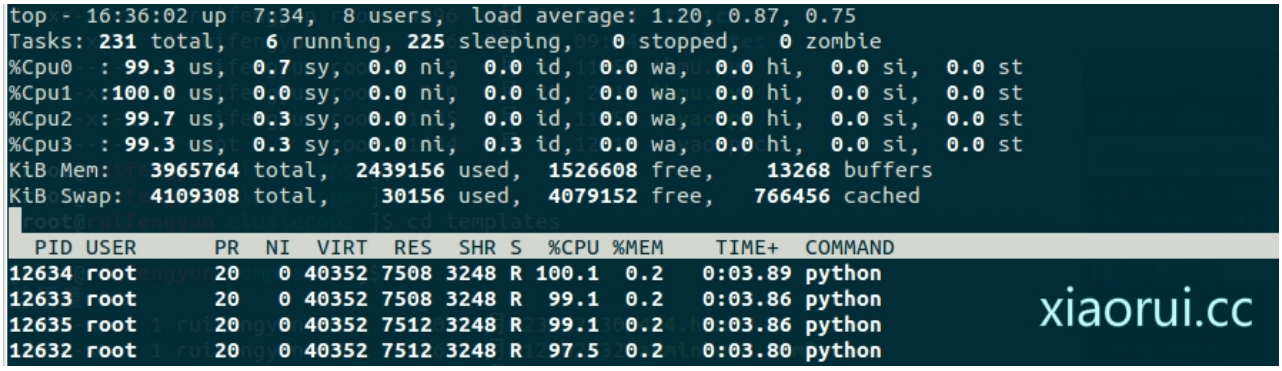
【这是单核的利用情况下】



图片 25.2 pic

开启 python 多核的情况下

【看到没有，多出了 4 个进程，这四个进程是绑在不同的 cpu上，这四个 cpu 跑的很均匀】



图片 25.3 pic

测试的代码：

```
 #-*- coding: UTF-8 -*-

import math, sys, time
import pp
def IsPrime(n):
    """返回 n 是否是素数"""
    if not isinstance(n, int):
        raise TypeError("argument passed to is_prime is not of 'int' type")
    if n < 2:
        return False
    if n == 2:
        return True
    max = int(math.ceil(math.sqrt(n)))
    i = 2
    while i <= max:
        if n % i == 0:
            return False
```

```
        i += 1
    return True
def SumPrimes(n):
    for i in xrange(15):
        sum([x for x in xrange(2,n) if IsPrime(x)])
    """计算从 2-n 之间的所有素数之和"""
    return sum([x for x in xrange(2,n) if IsPrime(x)])
inputs = (100000, 100100, 100200, 100300, 100400, 100500, 100600, 100700)
start_time = time.time()
for input in inputs:
    print SumPrimes(input)
print '单线程执行，总耗时', time.time() - start_time, 's'
 # tuple of all parallel python servers to connect with

ppservers = ()
 #ppservers = ("10.0.0.1",)

if len(sys.argv) > 1:
    ncpus = int(sys.argv[1])
    # Creates jobserver with ncpus workers
    job_server = pp.Server(ncpus, ppservers=ppservers)
else:
    # Creates jobserver with automatically detected number of workers
    job_server = pp.Server(ppservers=ppservers)
print "pp 可以用的工作核心线程数", job_server.get_ncpus(), "workers"
start_time = time.time()
jobs = [(input, job_server.submit(SumPrimes,(input,), (IsPrime,), ("math",))) for input in inputs]
for input, job in jobs:
    print "Sum of primes below", input, "is", job()
print "多线程下执行耗时: ", time.time() - start_time, "s"
job_server.print_stats()
```

pp 的官方例子给的很足，也相当的简练！

导入让 python 支持多核心的模块

1) Import pp module:

import pp

一个实例，含有cpu的核心数目 2) Start pp execution server with the number of workers set to the numb er of processors in the system

job_server = pp.Server()

可以支持网络的分布式运算！

```
 # tuple of all parallel python servers to connect with

ppservers = ()
 #ppservers = ("10.0.0.1",)

ncpus = int(sys.argv[1])
 # Creates jobserver with ncpus workers

job_server = pp.Server(ncpus, ppservers=ppservers)
```

3) Submit all the tasks for parallel execution:

f1 = job_server.submit(func1, args1, depfuncs1, modules1)

f2 = job_server.submit(func1, args2, depfuncs1, modules1)

f3 = job_server.submit(func2, args3, depfuncs2, modules2)
...etc...

4) Retrieve the results as needed:

r1 = f1()

r2 = f2()

r3 = f3()

...etc...

刚才说了 parallet python 是支持网络的分布式运算集群的!

Advanced guide, clusters

On the nodes

1) Start parallel python execution server on all your remote computational nodes (listen to a given port 35000, and local network interface only, accept only connections which know correct secret):

node-1> ./ppserver.py -p 35000 -i 192.168.0.101 -s "mysecret"

node-2> ./ppserver.py -p 35000 -i 192.168.0.102 -s "mysecret"

node-3> ./ppserver.py -p 35000 -i 192.168.0.103 -s "mysecret"

On the client

2) Import pp module:

```
import pp
```

3) Create a list of all the nodes in your cluster (computers where you've run ppserver.py)

```
ppservers=("node-1:35000", "node-2:35000", "node-3:35000")
```

4) Start pp execution server with the number of workers set to the number of processors in the syste m, list of ppservers to connect with and secret key to authorize the connection:

```
job_server = pp.Server(ppservers=ppservers, secret="mysecret")
```

5) Submit all the tasks for parallel execution:

```
f1 = job_server.submit(func1, args1, depfuncs1, modules1)

f2 = job_server.submit(func1, args2, depfuncs1, modules1)

f3 = job_server.submit(func2, args3, depfuncs2, modules2) ...etc...
```

6) Retrieve the results as needed:

```
r1 = f1()

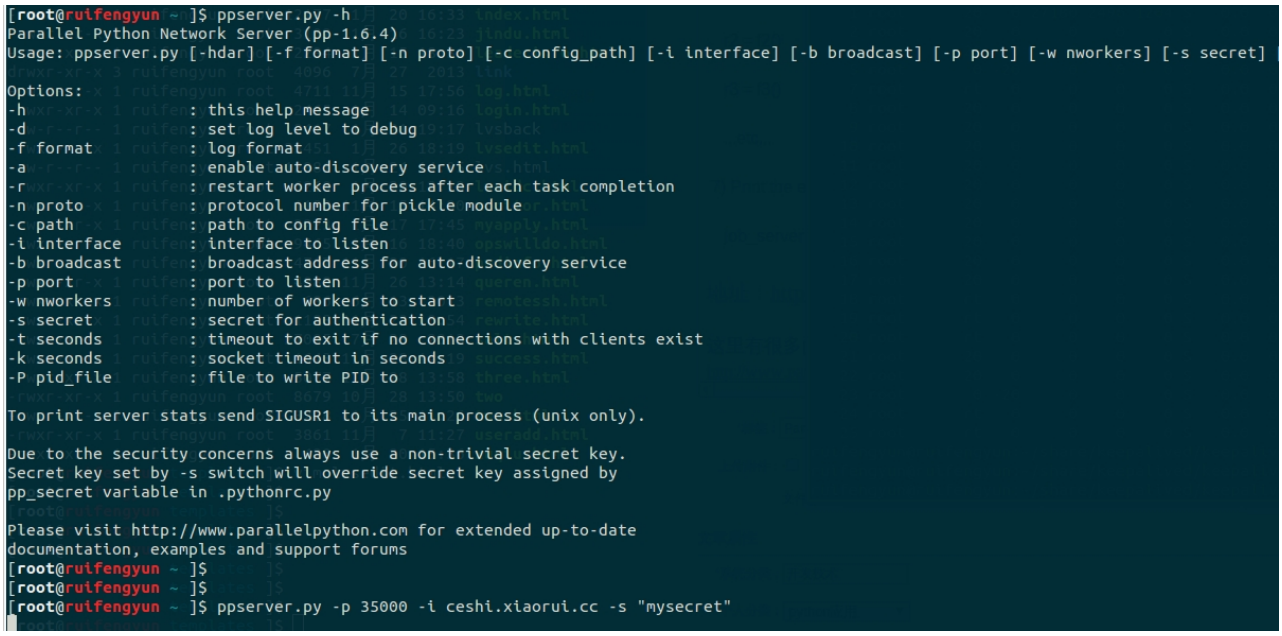
r2 = f2()

r3 = f3()

...etc...
```

7) Print the execution statistics:

```
job_server.print_stats()
```



图片 25.4 pic

地址：http://rfyiamcool.blog.51cto.com/1030776/1357112

这里有很多的例子，大家可以跑跑！

http://www.parallelpython.com/content/view/17/31/

安装 python 的 pp 模块直接运行就行

本文出自 “峰云，就她了。” 博客，谢绝转载！

26

关于 python 调用 zabbix api 接口的自动化实例 [结合 saltstack]

前言：

这两天一直做一个叫集群配置管理平台的自动化项目，写了有 20 多天了，项目做的还算顺利，只是一堆的接口需要写，有点烦。因为 clusterops 项目到最后肯定是要和监控平台做结合的，这两天也抽时间看了下。以前自己也写过不少类似 zabbix 的接口调用教程，当时看的时候，由于时间有限，也都是草草跑 demo。

请大家多关注下我的独立博客，更多的关于 zabbix 二次开发的话题，http://xiaorui.cc

zabbix 的接口挺好理解，任何的程序都可以写，甚至是 linux 的 curl 命令。我这边用 python 的 urllib、urllib2 来搞的，当然会 php 的就更好了，因为 zabbix 的接口是 php 写的，懂 php 可以直接用现成的。

zabbix 官网有大量的接口，你只要会用 zabbix，然后看下 api 的说明，应该就没啥问题了
https://www.zabbix.com/documentation/1.8/api

简单说三个例子，入个门。

获取 KEY

```
!/usr/bin/env python2.7
 #coding=utf-8

import json
import urllib2
 # based url and required header

url = "http://monitor.example.com/api_jsonrpc.php"
header = {"Content-Type": "application/json"}
 # auth user and password

data = json.dumps(
{
    "jsonrpc": "2.0",
    "method": "user.login",
    "params": {
    "user": "Admin",
    "password": "zabbix"
},
"id": 0
})
 # create request object

request = urllib2.Request(url,data)
```

```
for key in header:
    request.add_header(key,header[key])
 # auth and get authid

try:
    result = urllib2.urlopen(request)
except URLError as e:
    print "Auth Failed, Please Check Your Name And Password:",e.code
else:
    response = json.loads(result.read())
    result.close()
    print "Auth Successful. The Auth ID Is:",response['result']
```

获取 hostlist

```
 #!/usr/bin/env python2.7

 #coding=utf-8

import json
import urllib2
 #xiaorui.cc

url = "http://10.10.10.61/api_jsonrpc.php"
header = {"Content-Type": "application/json"}
 # request json

data = json.dumps(
{
    "jsonrpc":"2.0",
    "method":"host.get",
    "params":{
        "output":["hostid","name"],
        "filter":{"host":""}
    },
    "auth":"dbcd2bd8abc0f0320fffab34c6d749d3",
    "id":1,
})
 # create request object

request = urllib2.Request(url,data)
for key in header:
    request.add_header(key,header[key])
 # get host list

try:
```

```
    result = urllib2.urlopen(request)
except URLError as e:
    if hasattr(e, 'reason'):
        print 'We failed to reach a server.'
        print 'Reason: ', e.reason
    elif hasattr(e, 'code'):
        print 'The server could not fulfill the request.'
        print 'Error code: ', e.code
else:
    response = json.loads(result.read())
    result.close()
    print "Number Of Hosts: ", len(response['result'])
    for host in response['result']:
        print "Host ID:",host['hostid'],"Host Name:",host['name']
```

添加主机

```
 #!/usr/bin/env python2.7

 #coding=utf-8

import json
import urllib2
 #xiaorui.cc

url = "http://10.10.10.61/api_jsonrpc.php"
header = {"Content-Type": "application/json"}
 # request json

data = json.dumps(
{
    "jsonrpc":"2.0",
    "method":"host.create",
    "params":{
        "host": "10.10.10.67","interfaces":
        [{"type": 1,"main": 1,"useip": 1,"ip": "10.10.10.67","dns": "","port": "10050"}],
        "groups": [{"groupid": "2"}],"templates": [{"templateid": "10087"}]
        },
    "auth":"dbcd2bd8abc0f0320fffab34c6d749d3",
    "id":1,
}
)
 # create request object

request = urllib2.Request(url,data)
for key in header:
```

```
    request.add_header(key,header[key])
 # get host list

try:
    result = urllib2.urlopen(request)
except URLError as e:
    if hasattr(e, 'reason'):
        print 'We failed to reach a server.'
        print 'Reason: ', e.reason
    elif hasattr(e, 'code'):
        print 'The server could not fulfill the request.'
        print 'Error code: ', e.code
else:
    response = json.loads(result.read())
    result.close()
    print 'ok'zai
```

原文： http://rfyiamcool.blog.51cto.com/1030776/1358792

我个人觉得 zabbix 的 rest api 难点在于 key 相关的认证，会了之后，再看官网的 api 文档就一目了然了。

啥时候用?

在我的集群平台下，我可以把暂时下线的服务器，在平台上去除，但是大家有没有想到，你要是吧主机删掉后，监控端会有一堆的通知发给你，所以，在处理主机的时候，顺便调用 zabbix 的接口，把该主机的监控项目给删掉。

在我通过 saltstack 添加 lvs 后端主机的时候，我也同样可以调用接口，把后端的主机相应的监控都给加进去。

就先这样，有时间再丰富下该文章。

本文出自 “峰云，就她了。” 博客，谢绝转载!

27

Python 批量更新 nginx 配置文件

工作需要检查线上所有服务器 ngxin 的 host 配置，是否都添加禁止访问目录中带 /.svn/ 和以 tar.gz、tar、zip、等结尾 url，如果没有则添加，由于线上 Nginx 服务器将近百台，每台的 nginx 配置至少 10 几个，手工检查太慢了，本人也不想浪费太多时间做这些无用功。故用 python 写了一个检测脚本。来完成这些无聊事情。

想用 python 完成这些事情，思路大概为：先备份每台服务器原来的配置，然后遍历每台服务器内所有 host 配置，正则匹配 host 配置看是否已经添加相关配置，如有则跳过，遇到有但不全或没有的则在对应的位置上插入相应的配置。除文本插入时我遇到一个问题外，其他比较简单。因为 python 目前没有对文本进行插入操作的模块。整个插入过程需要自己一步一步实现。

当时我考虑文本插入思路：把原配置文件载入内存，正则找到要插入的位置，以此中心把原文件分成两部分，开辟新内存空间按顺序先存放第一部内容，然后存放出插入的内容，然后存放第二部内容。最后 flush 到硬盘。即通俗说法把原配置根据正则进行重定向。

我了解实现这种重定向有两个方法，第一种: 打开原文件，根据规则重定向到新建的文件上。第二种：使用 fileinput 对原文件重定向。

下面是代码，实现过程大概为：先判断目录 nginx 配置目录，如果存在则备份，然后遍历所有配置文件，匹配每个配置文件是否已添加相关内容，如果没有添加或者之前添加了部分内容，找到配置文件第一个花括号“}”，下一行插入相应的内容。

```
 #!/usr/bin/python

 #coding:utf-8

 #Author by Qfeian@20140215

"""
First backup file and then update all nginx configure
in the path "/usr/local/nginx/conf/vhosts".
Insert rule is to find the configure of the first "}"
after inserting content
"""
import fileinput
import os
import re
import sys
import string
import time
from subprocess import call
s_all = """
    location ~* ^(.*)\/\.svn\/ {
            deny all;
        }
```

```
        location ~* \.(tar|gz|zip|tgz|sh)$ {
                deny all;
        }
"""
s_svn = """
    location ~* ^(.*)\/\.svn\/ {
                deny all;
        }
"""
s_tar = """
        location ~* \.(tar|gz|zip|tgz|sh)$ {
                deny all;
        }
"""
def file_insert(fname, str):
        r = ur'}'
        f = open(fname)
        old = f.read()
        num = int(re.search(r, old).start())
        f_input = fileinput.input(fname, inplace=1)
        #for line in fileinput.input(fname, inplace=1):
        for line in f_input:
           if r in line:
              print line.rstrip()
              print "\n"+str+"\n"
              f.seek(num+2)
              print f.read()
              break
           else:
              print line.rstrip()
        f.close()
        f_input.close()
        #print "OK! The %s configure has been sucessfull added!" % fname
        print "Ok! 配置文件%s已经添加成功！" % fname
def file_list(f_dir):
    #Check the content of configure,and add the correcsponding content
    rsvn = ur'\\\/\\.svn\\\/'
    rtar = ur'\(tar\|gz\|zip\|tgz\|sh\)'
    if os.path.exists(f_dir): #check dir
        for f_name in os.listdir(f_dir): #list the dir all configure file
          f_path = os.path.join(f_dir,f_name) #Get the filename full path
          f1 = open(f_path)
          f1_old = f1.read()
          f1.close()
          if re.findall(rsvn, f1_old) and re.findall(rtar, f1_old):
```

```
            #print "Notice!  %s have been added ,ignore this configure ...." % f_path
            print "Notice! %s 已添加过相关的配置，忽略此配置文件" % f_path
            continue
        elif re.findall(rsvn, f1_old):
            file_insert(f_path, s_tar)
        elif re.findall(rtar, f1_old):
            file_insert(f_path, s_svn)
        else:
            file_insert(f_path, s_all)
    else:
        print "Warnning! dir %s isn't exists!" % f_dir
def file_backup(f_dir):
    # If the file has been backed up with a minute before,will not back up
    bak_dir = "/data/nginx_config_bak/" + time.strftime('%Y%m%d_%H_%M')
    if not os.path.exists(bak_dir):
        os.makedirs(bak_dir)
        if os.path.exists(f_dir):
            cp = "cp -rp"
            cmd = "%s %s %s" % (cp, f_dir, bak_dir)
            call(cmd, shell=True) #backup the configure
if __name__ == "__main__":
    f_dir = "/usr/local/nginx/conf/vhosts"
    file_backup(f_dir)
    file_list(f_dir)
    print "-"*90,"\n 所有 Nginx 配置文件更新完成 !\n"
```

参考：

http://ryanstd.iteye.com/blog/480781

http://docs.python.org/2/library/fileinput.html

28

Python 通过 amqp 消息队列协议中的 Qpid 实现数据通信

简介:

这两天看了消息队列通信，打算在配置平台上应用起来。以前用过 zeromq 但是这东西太快了，还有就是 rabbit mq 有点大，新浪的朋友推荐了 qpid，简单轻便。自己总结了下文档，大家可以瞅瞅。

AMQP（消息队列协议 Advanced Message Queuing Protocol）是一种消息协议，等同于 JMS，但是 JMS 只是 java 平台的方案，AMQP 是一个跨语言的协议。

AMQP 不分语言平台,主流的语言都支持，运维这边的 perl，python，ruby 更是支持，所以大家就放心用吧。

主流的消息队列通信类型:

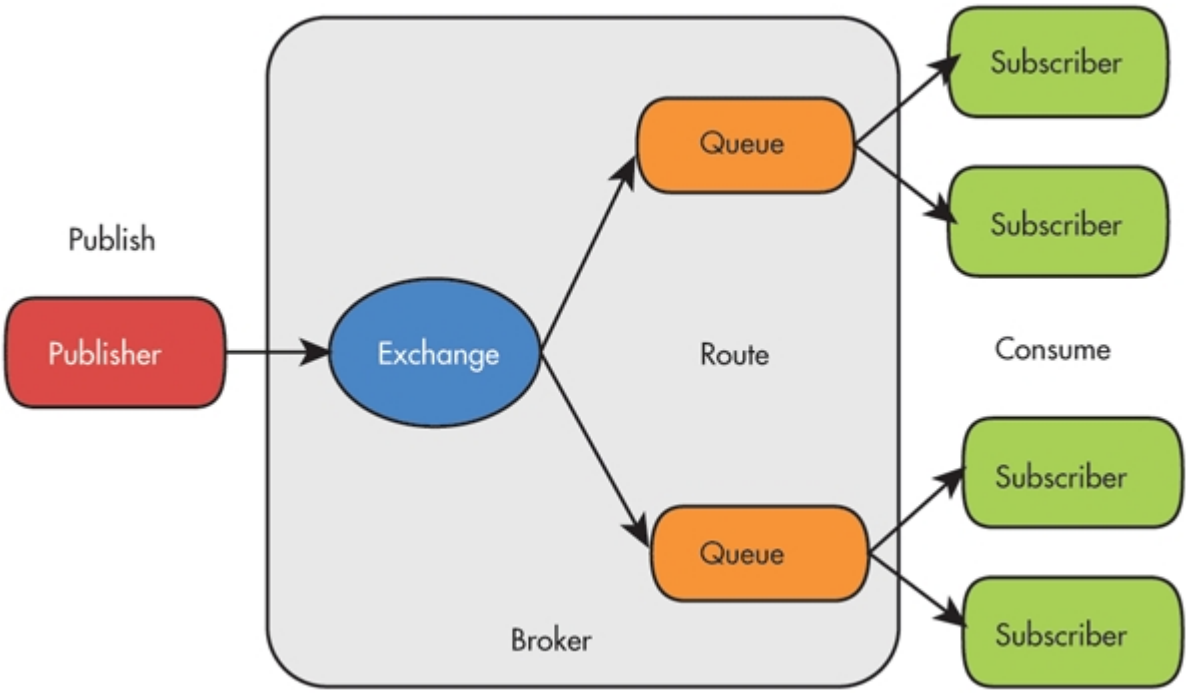
```
点对点：A 发消息给 B。
广播：A 发给所有其他人的消息
组播：A 发给多个但不是所有其他人的消息。
Requester/response：类似访问网页的通信方式，客户端发请求并等待，服务端回复该请求
Pub-sub：类似杂志发行，出版杂志的人并不知道谁在看这本杂志，订阅的人并不关心谁在发表这本杂志。出版的人只管将信息发布出
Store-and-forward：存储转发模型类似信件投递，写信的人将消息写给某人，但在将信件发出的时候，收信的人并不一定在家等待，
其他通信模型。。。
```

Publisher --->Exchange ---> MessageQueue --->Consumer

整个过程是异步的.Publisher,Consumer 相互不知道对方的存在，Exchange 负责交换/路由，依靠 Routing Key,每个消息者有一个 Routing Key，每个 Binding 将自已感兴趣的 RoutingKey 告诉 Exchange,以便 Exchange 将相关的消息转发给相应的 Queue!

几个概念

几个概念
Producer,Routing Key,Exchange,Binding，Queue,Consumer.
Producer: 消息的创建者，消息的发送者
Routing Key：唯一用来映射消息该进入哪个队列的标识
Exchange：负责消息的路由，交换
Binding:定义 Queue 和 Exchange 的映射关系
Queue：消息队列
Consumer：消息的使用者
Exchange类型
Fan-Out:类似于广播方式，不管 RoutingKey
Direct:根据 RoutingKey,进行关联投寄
Topic:类似于 Direct,但是支持多个 Key 关联，以组的方式投寄。
　　key以.来定义界限。类似于 usea.news,usea.weather.这两个消息是一组的。



图片 28.1 pic

QPID

Qpid 是 Apache 开发的一款面向对象的消息中间件，它是一个 AMQP 的实现，可以和其他符合 AMQP 协议的系统进行通信。Qpid 提供了 C++/Python/Java/C# 等主流编程语言的客户端库，安装使用非常方便。相对于其他的 AMQP 实现，Qpid 社区十分活跃，有望成为标准 AMQP 中间件产品。除了符合 AMQP 基本要求之外，Qpid 提供了很多额外的 HA 特性，非常适于集群环境下的消息通信!

基本功能外提供以下特性：

采用 Corosync(?)来保证集群环境下的 Fault-tolerant(?) 特性
支持 XML 的 Exchange,消息为 XML 时，彩用 Xquery 过滤
支持 plugin
提供安全认证，可对 producer/consumer 提供身份认证
qpidd --port --no-data-dir --auth
port:端口
--no-data-dir:不指定数据目录
--auth：不启用安全身份认证

启动后自动创建一些 Exchange,amp.topic,amp.direct,amp.fanout

tools:

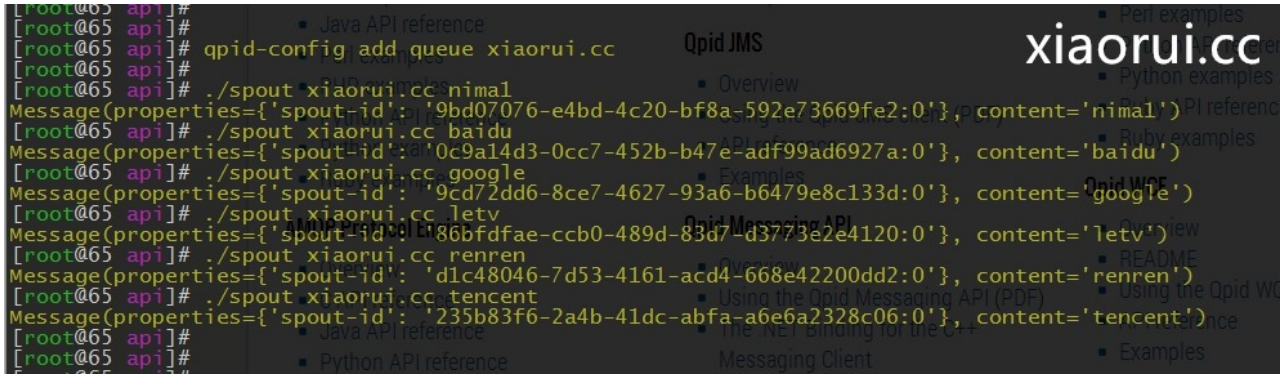
Qpid-config:维护 Queue,Exchange,内部配置 Qpid-route:配置 broker Federation(联盟? 集群?) Qpid-tool:监控

咱们说完介绍了，这里就赶紧测试下。

服务器端的安装：

```
yum install qpid-cpp-server
yum install qpid-tools
/etc/init.d/qpidd start
```

发布端的测试代码：



图片 28.2 pic

一些测试代码转自: ibm 的开发社区

```
#!/usr/bin/env python
```

```
 #xiaorui.cc

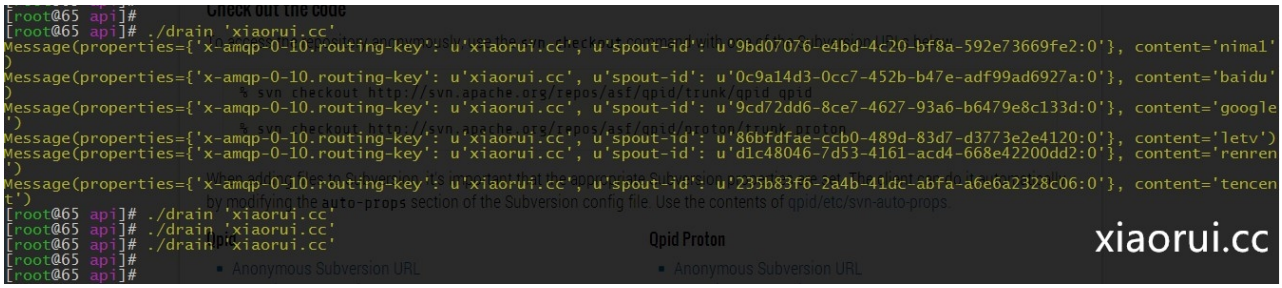
 #转自 ibm 开发社区

import optparse, time
from qpid.messaging import *
from qpid.util import URL
from qpid.log import enable, DEBUG, WARN
def nameval(st):
  idx = st.find("=")
  if idx >= 0:
    name = st[0:idx]
    value = st[idx+1:]
  else:
    name = st
    value = None
  return name, value
parser = optparse.OptionParser(usage="usage: %prog [options] ADDRESS [ CONTENT ... ]",
                               description="Send messages to the supplied address.")
parser.add_option("-b", "--broker", default="localhost",
                  help="connect to specified BROKER (default %default)")
parser.add_option("-r", "--reconnect", action="store_true",
                  help="enable auto reconnect")
parser.add_option("-i", "--reconnect-interval", type="float", default=3,
                  help="interval between reconnect attempts")
parser.add_option("-I", "--reconnect-limit", type="int",
                  help="maximum number of reconnect attempts")
parser.add_option("-c", "--count", type="int", default=1,
                  help="stop after count messages have been sent, zero disables (default %default)")
parser.add_option("-t", "--timeout", type="float", default=None,
                  help="exit after the specified time")
parser.add_option("-I", "--id", help="use the supplied id instead of generating one")
parser.add_option("-S", "--subject", help="specify a subject")
parser.add_option("-R", "--reply-to", help="specify reply-to address")
parser.add_option("-P", "--property", dest="properties", action="append", default=[],
                  metavar="NAME=VALUE", help="specify message property")
parser.add_option("-M", "--map", dest="entries", action="append", default=[],
                  metavar="KEY=VALUE",
                  help="specify map entry for message body")
parser.add_option("-v", dest="verbose", action="store_true",
                  help="enable logging")
opts, args = parser.parse_args()
if opts.verbose:
  enable("qpid", DEBUG)
else:
```

```
  enable("qpid", WARN)
if opts.id is None:
  spout_id = str(uuid4())
else:
  spout_id = opts.id
if args:
  addr = args.pop(0)
else:
  parser.error("address is required")
content = None
if args:
  text = " ".join(args)
else:
  text = None
if opts.entries:
  content = {}
  if text:
    content["text"] = text
  for e in opts.entries:
    name, val = nameval(e)
    content[name] = val
else:
  content = text
conn = Connection(opts.broker,
                  reconnect=opts.reconnect,
                  reconnect_interval=opts.reconnect_interval,
                  reconnect_limit=opts.reconnect_limit)
try:
  conn.open()
  ssn = conn.session()
  snd = ssn.sender(addr)
  count = 0
  start = time.time()
  while (opts.count == 0 or count < opts.count) and \
        (opts.timeout is None or time.time() - start < opts.timeout):
    msg = Message(subject=opts.subject,
                  reply_to=opts.reply_to,
                  content=content)
    msg.properties["spout-id"] = "%s:%s" % (spout_id, count)
    for p in opts.properties:
      name, val = nameval(p)
      msg.properties[name] = val
    snd.send(msg)
    count += 1
    print msg
```

```
except SendError, e:
  print e
except KeyboardInterrupt:
  pass
conn.close()
```

客户端的测试代码：



图片 28.3 pic

```
#!/usr/bin/env python

#xiaorui.cc

##转自 ibm 开发社区

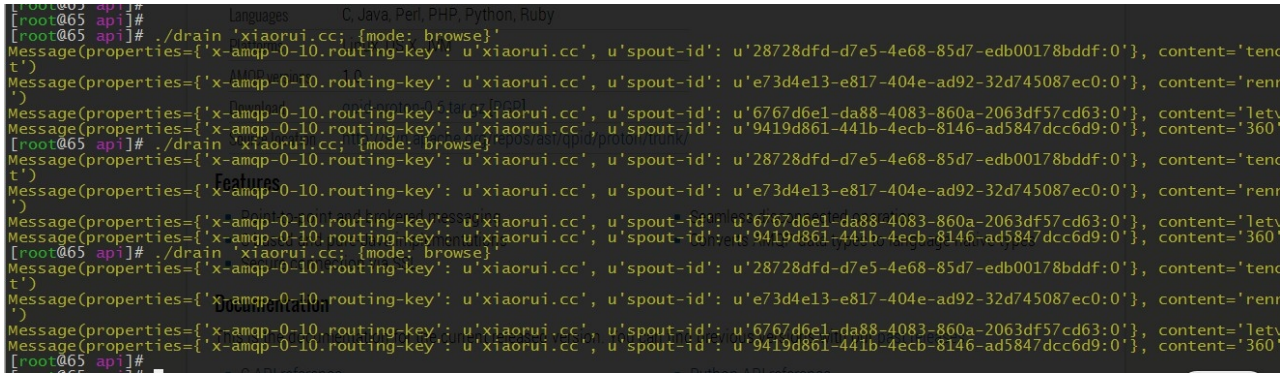
import optparse
from qpid.messaging import *
from qpid.util import URL
from qpid.log import enable, DEBUG, WARN
parser = optparse.OptionParser(usage="usage: %prog [options] ADDRESS ...",
                               description="Drain messages from the supplied address.")
parser.add_option("-b", "--broker", default="localhost",
                  help="connect to specified BROKER (default %default)")
parser.add_option("-c", "--count", type="int",
                  help="number of messages to drain")
parser.add_option("-f", "--forever", action="store_true",
                  help="ignore timeout and wait forever")
parser.add_option("-r", "--reconnect", action="store_true",
                  help="enable auto reconnect")
parser.add_option("-i", "--reconnect-interval", type="float", default=3,
                  help="interval between reconnect attempts")
parser.add_option("-I", "--reconnect-limit", type="int",
                  help="maximum number of reconnect attempts")
parser.add_option("-t", "--timeout", type="float", default=0,
                  help="timeout in seconds to wait before exiting (default %default)")
parser.add_option("-p", "--print", dest="format", default="%(M)s",
                  help="format string for printing messages (default %default)")
parser.add_option("-v", dest="verbose", action="store_true",
```

```
                help="enable logging")
opts, args = parser.parse_args()
if opts.verbose:
  enable("qpid", DEBUG)
else:
  enable("qpid", WARN)
if args:
  addr = args.pop(0)
else:
  parser.error("address is required")
if opts.forever:
  timeout = None
else:
  timeout = opts.timeout
class Formatter:
  def __init__(self, message):
    self.message = message
    self.environ = {"M": self.message,
                    "P": self.message.properties,
                    "C": self.message.content}
  def __getitem__(self, st):
    return eval(st, self.environ)
conn = Connection(opts.broker,
                  reconnect=opts.reconnect,
                  reconnect_interval=opts.reconnect_interval,
                  reconnect_limit=opts.reconnect_limit)
try:
  conn.open()
  ssn = conn.session()
  rcv = ssn.receiver(addr)
  count = 0
  while not opts.count or count < opts.count:
    try:
      msg = rcv.fetch(timeout=timeout)
      print opts.format % Formatter(msg)
      count += 1
      ssn.acknowledge()
    except Empty:
      break
except ReceiverError, e:
  print e
except KeyboardInterrupt:
  pass
conn.close()
```

Browse 模式的意思是，浏览的意思，一个特殊的需求，我访问了一次，别人也能访问。

Consume 模式的意思是，我浏览了一次后，删除这一条。别人就访问不到啦。

这个是浏览模式：



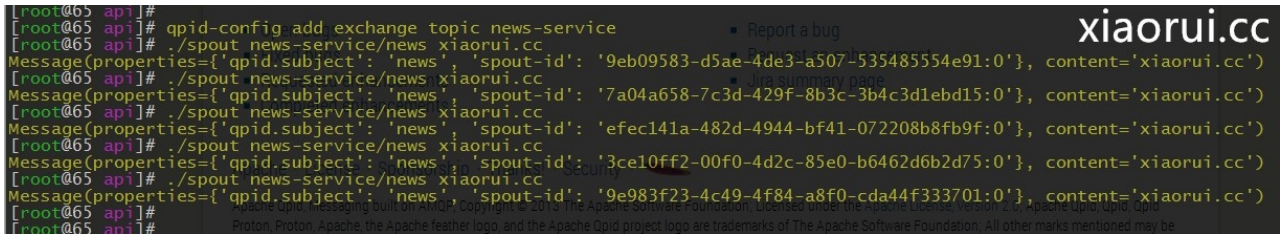
图片 28.4 pic

sub-pub 通信的例子

Pub-sub 是另一种很有用的通信模型。恐怕它的名字就源于出版发行这种现实中的信息传递方式吧，publisher 就是出版商，subscriber 就是订阅者。

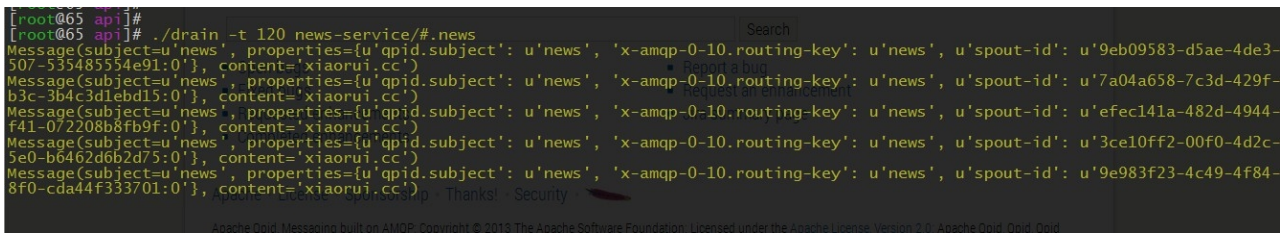
```
服务端
qpid-config add exchange topic news-service
./spout news-service/news xiaorui.cc
客户端:
./drain -t 120 news-service/#.news
```

PUB 端，创建 TOPIC 点!



图片 28.5 pic

SUB端，也就是接收端!



图片 28.6 pic

总结：

qpid 挺好用的，比 rabbitmq 要轻型，比 zeromq 保险点！ 各方面的文档也都很健全，值得一用。话说，这三个消息队列我也都用过，最一开始用的是 redis 的 pubsub 做日志收集和信息通知，后来在做集群相关的项目的时候，我自己搞了一套 zeromq 的分布式任务分发，和 saltstack 很像，当然了远没有万人用的 salt 强大。 rabbitmq 的用法，更是看中他的安全和持久化，当然性能真的不咋地。
关于 qpid 的性能我没有亲自做大量的测试，但是听朋友说，加持久化可以到 7k，不加持久化可以到 1500 左右。

本文出自 “峰云，就她了。” 博客，谢绝转载！

29

python simplejson 模块浅谈

一、背景知识

- JSON:

引用百科描述如下，具体请自行搜索相关介绍：

JSON(JavaScript Object Notation) 是一种轻量级的数据交换格式。它基于 JavaScript（Standard ECMA-262 3rd Edition - December 1999）的一个子集。 JSON 采用完全独立于语言的文本格式，但是也使用了类似于C语言家族的习惯（包括 C, C++, C#, Java, JavaScript, Perl, Python 等）。这些特性使 JSON 成为理想的数据交换语言。易于人阅读和编写，同时也易于机器解析和生成(网络传输速度快)。

表示方法：

- 数据在名称/值对中
- 数据由逗号分隔
- 花括号保存对象
- 方括号保存数组

示例：

```
{"programmers":[
{"firstName":"Brett","lastName":"McLaughlin","email":"aaaa"},
{"firstName":"Jason","lastName":"Hunter","email":"bbbb"},
{"firstName":"Elliotte","lastName":"Harold","email":"cccc"}
],
"authors":[
{"firstName":"Isaac","lastName":"Asimov","genre":"sciencefiction"},
{"firstName":"Tad","lastName":"Williams","genre":"fantasy"},
{"firstName":"Frank","lastName":"Peretti","genre":"christianfiction"}
]}
```

- HOWTO-UNICODE:

unicode 标准描述了字符如何对应编码点(code point),使用 16 进制表示 00 00.
PYTHON 中，basestring 派生了 unicode 类型和 str 类型
unicode 字符串是一个编码点序列，该序列在内存中会被表示成一组字节(0-255)，str 是指 8 字节流。
unicode 字符串可以通过 encode 函数转换为 str；str 可以通过 decode 转换为 unicode。编解码类型一般是 utf-8

示例：

```
>>> u"中国".encode('utf-8')
'\xe4\xb8\xad\xe5\x9b\xbd'    #将 unicode 字符串编码为 str
>>> '\xe4\xb8\xad\xe5\x9b\xbd'.decode('utf-8')
u'\u4e2d\u56fd'              #将 str 解码为 unicode 字符串
```

从文件中读和写入文件的操作都应该是操作的 8 位字节流，如果将 unicode 字符串写入文件，需要进行编码操作；如果从文件中读 unicode 字符串，首先读取出来的是 8 位字节流需要进行解码操作。

一般功能代码中都直接操作 unicode 字符串，而只在写数据或读数据时添加对应的编解码操作。

- 序列化和反序列化

当两个进程在进行远程通信时，彼此可以发送各种类型的数据。无论是何种类型的数据，都会以二进制序列的形式在网络上传送。发送方需要把这个对象转换为字节序列，才能在网络上传送；接收方则需要把字节序列再恢复为对象。

把对象转换为字节序列的过程称为对象的序列化，比如把一个字典对象以某种格式(JSON)写到文件中；把字节序列恢复为对象的过程称为对象的反序列化，比如读取某种格式化(JSON)的文件，构造一个字典对象。

根据 HOWTO-UNICODE 的知识，把网络可以看做是一个文件，发送方写数据到网络时需要进行编码，接收方读取数据时需要进行解码。也就是说**序列化的同时会进行编码，反序列化的同时会进行解码**。

二、simplejson

simplejson 是 json 标准模块的扩展（基础功能相同），是 pypi 提供的拓展模块，需要另行安装。不过可以使用 python 自带的 json 库，基本是相同的使用方法(提供的接口功能基本一致)。在 python 的 library 文档中将 JSON 归为网络数据控制类，很好的说明了他们的用途，主要用于网络数据控制，编解码等。但是也具有其他的用途，比如可以用来作为配置文件的读写模块，简单的文件操作等。

它提供的接口很少，容易掌握，而且大多数情况下会使用默认的参数。官方文档中阐明，默认的接口参数和不进行子类化会有更好的性能体现。下面我们对提供的接口进行讨论，并且仅展示必须参数，其他关键字参数将以**kwargs表示；

- simplejson.dump(obj, fp, **kwargs):将 python 对象写到文件中（以 JSON 格式）
- simplejson.dumps(obj, **kwargs)：将 python 对象表示成字符串(JSON 的格式)
- simplejson.load(fp, **kwargs)：从文件中(包含 JSON 结构)读取为 python 对象
- simplejson.loads(s, **kwargs)：从字符串中(包含 JSON 结构)读取为 python 对象
- class simplejson.JSONDecoder：load/loads 的时候调用，将 JSON 格式序列解码为 python 对象
- class simplejson.JSONEncoder：dump/dumps 的时候调用，将 python 对象编码为 JSON 格式序列

联系到上面的基础知识，我们可以知道，dump 的过程其实就是向文件句柄中写数据，即对象序列化的过程，需要进行编码，只是编码的格式不只是 unicode 和 str 的转换，而是更重要的 python 对象类型和 JSON 对象类型之间的转换。同理，load 的过程其实就是从文件句柄中读数据，即反序列化生成对象的过程，需要进行解码，只是解码的格式不只是 str 和 unicode 的转换，而是更重要的 JSON 对象类型和 python 对象类型之间的转换。

下面是 JSON 对象类型和 Python 对象类型之间的对应关系：

JSON	Python 2	Python 3
object	dict	dict
array	list	list
string	unicode	str
number (int)	int, long	int
number (real)	float	float
true	True	True
false	False	False
null	None	None

下面以一个例子来结束本文，例子中附带注释：

```
 #coding:utf-8

import simplejson as json

 #simplejson.dump(**kwargs)

fp = open('./text.json', 'w+')
json.dump([1,2], fp)        ##将 python 数组进行序列化，保存到文件中
fp.seek(0)
print "----dump----\n", u'使用 dump 将 python 数组对象保存在一个包含 JSON 格式的文件中，文件内容为：\n', fp.read()
print
fp.close()

 #simplejson.dumps(**kwargs)

r_dumps = json.dumps({"中国 obj":[1,2], "obj2":[3,4]})  #将 python 字典进行序列化，保存到字符串中
print "----dumps----\n", u'使用 dumps 将 python 字典对象转换为一个包含 JSON 格式的字符串，字符串结果为：\n', r_dumps
print

 #simplejson.load(**kwargs)

 #如果 json 文档格式有错误，将会抛出 JSONDecoderError 异常

fp = open('./text.json', 'r')
```

```
r_load = json.load(fp)          #将文件中的内容转换为 python 对象
print "----load----\n", u"使用 load 读取一个包含 JSON 数组格式的文件后，得到一个 python 对象，类型是：", type(r_load)
print
 #simplejson.loads(**kwargs)

 #如果 json 文档格式有错误，将会抛出 JSONDecoderError 异常



 #将字符串中的内容转换为一个 python 对象

r_loads = json.loads('''{"programmers":[
{"firstName":"Brett","lastName":"McLaughlin","email":"aaaa"},
{"firstName":"Jason","lastName":"Hunter","email":"bbbb"},
{"firstName":"Elliotte","lastName":"Harold","email":"cccc"}
],
"authors":[
{"firstName":"Isaac","lastName":"Asimov","genre":"sciencefiction"},
{"firstName":"Tad","lastName":"Williams","genre":"fantasy"},
{"firstName":"Frank","lastName":"Peretti","genre":"christianfiction"}
]}''')
print "----loads----\n", u"使用 loads 读取一个包含 JSON 字典格式的字符串后，得到一个 python 对象，类型是：", type(r_loa
print
```

运行之后的结果显示：

```
----dump----
使用 dump 将 python 数组对象保存在一个包含 JSON 格式的文件中，文件内容为：
[1, 2]
----dumps----
使用 dumps 将 python 字典对象转换为一个包含 JSON 格式的字符串，字符串结果为：
{"obj2": [3, 4], "\u4e2d\u56fdobj": [1, 2]}
----load----
使用 load 读取一个包含 JSON 数组格式的文件后，得到一个 python 对象，类型是： <type 'list'>
----loads----
使用 loads 读取一个包含 JSON 字典格式的字符串后，得到一个 python 对象，类型是： <type 'dict'>
```

30

python Howto 之 logging 模块

本文来源于对 py2.7.9 docs 中 howto-logging 部分加之源代码的理解。官方文档链接如下，我用的是下载的 pdf 版本，应该是一致的：https://docs.python.org/2/howto/logging.html

我们不按照文档上由浅入深的讲解顺序，因为就这么点东西不至于有“入”这个动作。

使用 logging 模块记录日志涉及四个主要类，使用官方文档中的概括最为合适：

logger提供了应用程序可以直接使用的接口；
handler将(logger创建的)日志记录发送到合适的目的输出；
filter提供了细度设备来决定输出哪条日志记录；
formatter决定日志记录的最终输出格式。

写 log 的一般顺序为：

一、创建logger：

我们不要通过 logging.Logger 来直接实例化得到 logger，而是需要通过 logging.getLogger("name")来生成 logger 对象。

不是说我们不能实现 Logger 的实例化，而是我们期待的是同一个 name 得到的是同一个 logger，这样多模块之间可以共同使用同一个 logger，getLogger 正是这样的解决方案，它内部使用 loggerDict 字典来维护，可以保证相同的名字作为 key 会得到同一个 logger 对象。我们可以通过实例来验证一下：

```
 #test_logger1.py

 #coding:utf-8


import logging
print logging.getLogger("mydear")
import test_logger2
test_logger2.run()          #调用文件 2 中的函数，保证两个模块共同处于生存期

 #test_logger2.py

 #coding:utf-8


import logging
def run():
   print logging.getLogger("mydear")
```

输出： <logging.Logger object at 0x00000000020ECF28>
<logging.Logger object at 0x00000000020ECF28>

结果表明两个文件中通过"mydear"调用 getLogger 可以保证得到的 logger 对象是同一个。而分别进行 Logger 类的实例化则不能保证。

有了 logger 之后就可以配置这个 logger，例如设置日志级别 setLevel,绑定控制器 addHandler，添加过滤器 addFilter 等。

配置完成后，就可以调用 logger 的方法写日志了，根据 5 个日志级别对应有 5 个日志记录方法，分别为logger.debug，logger.info，logger.warning，logger.error，logger.critical。

二、配置 Logger 对象的日志级别：

logger.setLevel(logging.DEBUG) #DEBUG 以上的日志级别会被此 logger 处理

三、创建 handler 对象

handler 负责将 log 分发到某个目的输出，存在多种内置的 Handler 将 log 分发到不同的目的地，或是控制台，或是文件，或是某种形式的 stream，或是 socket 等。一个 logger 可以绑定多个 handler，例如，一条日志可以同时输出到控制台和文件中。

以 FileHandler 和 StreamHandler 为例：

logfile= logging.FileHandler("./log.txt") #创建一个handler，用于将日志输出到文件中
console = logging.StreamHandler() #创建另一个handler，将日志导向流

handler 对象也需要设置日志级别，由于一个 logger 可以包含多个 handler，所以每个 handler 设置日志级别是有必要的。用通俗的话讲，比如，我们需要处理 debug 以上级别的消息，所以我们将 logger 的日志级别定为 DEBUG；然后我们想把 error 以上的日志输出到控制台，而 DEBUG 以上的消息输出到文件中，这种分流就需要两个 Handler 来控制。

```
logfile.setLevel(logging.DEBUG)
console.setLevel(logging.ERROR)
```

除了对 handler 对象设置日志级别外，还可以指定 formatter，即日志的输出格式。对 handler 对象设置日志格式，说明了可以将一条记录以不同的格式输出到控制台，文件或其他目的地。

```
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')

logfile.setFormatter(formatter) #设置 handler 的日志输出格式
```

formatter 创建时使用的关键字，最后会以列表的形式展现，这不是重点。

四、绑定 handler 到 logger 中

至此 handlers 和 logger 已经准备好了，下面我们就将 handlers 绑定到 logger 上，一个 logger 对象可以绑定多个 handler。

```
logger.addHandler(logfile) #logger 是通过 getLogger 得到的 Logger 对象
logger.addHandler(console)
```

五、使用 logger 真正写日志

```
logger.debug("some debug message.")
logger.info("some info message.")
```

看上去，中间步骤(创建 handler，设置日志级别，设置输出格式等)更像是配置 Logger，一旦配置完成则直接调用写日志的接口即可，稍后这些日志将按照先前的配置输出。

呜呼，好多内容啊，来点简单的吧.

下面的代码，是最简单的。导入 logging 之后就进行了写日志操作：

```
#coding:utf-8


import logging
logging.debug("debug mes")
logging.info("info mes")
logging.warning("warn mes")
```

控制台输出如下：

```
WARNING:root:warn mes
```

咦？发生了什么情况，为什么只输出了 warning？handler、logger、formatter 去哪儿了？

-_-!说好的最简单的呢？为了让自己讲信用，我尽可能把它解释成“最简单的”。

知识点 1：logger 间存在继承关系

logger 通过名字来决定继承关系，如果一个 logger 的名字是 "mydest"，另一个 logger 的名字是"mydest.dest1"（ getLogger("mydest.dest1") ）,那么就称后者是前者的子 logger，会继承前者的配置。上面的代码没有指定 logger，直接调用 logging.debug 等方法时，会使用所有 logger 的祖先类 RootLogger。

从上面的代码运行结果可以猜测出，该 RootLogger 设置的日志级别是 logging.WARN，输出目的地是标准流。从源码可以更清楚的看出来：

```
root = RootLogger(WARNING) #设置 WARNING 的级别
```

至于 rootLogger 的输出目的地的配置，我们跟踪 logging.debug 的源代码来看一下：

```
def debug(msg, *args, **kwargs): """ Log a message with severity 'DEBUG' on the root logger. """ if le
n(root.handlers) == 0: basicConfig() root.debug(msg, *args, **kwargs)
```

大约可以看到，如果 rootLogger 没有配置 handler，就会不带参数运行 basicConfig 函数(*请看知识点 2)，我们看一下 basicConfig

```
def basicConfig(**kwargs): _acquireLock() try: if len(root.handlers) == 0: filename = kwargs.get("filen
ame") if filename: mode = kwargs.get("filemode", 'a') hdlr = FileHandler(filename, mode) else: stream
= kwargs.get("stream") hdlr = StreamHandler(stream) fs = kwargs.get("format", BASIC_FORMAT) df
s = kwargs.get("datefmt", None) fmt = Formatter(fs, dfs) hdlr.setFormatter(fmt) root.addHandler(hdlr)
level = kwargs.get("level") if level is not None: root.setLevel(level) finally: _releaseLock()
```

因为参数为空，所以我们就看出了，该 rootLoger 使用了不带参数的 StreamHandler，也可以看到诸如 format 之类的默认配置。之

```
class StreamHandler(Handler): """ A handler class which writes logging records, appropriately formatt
ed, to a stream. Note that this class does not close the stream, as sys.stdout or sys.stderr may be us
ed. """
```

```
def __init__(self, stream=None):
    """
    Initialize the handler.

    If stream is not specified, sys.stderr is used.
    """
    Handler.__init__(self)
    if stream is None:
        stream = sys.stderr  ####
    self.stream = stream
```

不带参数的StreamHandler将会把日志流定位到sys.stderr流，标准错误流同样会输出到控制台

知识点 2：basicConfig 函数用来配置 RootLogger

basicConfig 函数仅用来配置 RootLogger，rootLogger 是所有 Logger 的祖先 Logger，所以其他一切 Logger 会继承该 Logger 的

从上面的 basicConfig 源码看，它可以有六个关键字参数，分别为：

filename：执行使用该文件名为 rootLogger 创建 FileHandler，而不是 StreamHandler

filemode：指定文件打开方式，默认是"a"

stream：指定一个流来初始化 StreamHandler。此参数不能和 filename 共存，如果同时提供了这两个参数，则 stream 参数被忽略

format：为 rootLogger 的 handler 指定输出格式

datefmt：指定输出的日期时间格式

level：设置 rootLogger 的日志级别

使用样例：

logging.basicConfig(filename = './log.txt', filemode = 'a', #stream = sys.stdout, format = '%(levelnam e)s:%(message)s', datefmt = '%m/%d/%Y %I:%M:%S', level = logging.DEBUG)

知识点 3 通过示例详细讨论 Logger 配置的继承关系

首先准备下继承条件：log2 继承自 log1，logger 的名称可以随意，要注意‘.’表示的继承关系。

#coding:utf-8

import logging log1 = logging.getLogger("mydear") log1.setLevel(logging.WARNING) log1.addHandle r(StreamHandler()) log2 = logging.getLogger("mydear.app") log2.error("display") log2.info("not displa y")

level 的继承

原则：子 logger 写日志时，优先使用本身设置了的 level；如果没有设置，则逐层向上级父 logger 查询，直到查询到为止。最极端的

从源代码中看更为清晰， 感谢 python 的所见即所得：

def getEffectiveLevel(self): """ Get the effective level for this logger.

```
    Loop through this logger and its parents in the logger hierarchy,
    looking for a non-zero logging level. Return the first one found.
    """
    logger = self
    while logger:
        if logger.level:
            return logger.level
        logger = logger.parent
    return NOTSET
```

handler 的继承

原则：**先将日志对象传递给子 logger 的所有 handler 处理，处理完毕后，如果该子 logger 的 propagate 属性没有设置为 0，则将日**

在上面实例代码的基础上，我们再添加一句代码，即：

```
#coding:utf-8

import logging log1 = logging.getLogger("mydear") log1.setLevel(logging.WARNING) log1.addHandle
r(StreamHandler()) log2 = logging.getLogger("mydear.app") log2.error("display") log2.info("not displa
y") print log2.handlers #打印log2绑定的handler
```

输出如下：

```
display []
```

说好的继承，但是子 logger 竟然没有绑定父类的 handler，what's wrong?

看到下面调用 handler 的源代码，就真相大白了。可以理解成，这不是真正的(类)继承，只是"行为上的继承":

```
def callHandlers(self, record): """ Pass a record to all relevant handlers.

    Loop through all handlers for this logger and its parents in the
    logger hierarchy. If no handler was found, output a one-off error
    message to sys.stderr. Stop searching up the hierarchy whenever a
    logger with the "propagate" attribute set to zero is found - that
    will be the last logger whose handlers are called.
    """
    c = self
    found = 0
    while c:
        for hdlr in c.handlers:         #首先遍历子 logger 的所有 handler
            found = found + 1
            if record.levelno >= hdlr.level:
                hdlr.handle(record)
        if not c.propagate:             #如果 logger 的 propagate 属性设置为 0，停止
            c = None    #break out
        else:                           #否则使用直接父 logger
            c = c.parent
    ...
```

额，最简单的样例牵引出来这么多后台的逻辑，不过我们懂一下也是有好处的。

下面，我们将一些零碎的不是很重要的东西罗列一下，这篇就结束了。

1. 几种 LogLevel 是全局变量，以整数形式表示，也可以但是不推荐自定义日志级别，如果需要将 level 设置为用户配置，则获取 leve

#假设 loglevel 代表用户设置的 level 内容

numeric_level = getattr(logging, loglevel.upper(), None) if not isinstance(numeric_level, int): raise Val ueError('Invalid log level: %s' % loglevel) logging.basicConfig(level=numeric_level, ...)

2. format 格式，用于创建 formatter 对象，或者 basicConfig 中，就不翻译了

%(name)s Name of the logger (logging channel) %(levelno)s Numeric logging level for the message (DEBUG, INFO, WARNING, ERROR, CRITICAL) %(levelname)s Text logging level for the message ("DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL") %(pathname)s Full pathname of the source f ile where the logging call was issued (if available) %(filename)s Filename portion of pathname %(mod ule)s Module (name portion of filename) %(lineno)d Source line number where the logging call was is sued (if available) %(funcName)s Function name %(created)f Time when the LogRecord was created (time.time() return value) %(asctime)s Textual time when the LogRecord was created %(msecs)d Mill isecond portion of the creation time %(relativeCreated)d Time in milliseconds when the LogRecord w as created, relative to the time the logging module was loaded (typically at application startup time) %(thread)d Thread ID (if available) %(threadName)s Thread name (if available) %(process)d Proces s ID (if available) %(message)s The result of record.getMessage(), computed just as the record is em itted

3. 写日志接口

logging.warn("%s am a hero", "I") #1 %格式以参数形式提供实参 logging.warn("%s am a hero" % ("I",)) #2 直接提供字符串，也可以使用format，template logging.warn("%(name)s am a hero", {'name':"I"}) #关键字参数
logging.warn("%(name)s am a hero" % {'name':"I"}) #甚至这样也可以 logging.warn("%(name)s am a he ro, %(value)s" % {'name':"I", 'value':'Yes'}) #原来%也能解析关键字参数，不一定非是元组 如果关键字和位置参数混用呢，%应该不会有什么作为了，最强也就能这样： logging.warn("%(name)s am a hero, %()s" % {'name':"I" ,": 'Yes'})#也是字典格式化的原理

4. 配置 logging：

上面已经讲了如果配置 handler，绑定到 logger。如果需要一个稍微庞大的日志系统，可以想象，我们会使用好多的 addHandler，S

方式一：使用配置文件

import logging import logging.config logging.config.fileConfig('logging.conf') # create logger

logger = logging.getLogger('simpleExample') # 'application' code

logger.debug('debug message') logger.info('info message') logger.warn('warn message') logger.error('error message') logger.critical('critical message')

#配置文件logging.conf的内容

[loggers] keys=root,simpleExample [handlers] keys=consoleHandler [formatters] keys=simpleFormatter [logger_root] level=DEBUG handlers=consoleHandler [logger_simpleExample] level=DEBUG handlers=consoleHandler qualname=simpleExample propagate=0 [handler_consoleHandler] class=StreamHandler level=DEBUG formatter=simpleFormatter args=(sys.stdout,) [formatter_simpleFormatter] format=%(asctime)s - %(name)s - %(levelname)s - %(message)s datefmt= ```

方式二：使用字典

请参阅 python2.7.9 Library 文档，链接:

https://docs.python.org/2/library/logging.config.html?highlight=dictconfig#configuration-dictionary-schema

1. 众多的 handler 满足不同的输出需要

StreamHandler，FileHandler，NullHandler，RotatingFileHandler，TimedRotatingFileHandler，SocketHandler，DatagramHandler，SMTPHandler，SysLogHandler，NTEventLogHandler，MemoryHandler，HTTPHandler，WatchedFileHandler，

其中前三种在 logging 模块中给出，其他的在 logging.handlers 模块中给出。

31

Python FAQ3-python 中 的原始(raw)字符串

本篇源自 py2.7.9-docs 的 faq.pdf 中的“3.23 Why can’t raw strings (r-strings) end with a backslash?”

更准确的说，原始字符串即以r修饰的字符串，不能以奇数个反斜杠结束；

原始字符串被设计用来作为一些处理器(主要是正则表达式引擎)的输入。这种处理器会认为这种未匹配的末端反斜杠是种错误，所以，原始字符串也就不允许以奇数个反斜杠结束。反过来，他们允许你使用斜杠来表示转义，包括\"表示"，\t 表示 TAB 等。当原始字符串用于这些处理器时，这个规则适用。

如果原始字符串不用于正则表达式等处理器，只是简单的代表一个字符串，那么该串中的 \ 就是 \,而不再具有转义的含义，这就是所谓的‘原始’。

下面我会一步步的解释字符串和原始字符串的区别

1.用于单独的字符串表示:

简单字符串中存在 \ 转义行为，而原始字符串中 \n 就是 \n 字符

```
>>> s = "i have\na dream"
>>> r = r'i have\na dream'
>>> print s
i have
a dream
>>> print r
i have\na dream
```

2.原始字符串用于正则表达式中

我们使用 windows 路径来做例子介绍原始字符串的一次转义

```
>>> path = r"\this\is\a\path\"
  File "<stdin>", line 1
    path = r"\this\is\a\path\"    #原始字符串不允许单数个\结尾，不管是用于正则还是普通字串
                              ^
SyntaxError: EOL while scanning string literal
>>> path = r"\this\is\a\path\ "[:-1]
>>> path
'\\this\\is\\a\\path\\'       #定义了一个待匹配的字符串
>>> reg1 = r'\\this\\is\\a\\path\\' #定义了自然字符串表示的正则表达式
>>> import re
>>> g = re.match(reg1, path)   #使用自然字符串进行匹配
>>> print g.group()
\this\is\a\path\              #匹配到了结果，表示真实的\字符可以被自然字符串以\\匹配上
>>>                          #\\转义的结果就是\
```

3.简单字符串用于正则表达式中

让我们使用上面的 path 变量来制作简单字符串用来匹配的例子

```
>>> reg2 = '\\this\\is\\a\\path\\'
>>> g = re.match(reg2, path)        #竟然报异常了，根据异常的意思是行尾是虚假的转义
Traceback (most recent call last):  #下面我们再探究原因，先把行尾的\\去掉，再次进行匹配
  File "<stdin>", line 1, in <module>
  File "D:\Python27\lib\re.py", line 137, in match
    return _compile(pattern, flags).match(string)
  File "D:\Python27\lib\re.py", line 244, in _compile
    raise error, v # invalid expression
sre_constants.error: bogus escape (end of line)

>>> reg2 = '\\this\\is\\a\\path'
>>> g = re.match(reg, path)        #按照原始字符串的理解，这里应该可以匹配上的，但是没有
>>> print g.group()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'NoneType' object has no attribute 'group'
```

为什么会出现差异，又为什么到处都建议正则匹配时要使用r'字符串'呢?

让我们分析下原始字符串和简单字符串的区别：简单字符串如果想输出‘\’,需要进行转义即'\'才会输出一个'\'；那原始字符串想要输出'\'，则直接写即可'\'。

这里有些乱，我觉得主要在于 str、repr 在捣乱：

```
>>> print path                    #这里调用str，人们习惯的显示方式
\this\is\a\path\
>>> path                          #这里调用repr，真实的显示方式(比str的显示仅多了一层转义)
'\\this\\is\\a\\path\\'
```

让我们全部将真实的显示方式当做参照物，即

path 的真实显示是：'\this\is\a\path\'
简单字符串的正则表达式 reg2 的真实显示是：'\this\is\a\path'
原始字符串的正则表达式 reg1 的真实显示是：'\\this\\is\\a\\path\\'
从真实的显示来看匹配就容易理解的多了，而且没有了原始和简单字符串之分，都看做是正则引擎应用的串。从上面可以看出 reg2中\ 只能匹配\，而 path 中是 \，需要像 reg1 中的 \\ 来进行匹配。

追根溯源向来比较绕，还是简单记住使用规则，匹配路径 \ 字符，需要普通字符串输入 4 个斜杠(\\)匹配上，而原始字符串仅需要 2 个斜杠(\)即可匹配上。这也是鼓励使用原始字符串进行正则匹配的原因。

极客学院

jikexueyuan.com

中国最大的IT职业在线教育平台



更多信息请访问

http://wiki.jikexueyuan.com/project/python-actual-combat/